



DEPARTMENT OF COMPUTER SCIENCE

MASTER'S THESIS

INTENT-ORIENTED QUERY REWRITING FOR DATA ANALYTICS

Author:
Gianna Lisa Nicolai

submitted on:
March 18, 2025

Supervisor
Prof. Dr.-Ing. Sebastian Michel

Reviewers
Prof. Dr.-Ing. Sebastian Michel
M. Sc. Patrick Hansert

Abstract

Traditional SQL query rewriting has long been a highly researched topic in database systems. It aims to improve query performance while keeping the same results by using rules or heuristics to change the query. In contrast to this is intent-oriented query rewriting. It comes into play when the data for the query is not available or accessible. Unlike traditional rewriting, it tries to rewrite the query such that the information insight is kept the same for a human, without necessarily keeping any kind of structure or semantics of the original query. This way, available data can be used to give the same insight as a query on unavailable or inaccessible data. Moreover, if some data is only available with a high latency, intent-oriented query rewriting can even speed up the query execution. These features make intent-oriented query rewriting a promising addition for data analytics applications. With recent advances in using Large Language Models for database applications like Text-to-SQL, we implement a prototype for an intent-oriented rewriting system for SQL queries.

Zusammenfassung

Das Umformulieren von SQL Anfragen ist ein populäres Forschungsgebiet in den Datenbanksystemen. Das Ziel hierbei ist, die Performanz der Anfrage zu verbessern, während das Ergebnis der Anfrage nicht verändert wird. Hierfür werden häufig Äquivalenzregeln oder Heuristiken verwendet. Im Gegensatz dazu steht das absichtsorientierte Umschreiben von Anfragen, welches zum Tragen kommt, wenn die Daten für eine Anfrage nicht in der Datenbank vorhanden sind. Das Ziel hierbei ist, die Anfrage so umzuformulieren, dass ein Mensch die gleiche Information aus dem Ergebnis der Anfrage ziehen kann wie bei der originalen Anfrage. Hierfür ist es nicht notwendig, dass die Struktur oder Semantik der Anfrage beibehalten werden oder dass das Ergebnis gleich bleibt. Dadurch ist es möglich, vorhandene Daten zu nutzen, um Einblicke zu erlangen, welche man sonst nur durch nicht zugängliche Daten bekommen könnte. Das macht absichtsorientiertes Umschreiben von Anfragen zu einer vielversprechenden Ergänzung für Datenanalyse. Mit den Fortschritten bei der Nutzung großer KI-Sprachmodelle für verschiedene Anwendungen wie Text-to-SQL, implementieren wir einen Prototypen für ein System zum absichtsorientierten Umschreiben von SQL-Anfragen.

Contents

1	Introduction	1
1.1	Problem Statement	3
1.2	Outline	3
2	Background	5
2.1	Large Language Models	5
2.1.1	Problems and Challenges	5
2.1.2	Existing Models	6
2.1.3	OpenAI Models	7
2.1.4	Prompting LLMs	7
2.2	Semantic Search	8
2.2.1	Embeddings	8
3	Related Work	9
3.1	Query Rewriting	9
3.2	NL2SQL	10
3.3	SQL Query Similarity	11
4	Approach	13
4.1	Architecture	13
4.2	Pre-Check Executability	16
4.3	Table Filtering	16
4.3.1	Embedding Filter	17
4.3.2	Simple LLM Filter	19
4.3.3	Complex LLM Filter	20
4.3.4	Table Summaries	22
4.3.5	Database Prefixes	26
4.4	Query Rewriting	28
4.4.1	Simple Rewriting	29
4.4.2	Rewriting via Natural Language	30
4.5	Prune and Rank	30
4.5.1	Pruning Algorithm	30
4.5.2	Ranking Algorithms	31
4.5.3	Similarity Measures	35
4.5.4	Combining Ranking Algorithms and Similarity Measures	39
4.6	Query Correction and Execution	40
5	Evaluation	41
5.1	Setup	41
5.2	Metrics Results	43
5.2.1	Query Execution Check	43
5.2.2	Table Filtering	44
5.2.3	Query Rewriting	49
5.2.4	Prune and Rank	52
5.2.5	Query Correction and Execution	57
5.2.6	Metrics Summary	60
5.3	User Study	60

5.3.1	Rewriting Evaluation	62
5.3.2	Post-processing Evaluation	65
6	Conclusion and Outlook	70
A	LLM Prompts	79
A.1	Simple Table Prefilter	79
A.2	Complex Table Prefilter	79
A.3	Table Prefilter Summaries	80
A.4	Finding Metadata	80
A.5	Simple Rewriting	81
A.6	Complex Rewriting	82
A.7	Ranking	82
A.8	Similarity Function	83
A.9	Self Correction	83
B	User Study Instructions	84
C	User Study Queries	85

List of Figures

1.1	Intent-oriented Rewrites of the User Query	2
4.1	Workflow of the Rewriting System	14
4.2	Global Architecture	15
4.3	Hallucinated Query due to too many Tables	17
4.4	Example Output of the LLM in the Complex Table Filter	21
4.5	Example Summary, Topics and Keywords of the LLM for one Table	24
4.6	Example Summary, Topics and Keywords of the LLM for one Query	25
5.1	Latency of the API Compared to Output Tokens	43
5.2	Runtimes of Table Filters	45
5.3	Tokens and Costs of Table Filters	46
5.4	Tables found in Average per Query and Filter	47
5.5	Tables found in Average per Query by Filter 4	48
5.6	Runtimes of Query Rewriters	49
5.7	Tokens and Cost of Query Rewriting Approaches	50
5.8	Runtimes of Ranking Combinations	53
5.9	Tokens and Costs of Query Ranking	55
5.10	Dependencies on the Number of DBSCAN Clusters	57
5.11	Runtimes of Execution and Correction	58
5.12	Tokens and Cost of Query Correction	59
5.13	User Study Evaluation per Rewriting Approach	63
5.14	User Study Evaluation per Input Query	65
5.15	Recall of the Ranking Results per Approach	66
5.16	Precision of the First 15 Ranking Results per Approach	67
5.17	Homogeneity of the First 15 Ranking Results per Approach	68

List of Algorithms

1	Embedding Table Filter Algorithm	18
2	Similarity Calculation via Table Similarity	36

List of Tables

4.1	Combinations for the Ranking Component	39
5.1	Amount of Corrected Queries after x Correction Iterations	60
5.2	Configurations for the Rewriter Evaluation	62

List of Listings

1.1	Query posed by a User	1
4.1	Example of a Prunable Query	31
4.2	Example Queries for SQL Similarity Calculation	38

Chapter 1

Introduction

Query rewriting is a popular topic in the context of database systems. It traditionally aims to optimize a query regarding its execution time and complexity, whether it is via rule-based optimization or—like in more recent research—via machine learning. Independent of the method, the aim is always the same: to find another query—using the same data—that has a better performance while still resulting in the same output.

But what if the data the query operates on is not available to the user? Despite the growing amount of digitally recorded data, e.g., from companies and organizations, most of this data is not available to the average person. Often this data, such as usage or personal data is stored in company-owned data stores. However, there are several different platforms that provide datasets free of charge, e.g., Kaggle [40], or that provide the possibility to search for datasets, like Google Dataset Search [33]. Via the usage of such platforms, users might be able to acquire different datasets.

This provides the case for intent-based query rewriting: There is a query that is executed on data that the user cannot access due to, e.g., missing access privileges or completely missing data. However, there might be other data that is available to the user. Thus, we want to rewrite the original query, such that it can operate on the available data while still giving the user the same insights with its results. This would allow the user to still analyze the data and answer the query’s underlying question.

In contrast to traditional query rewriting, the new query is not required to be structurally or even semantically equivalent to the original query, i.e., it does not have to return the same result as the original query. There is only one requirement: The information need the user had when posing the original query is still satisfied by the rewritten query. Let’s say a user posts the query seen in Listing 1.1. The goal the user has in mind is to find out which district of a city is the most attractive one, maybe he wants to move to the city and check out where he should look for apartments. With this query, he assumes that this data exists in a table. However, the user realizes that this is not the case. Here, intent-oriented rewriting comes into play.

```
1  SELECT district, attractiveness_rating
2  FROM city
3  ORDER BY attractiveness DESC;
```

Listing 1.1: Query posed by a User

When looking at the written SQL query as a human, one can deduce multiple possibilities on how to use related data about the city to get the same insight and thus satisfy the information need as the original query results would. In this case, the attractiveness of a district can be related to multiple aspects: crime rate, average income, average rent, amount of schools, number of parks,


```

1  SELECT district, rate FROM city_crimerate ORDER BY rate ASC;

1  SELECT district, count(*) AS amount FROM leisure_facilities
2  GROUP BY district ORDER BY amount DESC;

1  WITH locations AS(
2      SELECT ID, district FROM parks
3      UNION
4      SELECT ID, district FROM schools
5      UNION
6      SELECT ID, district FROM public transport
7  ),
8  countLocations AS (
9      SELECT district, count(*) AS location_number
10     FROM locations GROUP BY district
11 ),
12 averageRent AS (
13     SELECT district, avg(rent) AS avgRent
14     FROM city_rent GROUP BY district
15 )
16 SELECT district, location_number, avgRent
17 FROM countLocations c JOIN averageRent a ON c.district = a.district
18 ORDER BY location_number DESC, avgRent DESC;

```

Figure 1.1: Intent-oriented Rewrites of the User Query

public transport connections, amount of leisure facilities or offers, number of restaurants or bars, cleanliness, available supermarkets, and much more. Some of this information might be publicly accessible. So, depending on the available data different possible intent-oriented rewrites can be found, as seen in Figure 1.1. While the first query uses only the existing crime rate, the second counts leisure facilities in the district. The third query is more complicated, looking at parks, schools, public transport, and rent in a district.

As a human—with some understanding of SQL—one can see that all of the three rewritten queries would give us at least some insight into the attractiveness of different city districts. Thus, they still satisfy the same information need as the original query. From this example, we can deduce a few challenges and insights for intent-oriented rewriting.

As already explained above, there is no need for the rewritten queries to have the same structure or semantics as the original query, which can be seen in the second and third example. Moreover, all of the rewrites in our example will give a different result than the anticipated result of the original query. Some will give aggregated values, while others show existent numbers from other tables. This must not be the case for all intent-oriented rewrites, but it is a possibility. Another aspect of intent-oriented rewriting is that there are multiple different options to rewrite a query, depending on the available data. These rewrites can then vary in their complexity, just like the rewrites in Figure 1.1 show. While the first query is just a simple selection, the last query combines multiple tables to evaluate attractiveness. Thus, intent-oriented rewriting can, but must

not result in a unique rewrite of a certain complexity. This observation is also supported by the insight that the user’s intent might not be clear from the posed query and that different users may have different opinions on how things are related to each other. In our example, one person could say that a lot of bars make a district more attractive, while others could support the opinion that too many bars make it unattractive again. So, depending on a user’s opinion and intent different—and possibly multiple—rewrites might fit the original query.

As we can see there exist different challenges regarding intent-oriented rewriting, including the possible ambiguity and the potentially totally different structure of rewrites. Our goal in this thesis is to automate the rewriting process and solve some of the challenges we are faced with.

1.1 Problem Statement

In this thesis we are faced with the following problem: Given an SQL query q and a database D with tables $t_1, \dots, t_n$. We want to find multiple SQL queries $Q = \{q_1, \dots, q_m\}$ with the following properties:

- They are executable using the tables $t_1, \dots, t_n$.
- They have the same intent as q , i.e., their result satisfies the same information need as the expected result from q .

The queries should be ranked according to the following principles:

- The more similar the intents of $q_i \in Q$ and q are, the better.
- The query ranking uses the principle of novelty and diversity, i.e., if a query $q_i \in Q$ is ranked at the top, a query $q_j \in Q$ that is structurally and semantically similar to q_i should get a penalty on its ranking.

The goal is to produce rewrites that can still be used for data analysis in absence of the original data, thus allowing interesting insights despite missing data. There is still one thing that is hard to grasp mathematically: intent. A query’s intent is a combination of many different aspects, e.g., the topic, the wanted result, or the given conditions. Generally speaking, it is the user’s idea behind the query. Thus, we break human intent down to the information need of a query, i.e., what information the user wants to acquire with this query. To be able to somehow interpret what a user wants to achieve with a query and what information a certain table can give us, the tables and columns in the query q and the database D have to be named in a meaningful and descriptive way, which we assume to be the case.

1.2 Outline

In Chapter 2, we will have a look at Large Language Models (LLMs) in general and what possible problems can arise when working with them. We will also have a look at existing LLMs. Moreover, we will talk about semantic search, especially embeddings and SQL query similarity. Then we will take a brief look at currently ongoing research in related fields in Chapter 3. This includes query rewriting in general and the processing of natural language into SQL. We then

come to the description and explanation of our implemented rewriting system in Chapter 4. Here, we will first give an overview of the different components, before diving into the details of each single one. For each component, we will give an explanation of the different approaches implemented in this thesis. Chapter 5 consists of the evaluation of our rewrite system. Here, we used different methods to evaluate multiple aspects of the systems, one of those methods being a user study. We then conclude this thesis in Chapter 6, also mentioning some possibilities for future work.

Chapter 2

Background

2.1 Large Language Models

Large Language Models (LLMs) are a highly researched topic in machine learning. Their history is shortly explained by Wang et al. [85]. Generally, the task of language models (LMs) is to calculate the probability that a certain word will appear as the next word in a given text. Using this probability, it is possible to predict the next word or phrase for this sentence. LLMs follow the development of Pre-trained language models (PLMs), both using neural networks to be able to generate sentences. The main difference between LLMs and PLMs is the vast difference in the amount of parameters and training data. This, together with the way they are trained allows LLMs to generalize and adapt to different domains, thus outperforming their predecessor.

Due to their generative nature and adaptability, LLMs seem to be able to answer a lot of questions and complete different reasoning tasks (as observed in, e.g., chatbots). They do this without really knowing the answer. Instead, they use their deep neural network to predict which word is most likely to follow the current words [79]. Thus, they can be used in a lot of different fields, e.g., in medicine, finance, and also in different fields of computer science. One such field is information retrieval, as surveyed by Zhu et al. [93]. They are also applicable for different tasks concerning database systems, e.g., processing of natural language into SQL, or query rewriting, which we will see in Chapter 3. This suggests that LLMs, with their high adaptability and reasoning skills, are also applicable in our use case, intent-oriented query rewriting.

2.1.1 Problems and Challenges

Being very complex and advanced models, LLMs do not only come with many application and research opportunities but also with some challenges and problems. Some of the main challenges are summarized in a survey by Naveed et al. [59]. We will mention the most important ones in the following paragraphs. There are a few general concerns when using LLMs. Those involve the problem of safety and privacy. It can happen, whether deliberately induced or not, that LLMs generate harmful or inappropriate content. Moreover, it is possible for LLMs to hallucinate, a phenomenon also surveyed by Huang et al. [36]. Here, the LLM includes something in their response that might sound right but is not correct. This hallucination can conflict with the input from the user, the content the LLM created earlier, or plain facts. This is still a problem in LLMs today. Using LLMs in an automated context thus poses challenges regarding the reliability and safety of the output. Another concern regards privacy in LLMs. One aspect is that LLMs sometimes leak personal information in their answers. Another aspect—which is especially relevant in our use case—is that the LLM needs input to generate a response. We thus have to give the LLM some input, e.g., a database schema in our case. Depending on the used LLM

this information could be used further, e.g., for training, raising the question of privacy of our own data—especially if it is classified. Another ethical problem with the answers from LLMs is the bias they can have. If their training data contains some kind of social bias, this bias will be learned by the LLM and can therefore influence the output. This can lead to problems regarding ethical concerns, raising the question if LLMs are really fair. So, there are a few general problems regarding ethical concerns when working with LLMs.

In addition to these ethical challenges, there are also some practical problems when using LLMs. Although LLMs seem to be good at reasoning, there are some tasks that are hard for them to solve, some even seemingly easy for humans, e.g., planning, counting, or calculations. We witnessed this in our own tests when the LLM was supposed to put out a certain amount of something. While this task is seemingly easy, the LLM struggles to put out the right amount, as it in principle only works with probabilities and does not count anything. Moreover, due to their non-deterministic nature, it is also a challenge to incorporate the usage of an LLM into an automated process, as the output can always vary. LLMs like those from OpenAI [66] try and solve this problem by adding parameters to the input that make the response more deterministic, e.g., a seed for the sampling. However, this still does not result in complete determinism (as explained in their documentation [61]) and can introduce other problems, e.g., problems with the quality of the output. One further general problem is that depending on the used model and the company behind it, there are varied costs and latency that come with the usage of an LLM.

Depending on the application, some of these problems are more severe than others. For us the main challenges include the problem of hallucination and the capabilities of the LLM, but also the inclusion of the LLM into an automatic process. However, depending on the usage of the later presented system, other aspects could also be problematic, e.g., the costs and latency of the used LLM.

2.1.2 Existing Models

There are a lot of different pre-trained models from different companies that are currently available. Some of them are:

- GPT Models from OpenAI [64]
- Llama Models from Meta [55]
- Claude Models from Anthropic [4]
- Gemini Models from Google [32]
- Mistral Models from Mistral AI [57]
- DeepSeek Models from DeepSeek [18]

These models are only a small selection of available models, as new models emerge frequently. Some of these models are open source and can be downloaded (e.g., Llama), while others need payment (e.g., GPT). It is hard to compare the capabilities of different LLMs, as some are better for certain tasks than others. We first focused on arguably the most prominent models from OpenAI, which are also accessible via an API. After a closer look at ChatGPT [60], we decided to try out the OpenAI models in our system, since they seemed to work quite well

with SQL. Moreover, based on a small study of some LLMs from Aynetdinov and Akbik [6], the newer models from OpenAI seem to perform very well in comparison to other LLMs.

2.1.3 OpenAI Models

As mentioned above, some of the most prominent models are from OpenAI [66]. This includes GPT-4o, GPT-4o mini, and o1 [64]. We will summarize some important information from the documentation [62] of the OpenAI models here: The models from OpenAI can be accessed using their own API. Other than, e.g., Chat-GPT, the API has no memory, i.e., referencing old messages is not possible. For every API request and response, a certain message object is sent to or from the API. These messages among other things contain the question to the model and the answer from it. To figure out the pricing of each of these messages, they are split into tokens, where—according to OpenAI—one token approximately equals 4 characters [69]. The cost of the messages is calculated depending on the amount of in- and output tokens (they have separate price tags), and the used model [67]. Some models are cheaper than others, ranging from less than a dollar to over one hundred dollars for a million output tokens. Generally, there is a limit on the amount of in- and output tokens sent per message, the so-called context window, and also a limit on the output tokens. These limits differ depending on the model, e.g., GPT-4o has a context window of size 128,000 and 16,384 maximum output tokens [65].

There are even more aspects and possibilities when using the OpenAI models. Some were not important or applicable for us due to various reasons, e.g., streaming of answers like in a chatbot. Others were only introduced during the work on this thesis, e.g., new reasoning models with additional capabilities, therefore they are not all mentioned here.

2.1.4 Prompting LLMs

Posing a question to an LLM for it to generate a response is called prompting. Such a prompt can include different things, like further information, hints, or additional instructions. A simple prompt could, e.g., look like this:

```
1 Rewrite the following SQL query:
2 SELECT * FROM students
```

Such a prompt is of course very simple, and without any information on how to do the rewrite, the LLM might have a hard time giving a reasonable answer. Thus, there are different ways to improve such requests. This field of research is called prompt engineering. Some of these techniques can be found in an overview by Naveed et al. [59], like in-context learning or chain-of-thought reasoning. Depending on the quality of the prompt, it might lead to better results than other prompts. Moreover, different models might respond better to certain prompts than others, as mentioned by, e.g., Pourreza and Rafiei [72]. Since prompt engineering is a completely own field and as the prompting style also depends on the model, we did not focus on prompt engineering for this thesis.

2.2 Semantic Search

The idea behind semantic search is to try and find the meaning of a query instead of just using the terms contained in the query to search for relevant results. There are different technologies to do this, some mentioned in a survey by Dong et al. [20]. Our goal is to find queries with the same intent as the original query, so we can either use this for the rewriting of the query or to check if the produced query is still the same as the original one regarding the intent. Thus, we want to explain some aspects of semantic search here.

Generally, semantic search is defined in the context of information retrieval, e.g., extracting relevant web pages for a given search term using semantics instead of the exact term. Therefore, some approaches mentioned by Dong et al. [20] rely on long texts as search documents. However, one more recent method in semantic search are embeddings, as used by Silva-Fuentes et al. [80]. In this case, embeddings are used to expand the query with semantically similar words, allowing for better search results. Here, documents are also used for the query expansion, but the underlying idea—to use embeddings for finding similar words—can also be used in our case, either for the rewriting itself or for comparing different queries.

2.2.1 Embeddings

As already mentioned above, embeddings can be used to find semantically similar words to a given word. The main idea of embeddings is to map words to vectors using word co-occurrence statistics, either learned via a neural network or counted in a large corpus of words [2]. This means that words that often occur in the same context should have vectors that are relatively similar to each other. Thus, embeddings are a neat way to transform words into vector space for easier comparisons using, e.g., cosine similarity. This makes them really useful for natural language processing tasks, semantic search, and clustering, but also for our use case. For us, embeddings are helpful, since they can be used to measure the similarity of words or sentences. Thus, they could be used either on their own or in combination with other methods (e.g., LLMs) in different parts of the rewriting system. Since they are made for semantic search, it remains to test if they are useful for intent similarity checking as well.

Other than our use case, there are multiple applications for embeddings. One use case, i.e., finding similar words, is already described above. Moreover, there are embeddings that work well for sentences, e.g., used by Aynedinov and Akbik [6] for a similarity score calculation. The idea behind sentence embedders is the same as for words: to calculate a vector for a sentence such that semantically similar sentences have similar vectors. There are a lot of different options both for word and sentence embedders. Since embedders need to be trained on a large amount of words, for our use case pre-trained embedders, i.e., embedders that can be used directly, are the best option. There are websites where such pre-trained embedders are listed, like HuggingFace [37]. Some of the most popular pre-trained embedders are the models from spaCy [35] and—for sentences—embedders trained using Sentence-BERT [77]. There are also embedding models from, e.g., OpenAI [64]. However, they are not free to use but have costs per executed embedding [63]. To avoid these additional costs we will utilize free-to-use models in this thesis.

Chapter 3

Related Work

There are different topics that are related to intent-oriented rewriting. While we will go over the biggest ones here, there are further topics related to some of the single approaches we will introduce in Chapter 4. Thus, we will go over those shortly when coming to the respective approaches.

3.1 Query Rewriting

Two of the most obvious related topics are query rewriting and optimization. It is an essential part of relational database systems. Most of the time these two terms are used interchangeably as their goal is to modify the query with the aim to optimize the execution in some way, often either regarding speed or resource consumption. Some authors also separate them and see rewriting as a step before the optimization, e.g., Liu and Mozafari [49], or one step during the optimization, e.g., Ioannidis [38]. We will treat rewriting as a part of optimization.

There are different options on how to modify a query. However, for normal query rewriting and optimization, one important aspect is that the result of the query has to stay the same, even after modification. This is the main difference to intent-oriented query rewriting.

The classic approach uses relational algebra together with a set of equivalence rules to transform the query into an equivalent one regarding those rules. This equivalence includes that the query's results are maintained. These rules are explained in different textbooks, e.g., by Kemper and Eickler [41]. One drawback is that these rules only cover a subset of queries. If these methods find queries where no rules apply they either leave them as they are or new rules have to be found which are applicable to these queries. This makes the approach not easily generalizable.

Other than the traditional query optimization, there are multiple approaches for query optimization and rewriting using different methods. Some of them focus on special kinds of queries. Ordonez [70] focuses on optimizing recursive SQL queries using indexing and relational algebra rules. There are also different approaches for optimizing nested queries, summarized and corrected by Ganski and Wong [29]. These approaches also use logical transformations on the query. The mentioned approaches mostly keep the SQL queries equivalent regarding a certain set of rules. However, as mentioned above, this is not really generalizable to all kinds of SQL queries. Therefore, Dong et al. [21] use synthesis-based techniques for rewriting queries. Although their system does not need predefined rules it still does not generalize to complicated queries. Thus, some more recent approaches try to incorporate machine learning into the process. These methods do not always keep the query equivalent regarding rules as in traditional query optimization, however, they still always maintain the result of the query. One of these approaches is proposed by Zhou et al. [92]. Their approach

combines traditional rule-based methods with learned methods to overcome the problem that rewriting is not applied adaptively.

Liu and Mozafari [49] go one step further by using LLMs for query rewriting. They employ so-called natural language rewrite rules to guide the LLM in the process of rewriting the query. Their approach shows that LLMs produce promising results for query rewriting and can adapt to different queries. However, one drawback of their approach is that rewrites from LLMs can be incorrect and thus sometimes need correction. This adds a further step in their rewriting process. Ye et al. [87] also use LLMs for query rewriting. While their goal is not to optimize the query performance, they still use the LLM as a rewriter. To further improve the LLM’s result quality they propose to use distillation, a technique incorporating fine-tuning of the LLM. Their approach also shows promising results.

In conclusion there are a lot of different options for rewriting queries. They all agree that the rewrite must have the same results as the original query while improving performance. Those are two big differences to intent-oriented query rewriting, as here both aspects are not necessary for the produced rewrite.

3.2 NL2SQL

Another related field to query rewriting is the processing of natural language into SQL (NL2SQL or Text-to-SQL in short). The goal here is to take a query written in natural language and generate a valid SQL query. This allows users without any SQL knowledge to still query databases. Some of the approaches provide a full natural language interface (NLI) for databases, allowing the user to directly query the database using natural language. A survey by Affolter et al. [1] gives a summary of existing approaches. Some of them are based on keywords, e.g., SODA [8], while others use patterns, certain parsing methods or grammar-based approaches. All of them try to use some kind of rules to generate a valid SQL query. In the survey, it is also mentioned that current research on NL2SQL uses machine learning to improve on the existing approaches. They find that such approaches support a greater variety of possible queries, making this kind of query writing more flexible. However, they also note that—depending on the approach—a lot of training data is needed.

Some of these newer approaches using deep learning are summarized in a study by Kim et al. [42]. They also mention that for some time one of the biggest challenges for these methods was the missing training data. However, after a while new benchmarks for this field emerged, allowing the development of more deep-learning NL2SQL approaches. One of these benchmarks is called Spider [88]. Since it has a lot of different databases in varying domains, it claims to be more complex than other benchmarks. An update to this benchmark has also recently been released, named Spider 2.0 [45]. It contains data from real-world applications to push NL2SQL development even further.

Some of the more recent methods that were proposed after the above-mentioned study include ValueNet [12] and CatSQL [28]. The idea behind ValueNet is to use the database values and useful hints as an aid for the neural network. CatSQL on the other hand uses a so-called template-based approach, i.e., the neural network fills in the missing parts in the template.

After the emergence of Large Language Models (LLMs), different approaches

utilizing them for NL2SQL emerged. These approaches currently dominate the field, as also noted in a current survey from Li et al. [47]. Many of these methods combine the usage of an LLM with different hint or prompting strategies or other machine learning components. Two of the best methods include DIN-SQL [72] and DAIL-SQL [30]. Both use different prompting strategies for the LLM to get an SQL query. As noted by the survey, although there are a lot of different methods for NL2SQL today—all based on varying approaches—there is still room for improvement. This also applies to the surveys authors proposed strategy combining LLMs and pre-trained language models (PLMs). As to their findings, needed improvements include making the methods reliably produce correct results, finding more cost-efficient methods (as some LLMs cost money), and generating more training data. With this data, future research could make use of both PLMs and LLMs to improve the performance on NL2SQL tasks.

All of the mentioned approaches—both rule-based and machine learning-based—aim to support users in querying a database. Intent-oriented query rewriting also aims to do this, however, there is one main difference: While NL2SQL aims to help users who cannot write SQL, we want to help users lacking data to answer their queries. So while the two fields are closely related in terms of problems like, e.g., producing reliable SQL queries, they still have different aims. However, it would be possible to combine both supporting mechanisms, e.g., by allowing the user to write a natural language query as input to the intent-oriented rewriting.

3.3 SQL Query Similarity

When talking about rewriting an SQL query into a different one that is still keeping the same intent, talking about query similarity is important. There are different options for measuring SQL query similarity. Some of them are mostly based on the structure and semantics of the query, while others try to incorporate intent into their similarity measure. The main idea behind all of them is to somehow measure how similar two SQL queries are. As described in Section 1.1 we want to find queries with the same intent as the original one, but that are different from one another (i.e., heterogeneous). Thus, measures for both structural and semantic similarity and intent-based similarity are interesting to us. We will take a look at existing work for these in the following paragraphs.

We will first talk about measures for identifying structurally or semantically similar queries. In the end, these kinds of measures are not relevant for identifying queries that are the same regarding their intent. This is due to the fact that queries with the same intent do not need to have the same structure or semantics, or even the same query result. Therefore, these kinds of measures cannot be used to compare the intent of the query. However, they could still be used for identifying semantically similar queries, which is an important aspect of finding heterogeneous query results, one of the goals mentioned in Section 1.1. These metrics can include simple string similarity measures like Levenshtein [46] or more complex ones like the measures compared by Kul et al. [44]. Here, the columns in different parts of the SQL queries are used and compared to determine their similarity. Other options for query similarity are presented by Borodin and Kanza [10]. They show two versions of structural query similarity

measures. One of them analyses the used attributes, similar to the approaches mentioned by Kul et al. [44]. The other one calculates the distance between the SQL syntax trees. This method is however really time-consuming. Thus, other methods are better for fast comparisons. Another approach for query similarity is the graph-based approach of Köberlein et al. [43]. The authors try to find a path from one query to the other using edit operations, thus mostly focusing on the semantics of the query. Since our queries could differ a lot, this is not feasible for us. There are even more measures for comparing queries, however, we just focused on a few interesting ones for SQL queries.

Another approach for SQL query similarity is trying to measure the intent. Since we want to produce queries with the same intent, this is definitely an interesting topic for us. There are multiple approaches to do this. One option proposed by, e.g., Borodin and Kanza [10], suggests comparing the results of the queries. They claim that this also captures the intent of the query. However, as explained with examples in Chapter 1, results do not have to be the same in our case. Another option suggested by them is to build a vector model from the used attributes. But since the used columns and tables can also change for the same intent, this idea is also not applicable for us. Guo et al. [34] use snippets from relevant documents to calculate the query similarity, something that is not available to us. Even if we would adapt this to the resulting values of an SQL query, the problem that the result of the intent-oriented rewrite does not have to be the same remains. The last approach we will mention here is the one from Arzamasova et al. [5]. They present a metric that works with data spaces of interest to determine the similarity of queries. However, they restrict it to relatively simple queries. Moreover, as mentioned before, using the results to compare the queries is not helpful in our case.

As described above SQL query similarity is definitely important for intent-oriented query rewriting (e.g., for ranking the produced queries). However, depending on the wanted similarity measure (structure- or intent-based), only some or no measures exist that could be usable for us (due to the reasons described above). Therefore, this topic is definitely related to what we need in our implementation, but most of the existing work is unusable for us.

Chapter 4

Approach

To tackle the task of an automatic intent-oriented rewriting system, we first had to evaluate which different steps are important for the rewriting process and in which order they need to be performed. These required steps are also dependent on the wanted in- and output. Since we did not want to restrict the input—other than it being an SQL query as stated in Section 1.1—we first need to check if a rewrite is needed. If the system is just pre-processing queries before giving them to the database it could happen that a completely working SQL query is given to it. After checking this we want to somehow rewrite the query. The main question at this point is whether to produce multiple different rewrites and post-process them later or to produce a single rewrite for the query. Both options have different advantages and disadvantages. Producing a single rewrite and trying to tune it perfectly (which could be a lot of work itself) would avoid post-processing a lot of queries. On the other hand, other good rewrites would be completely ignored. This could be a problem, especially since intent-oriented query rewriting is not always unambiguous, as shown in Chapter 1. Moreover, the original user intent can be vague, thus producing multiple rewrites and being able to show the top-k alternatives to the user can be an advantage. This allows the user to pick the rewrite fitting the true intent of the input query. Due to this, we decided to build a system producing multiple rewrites and post-process them to choose good ones. Other than being fitting, we also require the rewrites to be executable on the provided database. To achieve this we can either let the rewriter produce rewrites without any database schema—which then need to be post-processed—or we try to find executable rewrites directly. The former option could result in a lot of pruning and only few—or even no—executable rewrites. Thus we decided to include the database schema in the rewriting. Since we also want to rank the queries—as described in Section 1.1—at least one post-processing step is needed. Depending on the quality of the rewrites we need further post-processing which includes some kind of correction or testing mechanism to make sure of the executability wanted for the final queries.

4.1 Architecture

With all the mentioned restrictions in mind, we came up with the workflow described in Figure 4.1. We divided the workflow into two parts: The first part is responsible for creating multiple rewrites for the query given as the input. In the following, we will call those alternative queries or rewrites. In the second part of the workflow, we then execute multiple post-processing steps on those alternative queries to make sure that we provide the user with executable and useful queries. To fulfil all wanted requirements for the queries, each of the two phases has multiple processing steps.

At the beginning of the first phase, we do an **execution check** for the given SQL query. Here we check if the given query is executable on the database.

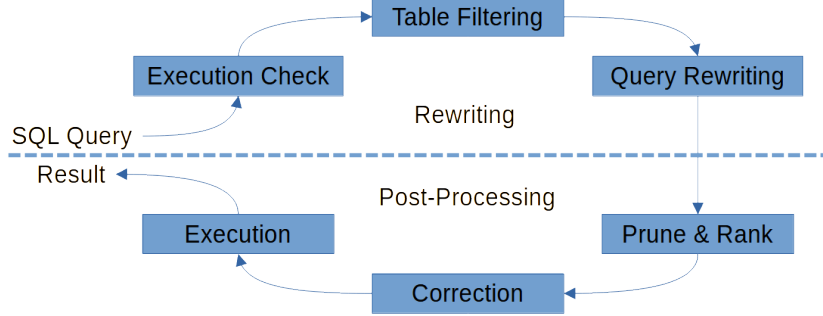


Figure 4.1: Workflow of the Rewriting System

The main reason for this is to reduce unnecessary workload for the rewriter since an executable query does not need rewriting and would unnecessarily use resources like LLM tokens. If the execution check determines the query to be non-executable, the rewriting process is started.

The second step is the **table filtering**. Its goal is to identify tables from the database usable for the rewriting of the given query. As described above, we want to include the table schema in the rewriting process, such that the resulting queries are executable on the given schema. One could suggest to just use the whole schema during the rewriting process instead of filtering out certain tables. While this can still be feasible on small databases with only a few tables, it can cause problems in larger databases. Most of those problems arise due to the usage of LLMs in the rewriting process. Giving an LLM all tables from a database can not only result in very high costs due to long requests (for fee-based LLMs) but also in hallucinations. These hallucinations can occur when there are too many options to choose from, which can result in bad rewrites. Thus, we need the table filtering to only choose appropriate tables for the rewrite.

After these two pre-processing steps the **rewriter** itself is the next step. There we want to produce multiple alternative queries that keep the intent of the original query. Similar to normal rewriting, there are multiple ways how to do this. As explained in Chapter 2, LLMs show promising results in query rewriting, thus being an intriguing option for this step.

After producing alternative queries we come to the post-processing phase, where we start with **pruning and ranking**. As explained above, we have to determine which queries from the rewriter are suitable alternatives and which are not. Thus, we can prune completely irrelevant queries and rank the remaining ones as demanded in Section 1.1. The ranking is done for two reasons: we preserve the option to show multiple queries to the user (if wanted) and we can further refine which alternative queries fit the original intent and which do not—even after pruning.

The next step in the workflow is the **correction**. When using an LLM for the rewriting there is no guarantee that the alternative queries are actually valid SQL. Thus, we check the executability of the queries and try to correct non-executable ones—if possible. It can happen here (assuming we want the top-k rewrites as the results) that we are left with too few executable rewrites, if the

ranking only gives exactly k results back and some are not correctable. Therefore, it would also be possible to do this step before the pruning and ranking. This could give us the possibility to directly prune non-correctable queries and avoid the problem that we get too few executable queries in the end, as described above. However, this would have the disadvantage that bad rewrites would also be corrected since we do not know which rewrites are unsuitable until after pruning and ranking. Depending on the complexity of the correction algorithm, this can result in an increase in runtime and—if an LLM is used—an increase in cost. Due to this, we decided to first do the ranking and pruning and then the correction. To make the process more robust to the elimination of some non-executable queries, it would however be an option to let the ranker return a few more rewrites following the requested k ones.

The final step of the post-processing is the actual **execution** of the query(s) on the database. To accomplish a full workflow from input to result we included this step in the process. This way the user can directly see the results from the produced query, and does not have to check the actual SQL query and schema to get the wanted data insight. Since some users may be interested in the SQL query as well, both query and result should be returned to the user. It would also be possible to skip this step, depending on the needs of the user. If the alternative SQL queries suffice, this step is not needed.

The presented system was first implemented with simple algorithms as a prototype. To enhance the overall performance and quality some parts were then improved with more sophisticated algorithms. In the following sections, we will have a closer look at the used algorithms and the improved versions (for the processing parts where such versions exist).

To implement all of the parts of our system both a database and an LLM are needed. Since both had to be accessed from different parts of the workflow, the whole architecture also includes the database and LLM API. The global picture can be seen in Figure 4.2. An example for a whole execution could then work as follows: The user poses an SQL query to the rewriting system. Using both the LLM and the database the system now produces multiple alternative queries which are post-processed. After the post-processing, the queries and the results are then given back to the user in a ranked fashion.

For the implementation, we used the programming language Python [26]. Since we use OpenAI models [64] for the LLM (as explained in Section 2.1.2), we use

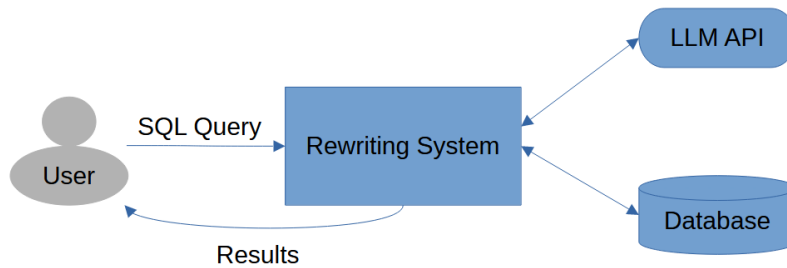


Figure 4.2: Global Architecture

the OpenAI API [62] for requests. As a database, we use DuckDB [25], since it is a fast and easy-to-use database system.

4.2 Pre-Check Executability

Checking the executability of a query on a given database is possible in two ways. One would be to use the *EXPLAIN* keyword, which is supported in the SQL syntax of various databases. Using this keyword avoids the actual query execution, but makes the database system create a physical execution plan. Thus, this also checks if the query would be executable on the tables. The main advantage is that it avoids the actual possibly time-consuming execution of the query. However, if the query is executable, it still needs to be executed afterwards to get the actual results from the database. The second way to check executability is to just execute the query on the database. By directly executing the executable queries we avoid one additional database access (if a result is wanted). This can be better for databases with a small amount of data since a direct execution would still be fast here. Since the databases used for testing the system did not have any or not a lot of data, we thus opted for just executing the queries. This could however easily be changed if the system needs to work with databases containing lots of data.

Another possible addition in this step is to check for easy rule-based intent-oriented rewrites. Since the next steps need a lot of resources, rewriting some queries in this early step could be an advantage. Finding a table in the database named with a synonym of the table used in the query could be an example of such a simple intent-oriented rewrite. Thus, depending on the use case and difficulty of the input using this step as a simple pre-processor could save resources. Since we first concentrated on the whole workflow and not on such optimizations, this was however not implemented.

4.3 Table Filtering

The table filtering is a really important step in the workflow. Here we need to choose the right tables to be used in the rewriting. Thus, we somehow have to compare the query intuition with the tables and determine which ones could be helpful to satisfy the user’s information need. If we choose too few tables or the wrong tables, then the rewriter cannot produce relevant rewrites. As described already, giving too many tables to the rewriter could result in high costs and hallucinations when working with LLMs. We can see an example of such a hallucination in Figure 4.3. Here the LLM suddenly takes soccer teams instead of baseball teams to rewrite the original query, thus obviously changing the intent of the request. This can be avoided by directly eliminating such non-fitting tables before the rewrite even happens.

Query intuition plays a big role in determining usable tables. If we only use tables that are, e.g., synonyms of the tables in the query, we could miss interesting tables that are, e.g., correlated to the original ones.

We considered multiple approaches for table filtering. One idea was to use knowledge databases like WordNet by Miller [56] and Fellbaum [24], or Free-

Original query:

```
1 SELECT best_team FROM baseball;
```

Rewrite from the LLM:

```
1 SELECT team_long_name FROM soccer_1_Team;
```

Figure 4.3: Hallucinated Query due to too many Tables

base from Bollacker et al. [9]. Such databases include topics or words and their relationships, thus they would be usable to find related topics for the tables in the query. There are two problems with this: First, we would still need to somehow identify the topics from the tables or extract the words making up the table names. Generally, extracting topics from a document could be done with latent semantic analysis (LSA) as described by Deerwester et al. [19]. However, LSA focuses on texts and not on single words like table names and can therefore not be used here. The other reason against knowledge databases is that LLMs also include a lot of their knowledge since they are trained on a lot of data. Therefore, it is easier to directly use an LLM that is able to deduce topics from the tables and compare those using the knowledge from its training—which is possibly more than the knowledge in such a knowledge database. The only drawback of the LLM compared to a knowledge database might be the cost, as some knowledge databases are openly available while many LLMs are fee-based. Another idea was to use some approaches from dataset search and schema matching to find fitting tables to those in the query. An overview of dataset search is given by Chapman et al. [16]. For schema matching a review was done by Evermann [23]. Both options have interesting ideas, as part of them is to find similar tables. However, they focus on finding tables exactly matching the request or the given table. Thus, incorporating the human intent and finding tables that are usable as viewed by a human would have to be incorporated anyway. So we chose not to follow these approaches.

With this information in mind, we came up with 4 different approaches that all try to capture the query and table meaning and intent. To achieve this, most of them use an LLM in some way. We will explain them in the following sections. For each filter there is a possibility that it does not return any tables that are usable for a rewrite. In that case, we abort the whole process with an exception that no rewrites could be found, as we cannot rewrite a query without any tables.

4.3.1 Embedding Filter

To build the prototype we wanted some simple filter that would be easy to implement. As mentioned in Section 2.2 embeddings are often used to search for results with the same meaning as the request. Thus, we decided to try word embedding on the tables of both the query and the database. Instead of only using an embedding we decided to use the *en_core_web_lg* pipeline [82] from spaCy [35]. The reason for this is that we did not only want to give table names directly into an embedder, as table names can be a combination of different words. Thus, an embedder working with single words might not know the ta-

ble names. If a word is not known by an embedder, it can happen that it is represented by the empty vector, which should be avoided as it is a bad representation. To avoid this, we used spaCy since it also incorporates tokenization and annotation of words in its whole pipeline. After multiple pipeline steps, the tokenized words can then be vectorized and averaged to represent the table. To be as precise as possible we use the largest available model for the English language, which generally has a higher accuracy evaluation according to spaCys website [81]. However, independent from the used model, it can always happen that some words might be unknown in the dictionary, resulting in said empty vectors. This is definitely a drawback, but since this happens rarely it is not a big problem.

The whole algorithm then works as seen in Algorithm 1.

Algorithm 1 Embedding Table Filter Algorithm

Input: The SQL query q , a threshold λ

Result: Qualifying tables from the database

$database_tables = getDatabaseTables()$

$query_tables = getSQLQueryTables(q)$

$qualifying_tables = []$

for $database_table$ **in** $database_tables$:

for $query_table$ **in** $query_tables$:

$doc1 = getSpacyVectorization(database_table)$

$doc2 = getSpacyVectorization(query_table)$

if $\text{cosineSimilarity}(doc1, doc2) > \lambda$:

$qualifying_tables.add(database_table)$

return $qualifying_tables$

After getting the tables from the database and the query (which is done using the sql-metadata package [11] in the whole implementation), the basic idea of the algorithm is to pairwise compare the spaCy vectors from each of the database and query tables. If the cosine similarity between the vectors of the tables is higher than a certain threshold, the database table will be added to the so-called qualifying tables, as seen in the algorithm. The threshold for this approach (and also those for the following approaches) was determined via testing.

Another option to adding the tables above a certain threshold would be to pick the top-k tables regarding the similarity. This would guarantee that there are always tables to work with. We still decided against this option, since we only want relevant tables for the rewrite. This is important since we use an LLM for the rewriting. As hallucination is a big problem in LLMs—as explained in Section 2.1.1—giving the LLM non-relevant tables will result in non-relevant queries, just like the one in Figure 4.3. Thus, we rather not give the LLM any tables and deduce that no rewrite is possible with our database than giving it non-relevant tables.

This whole approach is pretty simple, so we did not expect it to perform exceptionally well. Especially since we tried comparing different tables that have connected topics for a human, e.g., income and rent from the introductory example. Here the embeddings did not perform well. Therefore, we added more elaborate approaches to the system to improve the performance quality of this important step.

4.3.2 Simple LLM Filter

As mentioned in the beginning of this section we came up with various options for using an LLM for the table filtering. The first approach that came to mind when thinking about using an LLM was: Why don't we just ask the LLM which of our tables are relevant for the given query? So in our first approach, we wrote an LLM prompt that takes all of our database tables and is asked which ones are relevant for rewriting the given SQL query. Although this seems really simple there are some important aspects that we had to keep in mind for the implementation.

One thing to keep in mind is the limited amount of in- and output tokens one API call of the LLM can handle. The combination of in- and output and reasoning tokens is normally also restricted with the so-called context window mentioned in Section 2.1.3. For instance, for GPT-4o the context window currently has a size of 128,000 tokens, while the amount of maximal output tokens is limited to 16,384 tokens [65]. If we now want to give a prompt including a lot of table names to the LLM, it could happen that these limits are exceeded, especially if we have a large database. Therefore, we partition our table names into packages that are small enough for the LLMs context window and output size. The maximum package size can be calculated from the average amount of tokens needed for one table name, the maximum output length and the expected percent of returned tables (i.e., how many of the tables are actually deemed relevant).

The calculation of the maximum package size works as follows: First, we calculate how many underlines there are on average per table name (as underlines start a new token [68] and can be used in table names in SQL). Then we calculate the average amount of characters per word (one word are characters not separated by an underline, e.g., *city* and *supermarkets* in *city_supermarkets*) and divide this by 4 (as explained in Section 2.1.3 four chars are approximately one token). Then we multiply this number by the average amount of underlines in a single table name. We thus get the average amount of tokens for one table name. The context window—at least for GPT-4o—is a lot bigger than the maximum possible output tokens. Therefore, the number of input tables is restricted by how many tables could be returned by the LLM. So the last thing we need to approximate is how many tables will be returned by the LLM in percent (as a ratio to the input tables). We now use the following formula to calculate the number of tables in one package:

$$\left\lfloor \frac{\text{max_possible_output_tokens}}{\text{avg_tokens_per_table}} * \frac{1}{\text{percent_of_tables_returned}} \right\rfloor$$

Using this number we split up the tables into packages, which are given—together with the original query and a prompt—to the LLM after each other, such that the LLM can decide which tables are useful. The full prompt can be found in Appendix A.1. Due to the limited amount of in- and output tokens, it can happen that we have to make multiple LLM API calls for this method. This is however not a problem other than the possible latency and therefore runtime overhead.

The next step is to process the answers from the LLM by extracting the table names. After this, it would in theory be possible to go to the next step. However, when testing this approach we noticed that the LLM tends to hallucinate

additional table names that do not exist in the database. Thus, it can happen that an answer not only contains real table names from our database but also hallucinated tables that it was not given in the first place. In addition, the LLM sometimes makes small spelling errors when returning the tables. We therefore have to do some post-processing on the returned table names from the LLM to double-check if they—or a similar table—are actually in our database. To do this we go through all the tables suggested by the LLM. For each one, we check if it is in our database. If that is the case we add it to our result and go to the next table. Otherwise, we use the inbuilt string similarity function from Python [27] to try and find a similar table to the suggested one. To do this we find the most similar table and check if it is above a certain threshold (to avoid taking a random table if none are similar). If we find such a table in our database, we add it to the result. Otherwise, we continue with the next table without adding anything to the result. After going through all the suggested tables we can then return the result containing the real database tables. This is given to the rewriter to continue with the workflow.

One obvious drawback of this approach is the amount of used LLM tokens for large databases since the LLM gets all of the table names. To remedy this drawback while still using an LLM, we came up with the next approach for table filtering.

4.3.3 Complex LLM Filter

Another thought that may come to mind when working with an LLM is to use the apparent reasoning and creativity of the LLM. Here, that would mean just asking the LLM for some tables it would suggest for rewriting a given query. Unlike before, we do not give it any tables from the database. To make sure that we still find tables that are actually in the database some post-processing must be done on those suggestions. Those two parts make up our third approach, i.e., the second approach using an LLM. Since we do not need to give all tables to the LLM the token usage will definitely be less than the last approach, however, the post-processing could take some time.

The first part is prompting the LLM to imagine usable tables for rewriting the query. The prompt for this can be found in Appendix A.2. This time we do not have to be careful with the in- or output token amount, since we do not expect the LLM to exceed its own output limit when naming a few tables. We could however restrict the LLM to only output a certain number of suggestions. If this number is too high the LLM could produce hallucinated unnecessary tables. On the other hand, if the number is too low, we could miss interesting suggestions from the LLM. So, finding this number would need some testing. Moreover, after some executions of our prompt, we noticed that the LLM often returns a similar amount of tables anyway. Thus, we decided against limiting the LLM to a fixed amount of tables. An example of some tables suggested by the LLM can be found in Figure 4.4. As we can see the LLM both suggests tables that are directly related to the query, e.g., *baseball_teams*, but also tables that could help with identifying good teams, e.g., *baseball_standings* or *fan_ratings*. Thus, it also finds related tables and not only synonyms.

After processing the answer from the LLM and extracting its suggestion we now need to find fitting tables in the database as a second step of this filter. Since

```

1 SELECT best_team FROM baseball;

      ↓ LLM prompt

1 baseball_teams; mlb_teams; sports_clubs; team_rankings;
2 sports_statistics; baseball_standings; championship_winners;
3 stadium_locations; player_performance; fan_ratings; sponsorship_deals

```

Figure 4.4: Example Output of the LLM in the Complex Table Filter

the LLM also suggests related tables (and not only synonyms) we can focus on finding synonyms or tables with a similar name to the suggested ones (or to those in the original query). One option would be to just use embeddings as in the first approach and compare the tables. As explained in Section 2.2 embeddings are good for semantic search, so in our eyes using embeddings here is a valid option. So, we essentially do a pairwise comparison of the suggested tables and our database tables using the same technique and algorithm as described in Section 4.3.1. There is however one small change that we made to the algorithm. As explained before the spaCy library [35] used for the embedding already does some kind of tokenization. We wanted to compare this tokenization to our own version since we noticed that many suggested table names from the LLM consist of multiple words written in snake (*snake_case*) or camel case (*CamelCase* or *camelCase*), as also seen in Figure 4.4. Thus, our tokenizer splits these two naming variants into their single words. So, instead of just comparing the embeddings of all database and suggested tables pairwise, we use four different comparison methods for the tables. We compare:

- The spaCy embeddings of both suggested and database tables
- Our tokenization with spaCys embedding of the suggested tables and the spaCy embedding of the database tables
- The spaCy embedding of the suggested tables and our tokenization with spaCys embedding of the database tables
- Our tokenization with spaCys embedding of both suggested and database tables

This way we can ensure that the typical variants of writing the table names are definitely considered as single words in some of the comparisons. SpaCy can then use the averaged vectors of these words as a representation for the comparison. For all of these comparisons we use a threshold that has to be exceeded for a database table to be added to the result tables. We use a threshold just like in the embedding algorithm for the same reason as explained in Section 4.3.1. In short, we want to avoid adding unrelated tables.

With this algorithm, we wanted to avoid the high token cost from the second algorithm while still considering intent (more than the first approach). However, we still rely on the internal calculations of the LLM to find usable tables.

4.3.4 Table Summaries

For both of the above approaches utilizing an LLM, we have to rely on the internal reasoning of it to identify the correct tables. But instead of just relying on this reasoning we could also do the comparison step-wise to consciously include query and table summaries, topics, and keywords in the filtering. This is exactly the approach of the fourth table filter algorithm. We use this metadata since it reflects the intent of the query and can show relatedness regarding topics and intent. The approach works in the following steps:

1. Get metadata (summaries, topics and keywords) for all tables in the database
2. Save this metadata in a table of a metadata database
3. Get metadata (summary, topics and keywords) for the query
4. Compare the metadata of the query to that from each table
5. Select tables where the metadata is similar enough to the query

Getting Table Metadata

Extracting a summary or keywords from a table has been researched in multiple papers under the topics of table-to-text and table summarization. There are different approaches for doing this, including using artificial intelligence like LLMs. Some of the existing approaches focus on summarizing the data contained in the tables, e.g., Zhang et al. [90], Selçuk Candan et al. [13], and Lo et al. [53]. Others focus on generating a text describing the table (and often the existent values) using machine learning, e.g., Cao [14], Liu et al. [51], and Chen et al. [17]. There also exists research utilizing LLMs for table-to-text generation from, e.g., Gong et al. [31].

However, for our use case, we do not directly want to summarize table data but give an intuition on the possible use cases of the table for a rewrite. At this point, we did not want to incorporate data into such summarizations, although it would be a possible option to do this at a later point. Thus, the above-mentioned approaches did not apply to our use case. However, one approach from Zhang et al. [91] shows interesting results when using LLMs for table-based tasks. Although their fine-tuned LLM did not fit our task, we decided to try out LLMs for generating a table summary, topics and keywords from a given *CREATE TABLE* SQL statement. We use the table descriptions as SQL statements, as this representation is more effective than a textual representation (and still easily available), as evaluated in a study by Nan et al. [58]. Although they work on the problem of Text-to-SQL (described in Section 3.2), their results in this part of prompt design should be applicable in our prompts as well.

So for the first step of this approach, we decided to use an LLM to generate metadata for each of the tables in our database. The prompt for this can be found in Appendix A.3. To avoid too much latency we decided to directly ask the LLM for metadata for multiple tables, not just a single one per request. This brought us a similar problem as in our simple LLM approach from Section 4.3.2, namely that the in- and output tokens of an LLM are limited. With a lot of tables, this could be a problem, since both the *CREATE TABLE* statements for the input and the summaries for the output can take up a lot of tokens. We

therefore decided to calculate the maximum amount of tables that we could give to the LLM in one request.

The calculation works as follows: We first calculate the average amount of characters in one *CREATE TABLE* statement and divide it by 4 to get the approximate average amount of tokens. After subtracting the length of our prompt (roughly 300 tokens) from the maximum input length, we could then divide it by the average amount of tokens used for each table. This resulted in a maximum amount of tables that we could give in the input. For the output, we approximated the output length for each table by giving the LLM a token limit for the table summary and approximating the amount of tokens for the keywords and topics by multiplying their amount by four (i.e., assuming that each topic or keyword needs roughly four tokens). After summing all those numbers up (summary, topic and keyword tokens), we could divide the maximum amount of output tokens by it to get the maximum amount of tables usable to still fit the output length of the LLM. By taking the minimum of the maximum in- and output amount we got the maximum amount of tables per prompt. Using this number we could then split the tables into packages that the LLM could handle. One thing we noticed after trying some queries with this method, was that the more tables the LLM has to summarize, the worse it gets regarding reliability. For instance, it would happen more frequently that not all written tables got metadata but only some, or that the names of the tables returned with the metadata did not fit the table names from the database. This is a phenomenon already mentioned in Section 2.1.1, which is a result of the fact that the LLM does not really count how many summaries it already returned. We thus decided to add a parameter that would allow users to limit the amount of tables per prompt to a fixed size (given that this number is not bigger than the aforementioned calculated number). This allows a user to dynamically adapt this part to the varying capabilities of different LLMs.

After getting the result from the LLM we need to process it to separate the metadata for each table. This can lead to problems if the LLM uses a different format than the requested one. In that case, we ignore the problematic line in the return and continue. This can result in some tables without metadata in the first run of the program (when no data is available from before). It would be possible to re-prompt the LLM for metadata of those tables during the first run. However, this could result in really long processing times if the LLM has to do multiple iterations for some tables. Thus, we decided to ignore tables without metadata in the further algorithm steps and retry the creation of metadata for those tables in the second run.

An example of the result after all these steps can be seen in Figure 4.5. As we can see the summary is still relatively near to the actual columns in the table. However, the topics and keywords are more general, which can help in finding relevant tables that do not have exactly the same information, but just related data.

Saving Metadata

As mentioned before it can be both costly and time-consuming to create metadata for a large amount of tables—especially when using an LLM with API latency and costs per token—, we decided that it would be helpful to save the generated metadata in a database. This is the second step of our algorithm. As

```

1      CREATE TABLE baseball_team( founded_year INTEGER, league_id
      TEXT, team_id TEXT UNIQUE, franchise_id TEXT);

      ↓ LLM prompt and processing

1      Summary:
2      A table storing information about professional baseball
      teams, including their founding year, league affiliation,
      and unique identifiers.
3      Topics:
4      Team history, League affiliation, Franchise identity, Sports
      organizations, Baseball records
5      Keywords:
6      Baseball, Teams, League, Franchise, Sports

```

Figure 4.5: Example Summary, Topics and Keywords of the LLM for one Table

a result the first two steps only have to be done once, since we have a database with all needed information after executing them. Thus, after the first execution, we can use the metadata from the database for the following steps. We only have to recalculate metadata if the tables change or if the LLM return cannot be processed as described above. In those cases, we still need to calculate at least some new metadata in further runs of the program. The overhead for this is however negligible.

Getting Query Metadata

After saving all needed metadata for our tables we need to extract metadata from our query, such that we are able to do a comparison of the metadata. It would be hard to extract information from just the tables, as we normally only have their names and no further info. Thus, we decided to get our needed information from the whole query. To optimally compare the database tables to the query we wanted both topics and keywords from it, which we could then compare to those from our database tables. Moreover, we wanted to compare the query intent to our table summaries. A different option would have been to just compare a query summary to the table summaries, however, the intent of the query is more important for our rewriting than just a plain summary of the exact operations of the query. Therefore, we decided to get the intent (as a sentence), topics, and keywords from the query for comparison. Just like for the table summaries, there is research on extracting the intent from a search query. Guo et al. [34] use an approach tailored to web search, since they incorporate search result snippets and user clicks to find their intent. This is however not applicable in our case. Two other approaches from Srinivasan et al. [83] and Zhang et al. [89] employ machine learning techniques for intent description generation. However, both are searching for relevant documents with this. Thus, they assume that they have some relevant documents from which they can take the title and text to enrich their query before generating the intent. This is again not applicable in our use case. Anand et al. [3] propose using LLMs for the intent generation task. While they still use humans for confirmation, they initially generate the intents using GPT-4 [64]. Since we want an automated

```

1 SELECT best_team FROM baseball;

      ↓ multiple LLM prompts and processing

1 Intent:
2 Retrieve the name of the best baseball team from the database.
3 Topics:
4 team rankings, baseball statistics, sports performance, championship
   winners, competitive analysis
5 Keywords:
6 ranking, performance, competition, sports, winner

```

Figure 4.6: Example Summary, Topics and Keywords of the LLM for one Query

rewriter, it is not possible in our case to use human annotation in our workflow. Using an LLM to generate the intents is however still possible for us. Thus, we chose to use an LLM to generate the query intent, topics and keywords. The prompts for these generations can be found in Appendix A.4. In these prompts, it is possible to adjust how many keywords and topics are wanted. To avoid having too many (and to not overstrain the LLM) we set these values to 5 in our code. After processing the LLM output we then successfully obtain the intent of the query together with its topics and keywords. An example of the results can be found in Figure 4.6. As we can see the LLM does not only summarize the query but gives a general description. Moreover, the topics and keywords are pretty general, just like for the tables.

Comparison of Metadata

At this point in the algorithm, we are ready to compare the query and the tables in our database to find the relevant ones. Each of the tables thus needs to be compared to our query using the created metadata. Since the metadata should already capture the intent the most important aspect now is to find semantically similar metadata. This brings us back to embeddings. To compare the topics and keywords we used the same embedding from spaCy [35] as in the aforementioned approaches. We did this since this approach works quite well for single words, just like our topics and keywords. However, for sentences we wanted to try a different option, as the context in the sentence itself can be pretty important. We therefore opted to try out sentence embedders, which are also explained in Section 2.2. The idea is similar to the idea behind Sem-Score [6], where two sentences are given a similarity score by calculating the cosine similarity of their sentence embeddings. There are a lot of different sentence embeddings that are readily available. One popular sentence embedder is Sentence-BERT from Reimers and Gurevych [77]. On their repository [75] they provide multiple options for pre-trained sentence embedders for different tasks. One of those tasks is monolingual semantic search, which fits best for our use case. The best ranking models for this according to their rating [76] are *multi-qa-mpnet-base-dot-v1* and *multi-qa-mpnet-base-cos-v1*. We opted for the second version, as it also supports other similarity functions than cosine-similarity. This makes it more adaptable if we want to try different settings. With those different embeddings, it is now possible to determine the similar-

ity between the query and the tables. To do this we compare the following measures:

- The table summary and the query intent using the embedding *multi-qa-mpnet-base-cos-v1* and cosine similarity
- The table and query topics using the embeddings from spaCy and cosine similarity
- The table and query keywords using the embeddings from spaCy and cosine similarity

We do all of these comparisons separately and check if they are above a certain threshold. This way we get tables that are similar to the query in at least one of the three categories, excluding completely unrelated tables, which could be a problem when choosing the top-k similar tables (just like in the previous approaches). It is interesting to note here that the thresholds for the sentence embedding and the word embeddings differ in our implementation. Although the value range for all comparisons is the same (all use cosine similarity) we noticed during testing that sentences that are similar have lower similarity values than similar topics or keywords. Thus, we set this threshold to be a bit lower than the other ones in our implementation.

Using this comparison step we can now determine which tables are relevant for our query and add those to the result.

So, after combining all the steps for this approach we can find related tables, not only relying on their names and the query tables but also on the table’s summaries, topics and keywords. This can potentially give us better comparison results than in the other approaches. However, since most of the steps rely on an LLM, hallucination and cost could be a problem.

4.3.5 Database Prefixes

If we have a big database with many tables, it is possible that those tables originate from different smaller databases and are combined into one big database. For our use case, this would be optimal, since the more tables there are, the higher the likelihood that some of them are related to our original query.

One problem when combining tables from multiple databases can be name doubling. Since we cannot have multiple tables with the same name in one database, we somehow need to rename or change table names.

There are different possibilities for how such tables can be renamed in a combined database. One way would be to add a prefix to each table name consisting of the database name the table originates from followed by an underline, i.e., change the table name *team* from the database *baseball* to *baseball_team*. This way we avoid name doubling, at least if we do not have databases with the same names. An advantage of this technique is that we can later determine which tables were from the same database. This can give us information on, e.g., which tables contain data that belongs together and are therefore joinable. We can use this information in our table filtering in different ways.

Improving Table Filters

One way to use this information is to improve the table filtering. Some of the described approaches can be adapted in different parts to incorporate such information. Thus, we made adaptations to the second and third approaches of the table filtering. We made sure that those versions can be switched off if the database naming convention is different than described above.

One small adaption method was used for the simple LLM filter described in Section 4.3.2. As we can see in Appendix A.1 we added an optional part to the prompt. This change adds the information on the table naming convention to the LLM prompt. This way, the LLM is able to use the information on the database and table names to make more informed decisions on which tables could be relevant. Furthermore, it can potentially make better decisions when joining tables in a query.

The second adaptation was made in the complex LLM filter explained in Section 4.3.3. Here we did not change the prompt, since the LLM was asked to just think of some tables. However, we changed the processing of the suggestions. More specifically we adapted how the suggestions are compared to our database tables. Since we know that all tables from one original database have the same prefix, we could deduce that they are all relevant if something similar to the database name was suggested by the LLM. The drawback of this method is that many unnecessary tables could get added for a big database. On the other hand, those tables might be relevant if the LLM suggestion is correct. Therefore, we decided to do the comparison between one database table and one suggested table as follows:

- Tokenize the tables using our approach for snake (*snake_case*) and camel case (*CamelCase* or *camelCase*).
- Stem the tokenized words using the Porter Stemmer [71] (implemented [73] in the Python package NLTK [7]).
- Calculate the set intersection between the stemmed tokens of the database and the suggested table.
- Mark the database table as relevant if the set intersection has at least size one.

With the stemming we make sure to not miss the same word that is just written in different forms. This way we both get tables that are relevant due to their original table name, but we can also get all the tables from one original database if its naming fits the suggestion. Without the prefix naming convention, we could not guarantee the second part, so we would only get single fitting tables. Thus, we chose to use embeddings in the normal approach to find more tables than just single ones with the same naming parts.

Post-Processing Relevant Tables

After finding the relevant tables with one of the table filters we can also do some post-processing on those found tables and, e.g., add further tables to the relevant ones. We will explain two options for this.

For this step, we can also make use of the prefixes in table names—if they exist.

One option would be to add all the tables from the same original database to the relevant tables if one table from this database was deemed relevant. This could add more interesting tables with a similar topic to the found tables. However, if we have big original databases this also adds a lot of overhead and can add many tables that are irrelevant for our query. This can lead to a lot of hallucination from the LLM during the rewriting. Therefore, we decided not to implement this feature.

Another possibility—not using the prefixes—would be to add tables that join together two of the relevant tables but were not chosen in the filter. We can check this using foreign key constraints of the relevant tables. If two tables have column references from or to the same table, they could be interesting as well. Not only is it possible that they can be used for a consistent join in the database, but the probability that they are relevant for us is also higher. Thus, such tables could be added to the relevant tables. However, just like for the other approach, this could potentially add a lot of tables and thereby promote hallucination.

Both options could still be tested in the future to evaluate if their improvements outweigh their drawbacks.

4.4 Query Rewriting

This step is the main step responsible for producing rewrites. After finishing all preparations we have all the needed input for this step. The rewrite itself could be done with a cautiously crafted algorithm, e.g., one that tries to find replacements for the tables and columns using synonyms, correlations, hierarchies, and functional dependencies. However, this would only cover a small amount of rewrites, since a query that has the same intent can look completely different than the original query as explained in Chapter 1. Another option is to use machine learning. From Section 3.2 we know that LLMs have been used quite a lot for producing SQL from natural language, performing quite well in this field. Moreover, they have also been used in the field of rewriting, as explained in Section 3.1. This brought us to the conclusion that an LLM might also be good at intent-oriented query rewriting, as it can include all knowledge it has on synonyms, hierarchies, and more. So in theory it could include all the knowledge we would use for a hand-crafted algorithm. In addition, it is able to dynamically rewrite the queries, making it more adaptable than hand-crafted rules. Thus, for both implemented approaches we chose to try out LLMs as rewriters.

For the rewrites themselves, we have a few different restrictions. We want them to solve our problem statement introduced in Section 1.1. Thus, we want to have queries that are the same as the original query regarding the intent. Moreover, we do want multiple rewrites, as mentioned at the start of this chapter. To avoid just having multiple rewrites that are identical, we somehow need to introduce heterogeneity into the rewrite generation. This way we reduce the chance of having multiple rewrites that are basically the same and are all bad. Having rewrites that differ pairwise could also result in some irrelevant rewrites, e.g., if the LLM tries to think of a new rewrite possibility and starts to hallucinate, or if there are only a few good options for a rewrite. However, we accept this

possibility, knowing that we still have a post-processing part for the queries for pruning and ranking. To introduce this heterogeneity into the LLM generated rewrites we have to consider that some LLMs or their APIs do not have any kind of memory. This means that they cannot save the queries they already gave to us. This leaves us with two options: we either ask the LLM to produce all rewrites in a single prompt, or we have to send the already received rewrites back to the LLM each time to avoid getting the same queries back. The second option can get quite costly for fee-based LLMs due to a lot of tokens needed in each request. Moreover, we would introduce a lot of latency with many different LLM requests. Thus, we opted for the first variant.

Since both of our approaches use this variant it can happen for both that the LLM does not produce the amount of requested rewrites. In that case, we abort the process. This may seem a bit drastic, but depending on the amount of produced queries we do not have enough queries left after the post-processing, i.e., we would not be able to give the requested amount of alternative queries to the user. Moreover, this happens seldom if the amount of requested alternatives is relatively small, e.g., does not exceed 10–20 rewrites.

When using an LLM for rewriting there are a few things we want to consider. One relevant problem with LLMs is reliability. Since hallucination is a big problem in LLMs—as described in Section 2.1.1—it is important to check the results for plausibility. This is done via our post-processing phase, where we rank and prune the queries. Furthermore, there is the possibility to give the top-k of these queries back to the user for him to double-check them again. Another thing we need to consider is prompt engineering. There are a lot of different options for prompt engineering, some of them mentioned in the papers in Section 2.1.4. Some of them use examples or hints, while others are just strategies for enhancing the prompt writing itself. We opted to start with two really simple prompts that are directly given to the LLM without using multiple iterations, in order to keep the implementations simple. We found these prompts by formulating the task for the LLM in a prompt and then trying them out a few times, adapting non-working parts in the process.

4.4.1 Simple Rewriting

The simple rewriting is the first rewrite technique introduced for the prototype of the system. As the name suggests the prompting of the LLM is not too complicated. It can be found in Appendix A.5. It is inspired by the prompts for query rewriting by Liu and Mozafari [49].

The general idea here is to describe to the LLM what exactly it should do. To improve possible aggregations the LLM not only gets the table and column names, but also the columns data types. This can avoid non-executable aggregates of columns, e.g., averages on string columns. Moreover, the LLM is given all foreign keys that a table might have. We do this to improve the joining behaviour in the suggested rewrites. Using all this information we prompt the LLM to produce a certain amount of SQL rewrites that are diverse (as described above we want some heterogeneity).

After receiving the answer to our prompt we can separate the produced queries and start with the post-processing phase—if enough rewrites were found.

4.4.2 Rewriting via Natural Language

Inspired by the encouraging results of using LLMs in natural language processing like NL2SQL mentioned in Section 3.2, we decided to try one other approach for rewriting our queries.

In this approach, we do not give an SQL query to the LLM. Before prompting the LLM for the rewrite we use another LLM request to get the intent of the query. The problem of getting the intent of a query was already discussed in Section 4.3.4. We use the same solution here, namely prompting the LLM for the intent of the query. This is done with the first prompt of Appendix A.4. We then use this intent in our rewrite prompt instead of the SQL query, as seen in the prompt in Appendix A.6. Just like in the first approach, we give the tables with their foreign keys and the columns with their types. We also include the number of queries we want to produce. The complete prompt can then be posed to the LLM.

The answer from the LLM to the prompt contains the alternative queries in the same format as in the first approach. This allows us to process it in the same way to get the rewrites for further processing.

In comparison to the first approach, this one needs one more API call and thus more tokens and time. However, since the intent is already abstracted from the SQL query, the LLM can potentially write more general queries without adhering to the original query too much. Since a query with the same intent does not need to be structurally or semantically similar to the original query, this might give the second rewriter an advantage over the first approach.

4.5 Prune and Rank

This is an important step for checking the results of the rewriting done by the LLM. We want to make sure that only relevant rewrites are presented to the user. As we cannot rely on the LLM to not hallucinate during the rewrites, we thus have to check the produced alternative queries in the post-processing steps. There are two things we want to do here: First, we want to prune queries that do not make sense with our schema. It is possible that such queries seem correct and can be executed on the database, but with the knowledge of the database schema are simply wrong. Thus, we do not want those to be given to a user. We do this step first, as we want to exclude them from the ranking to avoid unnecessary computations (as they would otherwise still be eliminated, just afterwards). After pruning we go on to rank the queries. Here, we do not just select the best query, but establish a top-k ranking, as described in Section 1.1. This allows us to not only show the best query to the user but the top-k queries. Thus, we heighten the probability that we show differing relevant queries, as we could have ambiguous input queries and also multiple possible rewrites.

4.5.1 Pruning Algorithm

As mentioned above, we want to eliminate queries that will not work with our schema. An example of such a query can be found in Listing 4.1. This query may seem reasonable for finding students with pets. However, with more knowledge of the database schema, we know that the tables for dorm students and students

with pets do not originate from the same database. Thus, the IDs given to the students in both of these tables will not match with each other, resulting in false results of the query. Therefore, we want to eliminate such queries.

```

1 SELECT CONCAT(Fname, ' ', LName) AS full_student_name
2 FROM dorm_1_Student d, pets_1_Has_Pet hp, pets_1_Pets p
3 WHERE d.StuID = hp.StuID AND hp.PetID = p.PetID AND p.PetType = 'Cat'

```

Listing 4.1: Example of a Prunable Query

Although there may be multiple options for pruning queries, we currently only have one method implemented (more ideas can be found in Chapter 6). This method is based on the prefix naming of tables from the same database explained in Section 4.3.5. From this naming method, we can deduce one fact for the schema: Tables with different prefixes in their names are not from the same original database. Thus, the values in their columns will likely not match with one another. This leads us to the conclusion that it would not be reasonable to use both of these tables in the same query.

Based on this observation, we implemented a simple pruning algorithm, which is turned on if the database has the aforementioned naming schema. For a given query we first extract the tables used in that query. For the example query in Listing 4.1 that would be *dorm_1_student*, *pets_1_Has_Pet*, and *pets_1_Pets*. We then calculate the longest common prefix from all of these tables. In our example, this string would be empty. If we have a query only using the last two tables, the common prefix would be *pets_1_*. With these examples, we now notice that if all tables are from the same original database, their common prefix will end with an underline. However, this is not a foolproof way of checking that all tables are from the same original database. Let's say that we have the two original databases *book_1* and *book_2*. Tables from these databases will have the common prefix *book_*. So although they do not belong to the same original database, their common prefix will end with an underline. This problem only arises if the database names themselves already have underlines. As we cannot know how long the database names are, defining a threshold on how long the common prefix has to be to exclude this special case is very hard to do. Therefore, we just accept these false negatives in the pruning, since we still have the possibility to rank them in a bad position afterwards.

The current implementation of this step only works on databases with the mentioned naming schema, and for these kinds of databases it already eliminates some wrong queries. Depending on the ranker those queries would otherwise maybe not be recognized as wrong queries, which could lead to them being ranked at the top. This would impact the quality of our system. Therefore, this small pruning step possibly already greatly improves the quality of the results.

4.5.2 Ranking Algorithms

There are different parts that we need to think of when ranking the rewrites. One part is the used ranking algorithm. There are different possibilities on how to rank multiple objects based on their similarity to another object. Some of the existing methods are simpler than others. Most of these ranking algorithms need some kind of similarity measure between two objects.

In our case, we want to rank the alternative queries based on their similarity to the original input query. This similarity somehow has to capture the intent of the queries. We will explain the single components and how they are put together in the following.

There exist a lot of different ranking algorithms for various applications in information retrieval. When thinking about which algorithm to use we thus have to keep multiple aspects in mind: Which algorithms are applicable to our use case? How complex is the algorithm and does it need training data? Are the results from the algorithm only ranked via similarity to the original query or also via heterogeneity?

We know that our ranking should work as follows: It should retrieve the top- k queries that are the most relevant rewrites for our original query. Here, the most relevant rewrites are those that keep the intent of the original query. Thus, we somehow want to rank based on some intent-similarity measure. Moreover, since there could be multiple options for valid rewrites of a query—and we want heterogeneous results—we could use a ranking algorithm that not only favours relevant queries but also penalizes queries for being too similar. Such a ranking algorithm would fulfil all conditions from Section 1.1. There are various ways to do this.

Since we do not have training data using something like *Learning to rank* as explained by Liu [50] or probabilistic retrieval models will not work in our case. Moreover, using the result of the queries to rank them, e.g., as done by Li and Xie [48], is also infeasible since in intent-oriented query rewriting, the result of the rewrite does not need to be the same as the original one. We can however use simple ranking algorithms like maximum marginal relevance (MMR)—designed by Carbonell and Goldstein [15]—or different clustering methods for our ranking. Another option is to use LLMs for the ranking. A survey by Zhu et al. [93] summarizes how LLMs can be used as rankers without existent training data. With all of these options in mind, we decided on the ranking algorithms described in the following paragraphs. For most of the ranking algorithms we need different similarity functions. These will be described after the ranking algorithms in Section 4.5.3. We will split them into intent-based similarities and similarities between alternative queries (i.e., SQL similarity), two descriptions we will also use in the following explanations. In our implementation, the described ranking algorithms can be used in combination with all of the later described similarity functions.

Simple Sorting

The first ranking method is a simple sorting. It sorts the alternative queries based on a similarity measure to the original query. Thus, the query that is the most similar will be ranked first, the second most similar will be second and so on. The similarity measure for this is exchangeable, however, it should measure the intent of the query. This simple sorting does not take the heterogeneity of the results into account, thus not completely fulfilling the needed conditions from Section 1.1.

Maximum Marginal Relevance (MMR)

To fulfil all conditions, we therefore decided to implement MMR from Carbonell and Goldstein [15]. The main improvement compared to the simple sorting is that MMR also takes the similarity of elements to the already retrieved ones into account. It thereby boosts the heterogeneity of the results. To do this MMR uses two similarities: the similarity to the original query and the similarity between the retrieved elements, in our case also SQL queries. Therefore, this ranking algorithm needs a similarity measure for the intent similarity to the original query, but also a similarity measure to check how similar two alternative queries are. Using those two similarities we are able to iteratively find the next SQL query in the ranking using the MMR formula until we have our top-k queries.

Clustering Algorithms

Another option for the ranking is to use clustering algorithms. We first cluster the alternative SQL queries to find out which of them are similar to each other. For the sake of a heterogeneous result, we can then take only one or a few queries from each cluster into our result. We implemented this algorithm in two different ways using two clustering algorithms.

There exist different clustering algorithms. Since we do not have a lot of alternative SQL queries we looked at algorithms usable on small datasets. One guideline is given by Weigand et al. [86]. They suggest three different algorithms: K-means (e.g., as described by Lloyd [52]), DBSCAN (described by Ester et al. [22]), and some hierarchical agglomerative clustering (HAC) algorithm. From those three, HAC and k-means show similar results, while DBSCAN has a worse result. This is due to their data having one big cluster with high density. Since we do not know what the SQL queries from the LLM will be like, we therefore decided to try k-means and DBSCAN and did not implement HAC (as it had a similar result to k-means). For both algorithms, the implementation from the scikit-learn Python package [78] was used.

For k-means clustering, we had to transfer the queries into vector space. We did this using the same sentence embedding as in the table filtering (*multi-gamnet-base-cos-v1* [76]). With these embeddings, queries using similar tables and columns should be relatively similar to each other. Since we wanted to cluster similar alternative queries, we used this method. One important parameter of k-means is the number of clusters that should be calculated. We set this number to the amount of alternative queries we want to return. This way, we can return one query from each cluster, making the result really heterogeneous. To decide which of the queries of a certain cluster to return we picked the query from this cluster that was the most similar to our original query based on its intent. To do this we need some kind of intent-based measure. After getting the top query from each cluster we then rank the queries of the different clusters using our simple ranker with an intent-based similarity (since heterogeneity is already included via the clustering). Using this method we not only get the best query from each cluster but also rank the most relevant one at the top of our result list. There is however one problem with this method: Due to the nature of k-means, it would be possible to have a cluster consisting of a non-relevant outlier. This would then also be returned. However, due to our ranking afterwards, it would most likely still rank very low. This makes the k-means

approach feasible if a lot of heterogeneity is wanted in the result list.

The other clustering approach we implemented uses DBSCAN. In contrast to k-means, it does not define how many clusters it produces before but rather requires the amount of samples needed to form a cluster (first parameter) and the distance between two samples for them to be in one neighbourhood (second parameter). The second parameter was left at the default value (0.5). We set the first parameter such that each cluster has to have at least $\frac{\text{number of alternative queries}}{\text{number of output queries wanted}}$ elements. This would give us at most as many clusters as wanted output queries, thus enabling us to take at least one query from each existent cluster, boosting heterogeneity. However, to make sure that the minimum cluster size does not get too big, we used the minimum of the aforementioned calculation and five. This could lead to more clusters than wanted output queries, but it keeps the possibility of forming smaller clusters, avoiding too many outliers in the algorithm. Since small clusters can also have relevant queries, we chose to add this detail to our code. For the distance between the alternative queries during the clustering, we use the inverse of an SQL similarity measure. After calculating the clusters using DBSCAN we go over the different clusters: For each of them, we choose the query that is the most similar to our original query based on some intent similarity measure. We then take these top queries from the clusters and sort them also based on some intent similarity to the original query. We repeat these two steps as long as there are remaining queries in some of the clusters or until we find our top-k queries. If we still do not have our top-k queries after this we continue with the outliers. We rank them according to intent similarity to the original query and add them to the result to reach the wanted amount of queries. The described process has the following effects: Taking the queries from the different clusters again ensures a lot of heterogeneity. However, since it can happen that only one cluster is computed, it is possible that the heterogeneity is lost. By sorting the queries based on some intent similarity to the original query we still make sure to choose the most relevant queries first. Moreover, due to the nature of DBSCAN, we eliminate outliers. This can be an advantage if the query is an outlier due to being completely irrelevant and thus not being suggested often. It can also be a disadvantage if the query is an outlier but still relevant (e.g., if the query is so complicated that it was only written once by the LLM). So while possibly helping with irrelevant queries that are outliers, DBSCAN may sacrifice the heterogeneity of the result if too few clusters are detected.

LLM Ranking

For the last approach, we decided to include the usage of an LLM, to see if it would perform better than the more traditional ranking methods. In a survey by Zhu et al. [93] they mention three different unsupervised methods where LLMs are used as rankers. However, only two are applicable to rank multiple queries among each other compared to an original query. Since we want to sort all queries based on their similarity to the original query we used the so-called listwise method. Here we ask the LLM to return the alternative queries in a ranked order. According to the survey, this method is really performant compared to just asking the LLM for pairwise rankings (i.e., asking which of two queries is more similar to the original one). For listwise ranking, Sun et al. [84] and Ma et al. [54] suggest prompts for the LLM. However, we wrote a

new prompt, since our use case is a bit different to theirs. While the mentioned authors try to rank text passages for their relevance to a query, we try and rank SQL queries to an original query based on their intent. Thus, we wrote the prompt found in Appendix A.7. We tried to make the LLM think step-by-step (i.e., make a chain of thought), a technique mentioned in Section 2.1.4. When the queries are returned in ranked order they can be processed. We check for each returned query if it is in our list of alternative queries. If the LLM makes slight errors in its return, we just take the query with the least string distance (according to the inbuilt Python difflib similarity [27]) to the returned query. We let the LLM return full queries instead of just returning numbers for the queries in the hope that the LLM will use more reasoning and with the knowledge (from testing) that LLMs are not good with handling numbers. After all these steps we end up with a ranked list of queries. As mentioned in Section 2.1.1, LLMs cannot really count, therefore it can happen that not all alternative queries are ranked and returned by the LLM. For a small amount of alternative queries, this happens seldom, however, it can happen for a bigger amount of queries that need to be ranked. If the wanted amount of ranked queries is smaller than the given amount of alternative queries, the LLM normally returns enough queries. Thus, in this approach, we either need only a few alternative queries or enough to get at least the top-k. In both cases, we have a ranked list with enough queries in the end, which can be used in the next step.

4.5.3 Similarity Measures

After deciding on the ranking algorithms the next big question is: How do we measure similarity? As already described above we not only have to find possibilities to measure how similar two queries are regarding their intent, but also how similar two alternative queries are. The first measure is to identify which alternative queries have the same intent as the original query. Here, we want to find queries that are not necessarily structurally equal, but that have the same intent, as it is still important to keep that in the rewrites. On the other hand, we still want heterogeneity in the results. For this reason, we need a second similarity function. We cannot take the intent similarity, since we still want our results to have the same intent as the original query, so all alternative queries should be really similar regarding their intent anyway. Here, we want to find out if two queries do the same, i.e., use the same tables and columns in the same clauses. If they do, they should be regarded as similar, since they use the same idea to get the wanted result. Thus, we somehow need to measure if the query’s tables and columns are similar, i.e., measure the SQL similarity.

Intent-Similarity Measures

For both of these similarity measures, there are different approaches in literature. Some of them were discussed in Section 3.3. Although there are multiple approaches for intent-based similarity, most of them do not apply in our use case. Some approaches use the similarity of results, e.g., Borodin and Kanza [10], or documents snippets, e.g., Guo et al. [34]. We do not have something like document snippets of relevant results, thus we cannot use this approach. Moreover, there is no guarantee that the results of the rewritten query are the same, as explained in Chapter 1. Thus, we came up with the following measures for

intent-based similarity between an alternative and the original query:

1. Using the similarity of table embeddings between the queries
2. Using an LLM to calculate similarity scores
3. Using an LLM to get the intents and then comparing those via embeddings

Some of these approaches share the same ideas and thoughts as the approaches in the table filters.

For the first similarity measure, we used the same approach as in our embedding table filter described in Section 4.3.1. Here we compare tables to each other based on their spaCy [35] embedding using a large model [82]. The idea behind this approach is that different queries with the same intent should still use tables that are related to each other in some way. Here, we try and capture this relation with the embeddings. However, this does not always work well, e.g., for income and rent. So, the complete intent is hard to capture with embeddings. The full similarity calculation can be found in Algorithm 2.

Algorithm 2 Similarity Calculation via Table Similarity

Input: Two SQL queries $q1$ and $q2$

Result: The similarity of the two queries

$q1_tables = \text{getSQLQueryTables}(q1).\text{tokenize}()$

$q2_tables = \text{getSQLQueryTables}(q2).\text{tokenize}()$

$similarity = 0$

$comparisons = 0$

for $q1_table$ **in** $q1_tables$:

$doc_similarity = 0$

$doc_comparisons = 0$

if $q1_table$ **in** $q2_tables$:

$doc_similarity += 1$

$doc_comparisons += 1$

else:

for $q2_table$ **in** $q2_tables$:

$embedding1 = \text{getSpacyVectorization}(q1_table)$

$embedding2 = \text{getSpacyVectorization}(q2_table)$

$doc_similarity += \text{cosineSimilarity}(embedding1, embedding2)$

$doc_comparisons += 1$

$similarity += \frac{doc_similarity}{doc_comparisons}$

$comparisons += 1$

return $\frac{similarity}{comparisons}$

In the algorithm, we again use our own tokenization for splitting the table names to be sure that the table names are split according to both snake case (*snake_case*) and camel case (*CamelCase* or *camelCase*) (both appear often in table names), as in Section 4.3.3. The main difference to the table filter algorithm is to not add tables to a result set, but somehow use the calculated similarity between tables to approximate the query’s similarities. As can be seen in the algorithm we compare all tables of the first query to those in the second query in a certain way: for each table we check if it is in the second query as well. If this is the case the similarity is 1 for this table. Otherwise, we take

the average similarity of the table to all tables in the second query using the embedding and cosine similarity. In the end, we average all of these similarities to get the final result. As one can deduce from the algorithm, the result depends on the order the queries are given.

We can see this using an example: Assume that the first query has the table *t1* and the second has the tables *t1* and *t2*. If we use them this way around the similarity will be one. The other way around it will be less than one due to *t2* being compared to *t1* as well. Thus, we have to be careful to give the original and alternative queries to the algorithm in the same order every time. Another option here would have been to compare the summaries of the tables produced in the table filter described in Section 4.3.4. However, since the parts of the system are completely interchangeable, it could be that such table summaries would not be computed before the ranking. This would mean a lot of computational overhead in a step whose main purpose is to review and sort the rewrites to double-check the LLM results (i.e., just doing post-processing after the rewrites are already produced). Thus, we decided against this option.

For the second similarity function, we wanted to test how accurate an LLM would be regarding similarity scores. Since LLMs can incorporate a lot of knowledge, we deduced that incorporating intent into a similarity calculation could be something they could do. Thus, we gave it the alternative queries and the original query. The task of the LLM was to assign similarity scores to each alternative query according to the intent-based similarity to the original query. The prompt for this approach can be found in Appendix A.8. Here, we describe how exactly the LLM should measure the similarity and then ask it to give one similarity value between 0 and 1 for each query. We can then extract the values from the answer and use them for similarity. One problem in this approach is again the problem of counting. Depending on the amount of given queries, it can happen that not enough similarity scores are returned by the LLM (i.e., not one score per query). This problem can however be solved by giving the alternative queries to the LLM in batches. For a small amount of queries, returning a correct amount of scores works most of the time. The only drawback of this batched approach is the increase in runtime and tokens due to multiple API calls (each containing the instructions again).

The third and last similarity measure is based on the same idea as the table filter using summarization described in Section 4.3.4. In this table filter, instead of relying on the internal reasoning of the LLM, we wanted it to only do the first step of the process, i.e., identifying intents, topics and keywords. For the similarity measure, we now want to compare the alternative queries to the original query using their intent. Thus, we ask the LLM for the query intents of all of the queries using the prompts from Appendix A.4. With these, we get the intents as sentences from the LLM. Just like when comparing the table summaries to the intent in the table filter, we can now use the same sentence embedder (namely *multi-qa-mpnet-base-cos-v1* from SBERT [76]) as before (due to it being excellent for semantic search). Using this embedder we can then calculate the similarity of the query intents via the cosine similarity of the embeddings. But as just mentioned before there is one problem: LLMs cannot count. Thus, when we directly ask for the intents of multiple queries, the LLM will sometimes return too few intents. We also solved this by making batches of queries to give to the LLM (like above), accepting the drawbacks it causes.

SQL Similarity

Other than the intent-based similarity we also needed the SQL similarity between the alternative queries. As described above, there are approaches that use, e.g., structural similarity to compare SQL queries. One of these approaches is, e.g., the graph-based approach of Köberlein et al. [43]. However, some of these approaches are rather slow or complicated, or not applicable for us (as described in Section 3.3). Further metrics for SQL similarity are compared by Kul et al. [44], especially some that can be used to cluster SQL queries. Since we wanted some simple and fast comparison methods (to keep the runtime of this step low during, e.g., clustering) we took some inspiration from the query representations shown by them.

Thus, in addition to the described intent-based measures we used the following measures for the similarity between alternative queries:

1. Simple string comparison using the inbuilt Python library `diff` [27].
2. Comparing the used tables and columns from the different clauses.

The first measure is a very basic string comparison. If two queries are nearly the same regarding their strings, then they probably use a similar idea to fulfil the same intent, i.e., they use a similar query structure. Thus, just using a simple string comparison might already be enough to find out if two alternative queries use different approaches or not.

The second measure uses a certain query representation for the similarity calculation. As mentioned above this representation is inspired by the query representation presented by Kul et al. [44]. For our representation, we extract a few different parts from the query. Let's take the queries in Listing 4.2 as an example.

<pre>1 Query 1: 2 SELECT name 3 FROM students 4 WHERE residency = 'KL' 5 ORDER BY age</pre>	<pre>Query 2: SELECT ID, name FROM students WHERE age > 20</pre>
---	---

Listing 4.2: Example Queries for SQL Similarity Calculation

First, we extract all the tables used in the query in one set. For our example, this is just the table *students*. Then we extract all the table columns used in all the clauses of the query into a second set. This would be the set containing *name*, *residency* and *age* for our example. Finally, we extract the table columns used per clause, i.e., save a mapping from each clause to the used columns in that clause. In our example we therefore save the mapping: *select* → *name*, *where* → *residency*, *order_by* → *age*. We do these three steps for both alternative queries. To compare the sets we use the Jaccard value [39], i.e., the size of the intersection of two sets divided by the size of the union of the two sets. We first calculate the Jaccard value of the two sets of tables and of the two sets of columns separately. We then go over the different SQL clauses in the produced mappings and calculate for each clause the Jaccard of the table columns used there. After going over the SQL clauses we calculate the average of the Jaccard

Ranking Algorithm	SQL Similarity	Intent-based Similarity
Prototype (MMR)	string comparison	string comparison
Simple Sorting	-	all three possible
MMR	both possible	all three possible
DBSCAN Clustering	both possible	all three possible
K-means Clustering	-	all three possible
LLM Ranking	-	-

Table 4.1: Combinations for the Ranking Component

values of the clauses to get one combined Jaccard from the clauses. As a last step, we calculate the average of the three overall calculated values: the Jaccard from the tables, the Jaccard from the columns, and the combined Jaccard from the clauses. This gives us a similarity value for the two queries based on the used tables and queries. Since we want to find queries that use the same idea to get the desired information this should give us a good approximation for a similarity.

Both of these approaches are relatively easy. They are not made to capture any intent, just to compare two SQL queries. As explained in Section 3.3 there are more sophisticated approaches to calculate query similarity, however for our use case a simple approximation might already be enough. Moreover, some of the existing approaches are rather slow. Therefore we only implemented the two described similarity measures for SQL similarity.

4.5.4 Combining Ranking Algorithms and Similarity Measures

The ranking algorithms and similarity measures can be combined in multiple ways. As described above, some ranking algorithms need two similarity functions (one of each type), while others do not need any similarity measures. A summary of all possible combinations for those is shown in Table 4.1. All of these combinations are possible in our implementation. This is the case, since we implemented all explained algorithms and functions in a way that the algorithms work with any given similarity function of the right type. This makes it possible to later evaluate all combinations in Chapter 5.

One thing we did not mention yet is the prototype. The first thing we implemented was MMR. However, at that point, we did not differ yet between SQL and intent-based similarity. So, the used similarity function in the prototype was only the inbuilt string comparison from Python [27] for both similarities.

Using one of these 20 combinations will then make it possible to rank the alternative queries according to their intent. It can happen that the ranking does not work, however, this is only the case for approaches working with the LLM. As LLM output can sometimes be unreliable, it can happen that the LLM does not return the desired output structure or the needed amount of something. In that case, we either need to repose the request, or accept that the ranking did not work in this case. However, when taking certain measures, e.g., asking for similarities in batches, it only happens rarely that a ranking does not work.

4.6 Query Correction and Execution

As described at the start of this chapter we want our ranked alternative queries to be executable. Thus, we want to check for each query if it is executable on our database and if not, we want to correct it if possible. This seems quite easy for syntax errors but gets a bit more complicated if the query has, e.g., wrong tables in it. We found an approach for query correction using an LLM by Pourreza and Rafiei [72]. They describe how a generated SQL query can be corrected by an LLM using a correction prompt. They observed that there is one kind of correction prompt that works best for GPT-4 [64], thus we deduced that it could also work quite well with the current GPT models. Inspired by this prompt we therefore wrote a self-correction prompt which can be found in Appendix A.9. We kept some of the aspects of the original prompt, like the first instructions and some of the information, i.e., the query, the database tables and the foreign keys. However, instead of providing instructions to correct the query we decided to provide the LLM with the error message from the database. To do the actual correction we apply the following steps:

1. Try to execute the query on the database.
2. If the query is executable return the result and continue with the next query, otherwise save the error message.
3. For n iterations:
 - (a) Try to correct the query with the self-correction prompt and the LLM.
 - (b) If the query is executable now return the result and continue with the next query, otherwise try again with the answer from the LLM.
4. If the query is not correct after n iterations it is deemed non-correctable.

These steps do not clearly separate the query correction from the execution. We could separate them more clearly by using the *EXPLAIN* keyword from SQL to test the query’s executability during the correction mechanism. However, if we want to display the results to the user anyway, we do not add any overhead by directly executing the queries that were ranked as a top- k query. If the user only wants alternative queries but no results, this could of course be changed. In theory, the algorithm could be applied to all of the alternative queries. But to avoid high costs and latency due to the LLM and database accesses we decided to only correct the top- k queries that should be given back to the user. As described at the start of this chapter this can lead to some problems if not all queries are correctable, as we will not have enough executable ranked queries in that case. However, from our experience, non-correctable queries do not appear often. To be safe it would still be an option to rank and correct a few more queries than the top- k to have a bit of a buffer.

If there are not enough executable queries to return k queries to the user we will abort the process and return an error message to the user. Otherwise, after this step, we are left with the top- k queries and their results. Thus, we can display those to the user, leaving him with multiple—hopefully relevant—ranked intent-oriented rewrites.

Chapter 5

Evaluation

For this evaluation, we want to compare the different implementations of the parts in our rewrite system. There are a few things to consider for the evaluation of the whole rewriting system. One of them is the question of relevant measures other than the runtime of the program. Due to the usage of an LLM, it is also interesting how many tokens were used in the prompts and in the return statements from the LLM. These tokens also define the cost of an approach—at least for LLMs that are not open source. Those measures can all be taken from the program execution. Another question is how to measure the quality of the rewritten queries. This is much more difficult since human intent is hard to grasp as a measurement. Moreover, intent-oriented query rewriting can be ambiguous, both due to an unclear user intent and due to multiple possible rewrites. These rewrites can then produce outputs that have the same information as the original query, but that are very different regarding presented tables and columns. Thus, using some kind of ground truth queries or results is not easy. Therefore we opted to make a user study to evaluate the quality of the rewrites. Using the results of this study as a ground truth, we were also able to evaluate the post-processing part’s quality.

We therefore have two different evaluation parts—measured metrics and user study results—and will split this chapter as follows: We will first go through the evaluation and comparisons regarding the measured metrics and then proceed to talk about the results of the user study and its implications. Then we can draw a conclusion.

5.1 Setup

Before going over the results we want to mention some general aspects for all the evaluation parts. All experiments were run on a system with an Intel(R) Core(TM) i5-8265U CPU and 16 GB of RAM. The connection used to reach the API was an Ethernet connection. The used database system in the implementation is DuckDB [25], which therefore was used in all evaluation executions. For all of the experiments where an LLM was used, we decided on using GPT-4o [65] (with the default API parameters). This was—at the time of starting this thesis—the most performant model from OpenAI [66]. We did some testing with GPT-4o mini as well (as it is cheaper), however, it did not perform as well as GPT-4o (e.g., it did not perform well with joins). Since we want to test what this system can achieve with a highly performant LLM, we thus opted to use GPT-4o only.

One problem, regardless of the used LLM, is that reproducing the results of LLMs is pretty hard, since they are non-deterministic (as already mentioned in Section 2.1.1). There are different options to make them more deterministic, at least for the OpenAI models (as written in their documentation [61]): One can either use the so-called temperature parameter or use the seed parameter.

The first parameter can make the LLM responses less random, while the second uses the same seed for the response instead of a random one. Both of these options could however worsen the performance of the model, depending on how they are set. Another option to make the results reproducible is to cache the GPT responses and use them later when trying to reproduce results. This could however mess with the timings and costs during the evaluation, and can also introduce a bias, as only one response is used. So the main problem is that the system relies on an LLM, which in turn relies on randomness for good results. Thus, making all the results from our experiments reproducible is not easy. The best option left is to write down the exact model used. Since the produced results from one model can vary, but are normally still relatively similar, this allows at least a reproduction of the results to some extent (as long as this model is available, which is a problem on its own). As already mentioned we used GPT-4o in all evaluation runs, the exact version being *gpt-4o-2024-08-06*. We also saved all of the produced Logs of the program for each of the runs for better comprehensibility.

The last important aspect is the used database for the tests. We wanted to have a lot of tables coming from databases with different topics. The other restriction was that the naming of tables and columns had to be meaningful, as assumed in the Problem Statement (1.1). After looking at some benchmarks from related topics (mainly NL2SQL) we found the benchmark Spider from Yu et al. [88]. The schema and data can be downloaded from their website [74]. The data contains 206 different folders, each with an SQL file including the schema definition. In total, there are 957 tables of 138 different domains. From the benchmark definition, it is clear that each of the folders is supposed to be its own database. However, for our use case, we wanted one big database with a lot of tables. Just putting all the tables from the databases in one big database was not possible due to duplicate table names. We thus used the idea from Section 4.3.5 to combine them: Each table got a prefix with the name of the database in front of it. Since all databases have unique names we could combine all tables into one database this way. The benchmark also offers corresponding data to the tables. However, our algorithms do not need the data, only the schema, so we did not fill the tables with any data. This prepared database was then used in all the tests.

Using this setup we measured the following metrics:

- Runtime of the components
- Cost of the components (using the GPT-4o costs of \$2.5 per 1 million input tokens and \$10.00 per 1 million output tokens [67])
- Precision of the components

These metrics are the main metrics we measured during the evaluation. Depending on the component further metrics were measured, which are explained in the respective section. We evaluated the different components separately, looking at all of the implemented approaches. We will go into the details of the combinations and measured metrics for each part in the following sections.

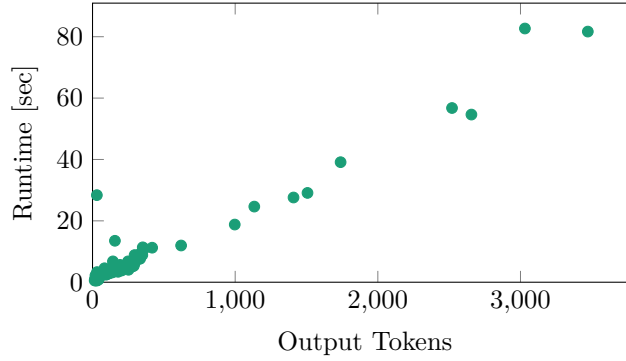


Figure 5.1: Latency of the API Compared to Output Tokens

5.2 Metrics Results

There are a few different metrics we measured for the various parts of our system. In general, we measured both the runtime and the token usage. We measured the input and output tokens separately to be able to also compare the cost of the API calls (the cost is calculated based on the cost of GPT-4o [67]). For a better comparison of the different implementations of each component, we will look at these measurements separately per component. For some of the different components we also made even more measurements, which will be described in the section of the respective component. To take all the measurements we ran each component configuration with 10 different queries (those shown in Appendix C). Thus, we got multiple measurements for each component for a more thorough evaluation.

As there are many components that use the GPT-4o API we also wanted to measure how large the latency is and if it is dependent on anything. The results can be seen in Figure 5.1. As we can see, the latency gets higher, the more output tokens are produced. Although there are some outliers, in general more tokens mean more runtime. We also looked at the input tokens to see if there is a dependency. However, we could not see a clear dependency between input tokens and runtime. It is still worth mentioning that—at least in our case—the number of input tokens is also often higher if the amount of output tokens is higher, e.g., if the LLM ranks the queries it also gets a large input (all alternative queries) before generating a long output.

5.2.1 Query Execution Check

This part of the system only has a single implementation and no LLM is used there. We only use the database to check if the query needs a rewrite. Thus, the only thing we measured was the time it took to check the execution on the database. This is mostly dependent on the size of the database and the latency of the used database. Both were not changed during the evaluation. As a result, all of the timings varied only between 0.4 and 0.6 seconds, with an average of around 0.56 seconds.

5.2.2 Table Filtering

To see how the different filters from Section 4.3 perform, we measured some different metrics for their execution. For the table filtering there are multiple interesting metrics. Since some of the approaches use an LLM we were able to measure the used tokens here in addition to the runtime for all of the different approaches. This makes it possible to compare the different costs of the methods. Furthermore, the amount of tables found in the filtering is an interesting measure for all filters. Moreover, we measured how many tables suggested from the LLM in the simple LLM filter presented in Section 4.3.2 were actually correctly written and not imagined. Another interesting filter with more statistics is the last one, i.e., the one using summaries (from Section 4.3.4). We will first go through the different measures separately, comparing the approaches, and then dive into some of the approaches in more detail. Since some of the approaches (namely the simple and complex LLM filters) have different options (either taking prefixes of table names into account or not) we will include those as well, thus evaluating 6 methods. To make the graphics clearer the filters are labelled as follows:

- F1 for the embedding filter
- F2.1 for the simple LLM filter
- F2.2 for the simple LLM filter using the prefixes of the table names
- F3.1 for the complex LLM filter
- F3.2 for the complex filter adapted for prefixes in tables names
- F4 for the filter based on summaries

Runtime

The runtimes of the 6 different approaches can be seen in Figure 5.2. As we can see they differ quite a lot from each other. While the first three filter types are all roughly in the same range, the last method takes roughly 6 to 8 times as long. One reason for this could be the amount of comparisons done during the filtering process. While all of the filters have to do some kind of comparison, the last filter method compares three different aspects of all available tables to the query using two different embedding methods. Doing this for nearly 1000 tables takes a while. Although F1 and F3 also compare the query tables to either the database or the proposed tables from the LLM, they only use a single embedding method for one comparison (or multiple comparisons for F3.2). As F4 is the only one using a different embedding, this could be another factor for the runtime. To add to the runtime, F4 has to make multiple calls to the LLM to get all the needed information about the input query. Since the API has some latency when called this also adds to the runtime. All in all, this makes the other filters a lot faster. The fastest approach is the second filter. Here, we do not need a lot of comparisons, since we directly ask the LLM to give us the relevant tables. The only comparisons in this approach compare the return of the LLM to the database tables. However, if those are accurate (which they mostly are, as explained in Section 5.2.2), then the comparisons do not take long, as the tables are directly found.

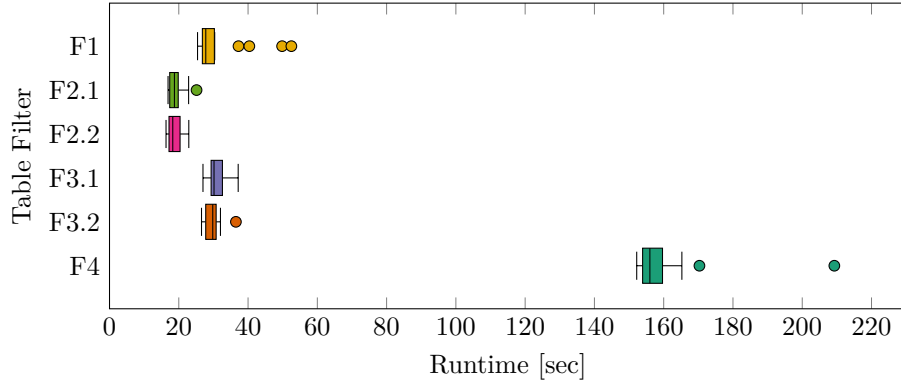


Figure 5.2: Runtimes of Table Filters

As we can see none of the approaches is faster than 20 seconds on average. This is probably due to the fact that we have nearly 1000 tables in our database, i.e., there are a lot of comparisons to do and the LLM latency can get higher for longer prompts.

Tokens and Cost

The next measurement is the comparison of the number of in- and output tokens used by each filter on average per execution. Using this measure we also calculated the average cost of the filters. Both of these measurements can be seen in Figure 5.3.

There is one filter without any cost, namely F1. As it just uses embedding and no LLM, there is no token usage connected to this filter. Thus, regarding cost it performs the best. As we can see the worst filter regarding tokens and cost is by far F2. This was to be expected since this filter gives all of the existent database tables to the LLM in each call. Therefore, it uses a lot of tokens, making it really costly.

All of the other filters are quite comparable regarding their token usage and cost. F4 uses a few more tokens than F3 because it has to make multiple requests to the API to get all the information on the query.

Generally, we can see that although there are only a few output tokens, the cost of them is quite high in comparison. This is due to the fact that the output tokens are four times as costly as the input tokens.

Number of Tables

The goal of the filters is to reduce the number of tables given to the rewriter to a reasonable number of relevant tables. Thus, we will also take a look at how many tables are selected by each filter. This is displayed in Figure 5.4. Although it is hard to say how many tables are enough or too many for a query, comparing the number of tables returned by the algorithm can tell us something about the selectivity of the filters. Moreover, we can see how much the algorithms generalize from the original tables. Since there is a high variance depending on the given input query, we decided to split the plot to see the number of tables selected per query and filter.

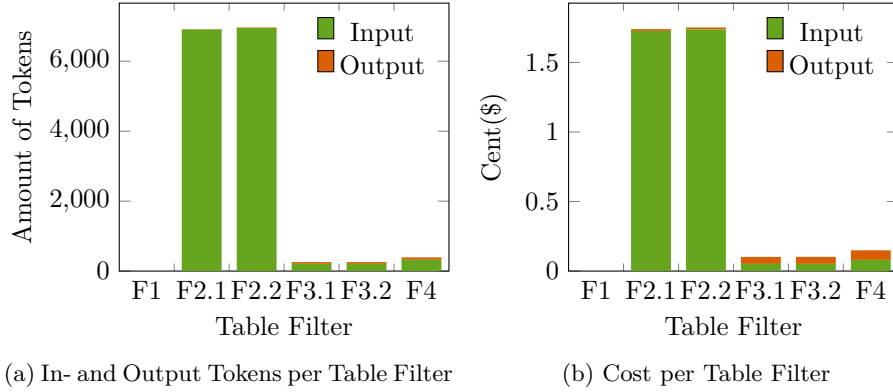


Figure 5.3: Tokens and Costs of Table Filters

As we can see the amount of tables deemed relevant not only varies between the queries but also between the approaches. Both F3 and F4 find a lot of tables compared to the first two approaches. While the first approach often finds zero tables, the second one finds at least some tables. Here, it varies if variant F2.1 or F2.2 finds more tables depending on the query. For the third approach the variant using name intersections, i.e., variant F3.2, finds more relevant tables for each query than variant F3.1. This is probably due to the fact that our database actually uses prefixes in table names, therefore all tables from one original database are selected if the LLM suggest a table with the original database’s name. F4 relatively consistently finds a moderate amount of tables. There is one peak for Query 3, which can be explained by the topic of the query (as later explained in Section 5.2.2).

Looking at all approaches we can definitely say that F1 is the most restrictive, followed by F2 (both variants), F4, F3.1, and F3.2 as the least restrictive one.

Filter 2

In this simple LLM filter, we ask the LLM which of the given database tables are usable. As mentioned in Section 4.3.2 we double-check that the names returned by the LLM are the real table names and not some hallucinated ones. To check how accurate the LLM is we measured how many of the suggested tables were correct, slightly off (i.e., having a high string similarity to existent tables) or completely wrong. After multiple executions with this filter, a total of 85 tables were returned. 81 of them were correct names from the database, none were slightly off, and 4 were completely wrong. So, 95.3% of the time correct tables were found. This is quite a high percentage, however, it is not perfect. This backs up our findings from testing, validating that double-checking the LLM is still an important step.

Filter 4

For Filter 4 we always showed the processing of queries itself in the plots. However, there is one further step that has to be done for this filter: getting the summaries for all database tables. We will talk about the statistics for this step here.

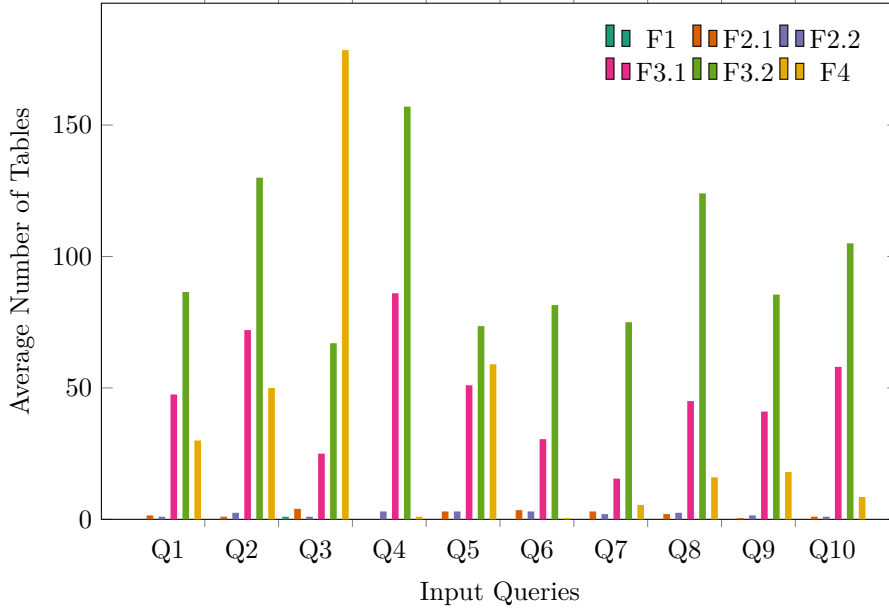


Figure 5.4: Tables found in Average per Query and Filter

The first measure is the time it takes for the filter to get all the summaries for the database tables, including the latency of the LLM API. In our case, this took a bit over 1100 seconds, i.e., more than 16 minutes. This is a really long time. Some of it is due to the latency of the LLM. As we make multiple requests to get all the summaries, this latency can add up. Furthermore, we need to process and save the summaries in a database, which also takes some time. Broken down to the time per table, this is just above 1 second, which is still a feasible amount of time for our number of tables. However, if there is a higher number of tables the runtime can get out of hand.

During the requests to the LLM, we also measured how many of the tables could get a summary during the first and second tries. In the first try 777 out of the 957 tables got a summary, i.e., 81.2%, which is an acceptable percentage. For the rest of the tables, the LLM produced some kind of unreadable output. Afterwards, the LLM returned a readable summary for all of the missing tables on the second try. It would probably be possible to increase the 81% by making the parsing of the LLM answer more adaptable.

All of these requests to the LLM also add up tokens and thus costs. For trying to get the summary of nearly 1000 tables, the LLM needed 60,669 input and 53,897 output tokens. This adds up to a total of 69.06 cents of cost. For the second try, it needed another 11,719 input and 10,709 output tokens. This adds up to another 13.64 cents. So in total 82.7 cents were needed for 957 tables. As this number depends on the amount of tables, summarizing all tables in a big database could get quite costly, e.g., if we look at 1,000,000 tables.

Another interesting measure is to look at how many tables are chosen via summary, topics and keywords. An overview of this can be seen in Figure 5.5. As we can see a lot of tables are selected via topics and also some via the summary.

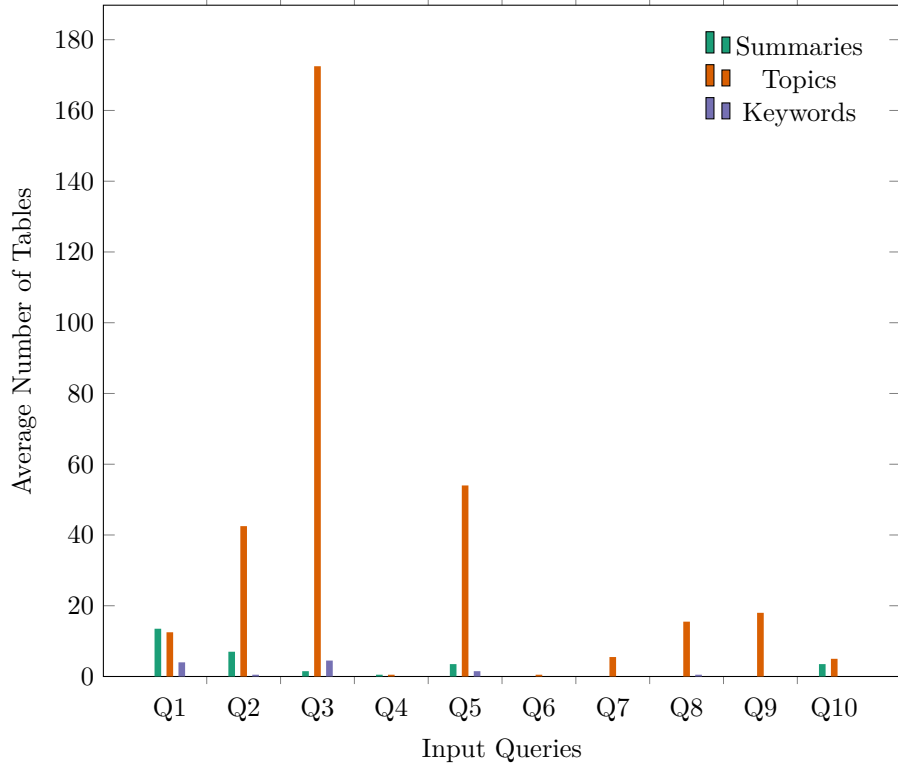


Figure 5.5: Tables found in Average per Query by Filter 4

The reason this number is lower for the summary might be that the intents of the queries are still relatively specific (as seen in Example 4.5) and therefore not too similar to the table summaries too often. The reason for the selection via keywords being that low might be due to the ordering in the code. If a word is similar regarding topics, keywords are not even considered anymore. Since topics and keywords are often similar (e.g., as in Example 4.6), swapping them in the code would most likely also interchange the seen results. Overall we can see a big variance in the number of selected tables, which is a result of the original queries having really different domains. As there is a different number of available tables per domain, the number of tables found also varies a lot. This can be seen for Query 3, where a lot of tables are found via the topic. Since this query is about meetings, a lot of different domains can be interesting, as there are meetings in a lot of fields. Therefore, this peak is quite high.

In conclusion, we can say that Filter 4 is doing a good job at what it is supposed to do regarding just the measurements: It filters tables and is pretty effective at producing table summaries. But both the runtime and the token usage for this step are pretty high, i.e., not good. However, as this processing step is only done once it can be pre-computed. Therefore, the increased time consumption is not included in the response time to the user. Moreover, the high cost only has to be paid once. Thus, if done efficiently, the needed pre-computation is not a big problem for this filter.

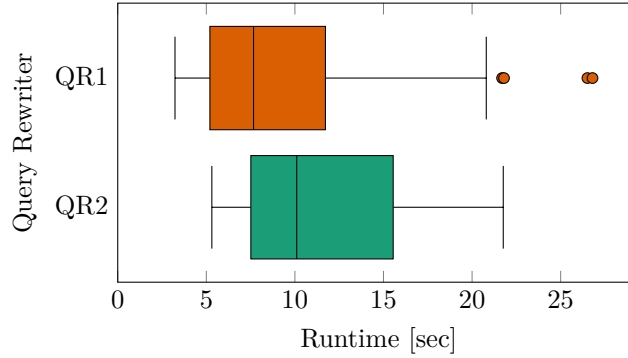


Figure 5.6: Runtimes of Query Rewriters

5.2.3 Query Rewriting

For the query rewriting, we measured the basic metrics runtime and token usage (since an LLM is used) to compare the two different approaches from Section 4.4. Another interesting metric here is how many rewrites were found (given that there were relevant tables) which could actually be used in the end, i.e., were not pruned or non-correctable. This can already tell us which prompt may have worked better for the LLM we use. Thus, we will already look at the number of pruned or non-correctable queries in this section. We will refer to these approaches as follows: the first approach is the simple rewriter (QR1), and the second approach is the one using natural language (QR2).

Runtime

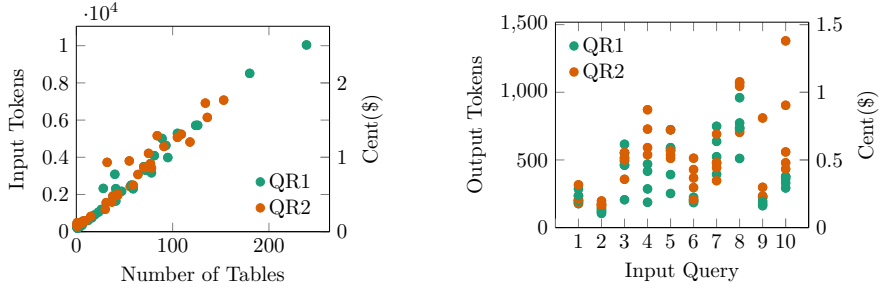
The runtimes of the two approaches can be seen in Figure 5.6. For the first approach, we can see that the average runtime is a few seconds lower than for the second approach. During the execution the second rewriter makes two calls to the LLM, first getting the query intent and then prompting for the rewrite. In comparison, the first rewriter only makes one call to the API. Thus, the latency to the API is included two times for the second approach. This explains the gap between the average runtimes. Depending on the wanted latency of the rewrite system for the user, these seconds can make a difference in the whole workflow.

We can also see that there are some outliers for the first approach, which take up to 26 seconds. A reason for this could also be the latency of the LLM API. Depending on the current amount of requests it might happen that the load on the API is high, making it slower and increasing the latency for each single call (as seen with the outliers in Figure 5.1).

Tokens and Cost

Just like for the table filter, we can measure the amount of tokens used and the resulting costs for both approaches. The results of the evaluation can be seen in Figure 5.7.

As seen in Figure 5.7a, the usage of input tokens is dependent on the number of tables chosen as relevant by the table filter. Since both approaches incorporate



(a) Input Tokens and Cost per Query Rewriting depending on the Amount of Tables found by the Filter (b) Output Tokens and Cost per Query Rewriting depending on the Input Query

Figure 5.7: Tokens and Cost of Query Rewriting Approaches

the usable tables in their prompt, this was to be expected. Compared to this the query itself—which is also part of the prompt—does not really make a difference in the used amount of input tokens (especially if there is a big amount of tables). While the first approach has two requests to the API, we cannot see that this has a big impact on the input tokens per query. As the request to get the intent is quite short (roughly 65 tokens plus the query tokens) it does not have a high impact compared to the table definitions in the rewrite request. Thus, we cannot see a high difference between the used input tokens of the approaches. Something to note in Figure 5.7b is that due to the second rewriter making two API calls, a higher amount of output tokens is used for the same query in most cases. Since the amount of output tokens is in total not as high as the amount of input tokens, we can see the difference the two API calls make (one returns the query intent, the second returns the rewrites for the query). Moreover, the amount of tokens used in the output is—at least for some queries—dependent on the query in both approaches. The reason for this could be that for some queries the complexity and thus length of the rewrites is quite similar for most rewrites, e.g., if there are not many different rewrite options or if the rewrite options are equally complex. An example for this is Query 2. Since it is a simple count over one table, most of the rewrites do the same for different tables. This results in similar rewrites in each run of the rewriter (in our case each run produces 10 rewrites). However, the query length can also vary a lot, e.g., for Query 8. This query is much more complex than Query 2 and has multiple rewrite options. We can see the same dependencies as described above in the cost of the LLM calls (as the cost is calculated from the tokens). Depending on the amount of tables and the query, costs can reach up to 3.5 cents for one rewriting iteration producing 10 rewrites (e.g., for over 200 tables and Query 8 for Rewriter 1). To keep this number low, a good table filter is needed to reduce the amount of input tokens. The cost of the output can only be reduced by producing fewer rewrites or by using the first rewriter since it is generally a bit less costly. However, as most of the queries cost between 1.5 and 2 cents, both rewriters are still feasible to use.

Number of Rewrites

To check how reliable the LLM is regarding its ability to produce executable and meaningful rewrites we wanted to check three different things:

1. Does the LLM always produce the wanted amount of rewrites?
2. How many of the rewrites will be pruned later on in the workflow?
3. How many of the rewrites need a correction to be executable?

These questions can be answered by measuring the amount of such rewrites in the workflow.

The first question can be answered pretty easily: It only happened once during nearly 200 executions that not enough rewrites were suggested by the LLM. This happened for Rewriter 2. Thus, it is really rare that not enough rewrites are returned by the LLM, at least for up to 10 demanded alternative queries. So, we can say that there normally is no problem with producing the requested amount of rewrites.

For the second question, we will look at how many alternative queries were directly pruned by our pruning algorithm presented in Section 4.5.1, i.e., deemed unusable before the ranking happened. This is already a small insight into the quality of the rewriters, as only low-quality queries are pruned. We looked at the performance of both Rewriter 1 and 2 during the multiple evaluation runs. For the first rewriter, 19 out of 190 alternative queries were pruned (10%). Considering that the LLM knows about foreign keys and thus about the connections between database columns, this is a pretty high number. For Rewriter 2 this number is even higher at 24 out of 180 rewrites, i.e., 13.33%, however not significantly higher than for the other rewriter. As there is only a small difference between the two results, and the LLM gets the database schema in both cases, the source of this error is probably the LLM. When looking closer at the results we noticed that more queries were pruned if there were more available tables to choose from. In all cases where queries were pruned (except one) over 40 tables were suggested to the LLM. As explained in Section 4.3 the LLM starts to hallucinate more if it is given a lot of options. Thus, it could be the case that more queries need to be pruned if there are too many tables available to choose from.

The last question is about the ability of the rewriters to produce executable and correct SQL using the given tables. To evaluate this we analyzed how many alternative questions needed a correction, and how many were so wrong that they could not even be corrected. For the first rewriter 59 out of 450 queries were not directly executable, i.e., 13.11%. However, only 9 out of these were so wrong that they could not be corrected, which is 2% of the 450 queries. For the second rewriter 42 out of 430 rewrites needed a correction, so 9.77%. Out of these 11 were non-correctable, i.e., 2.56% of the rewrites. From these numbers, we can deduce that both rewriters perform equally well when writing SQL. Thus, changing the LLM input from SQL to a query intent only has a small or no impact on the ability of the LLM to write executable SQL queries. Unlike for the pruning, we could not find a connection between the tables given for the rewrite and the amount of alternative queries needing a correction, i.e., there are errors both for a few and many tables given to the LLM. In total we can deduce the following: As only roughly 10% of the queries need a correction, and

most are correctable, i.e., not too wrong, the LLM seems to perform quite well in writing executable SQL queries.

5.2.4 Prune and Rank

While we only implemented one approach for pruning, there are multiple different approaches for the ranking algorithms and the similarity functions. Thus, we have different variants that we want to compare. Since we already looked at the pruned queries in the last section, we will not go into more evaluation of the pruning. To compare the ranking algorithms there are two metrics we measured: runtime and token usage. Both also include the pruning, however since it is always the same and does not use the LLM, i.e., does not need tokens, this does not distort the comparison results. Another measured metric only concerns the DBSCAN clustering algorithm described in Section 4.5.2. Here we looked at the amount of clusters that were determined and also at the token compared to the amount of clusters.

To compare all of the different ranking algorithms and similarity functions in the figures, we numbered them according to the used ranking algorithm and functions. Since we have 6 different ranking algorithms, the first number refers to the used algorithm: 1 for the prototype, 2 for the simple ranking, 3 for MMR, 4 for clustering using DBSCAN, 5 for clustering using k-means, and 6 for the LLM ranking. The next number refers to the used query measure between alternative queries (i.e., the SQL similarity): 1 for a simple string difference, and 2 for the more complex similarity using Jaccard on the different SQL components. For the last number, we have three options regarding intent similarity: 1 for the embedding, 2 for using the LLM for similarity scores, and 3 for combining the LLM and embeddings. These three numbers are combined with dots to form the approach. If a certain similarity function is not needed, a “-” is inserted instead (i.e., 5.-2 for k-means using the LLM to get intent similarity). The combinations of the different components are done as shown in Table 4.1, leading to 20 combinations in total. To make the distinction easier, all approaches using the same ranking algorithm are coloured in the same colour.

All of these 20 combinations were executed for 10 different original queries. For each of the original queries, there were roughly 45 alternative queries. So for each original query, each approach had to look at roughly 45 alternative queries, rank those, and identify the top 25 queries.

Before going into the evaluation results there is one important thing to mention: when testing the similarity functions using the LLM we noticed one drawback of it (already outlined in Section 4.5.3). If asked to give back a certain amount of similarities, it often happens that there are fewer numbers given back. The same happens if the LLM should give back a certain number of intent descriptions. So, the LLM is bad at counting. However, if the numbers are smaller (i.e., 10 or less) the LLM can still do it. Thus, our similarity functions that use the LLM will not ask for all similarities or intents at once, but make multiple API calls in batches. In our case, this reduced the amount of errors made by the LLM to 5 out of 60 cases for the second intent similarity function. It is hard to fully eradicate this error, one option to cope with it is to fill up missing numbers (at least when the returns are numbers) with, e.g., zeroes.

A big drawback of this method is that the runtime could potentially go up if

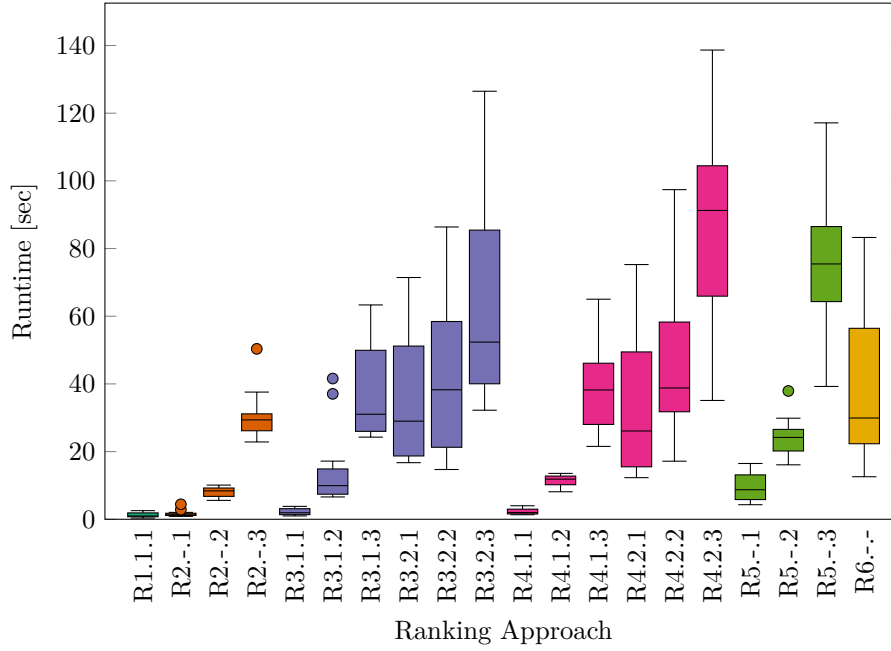


Figure 5.8: Runtimes of Ranking Combinations

the API takes longer than expected for the amount of output tokens (which can sometimes happen). Moreover, the cost is higher, since the prompt with the instructions for the LLM has to be given to it multiple times.

Runtime

The runtimes per execution of the 20 different approaches can be seen in Figure 5.8. As we can see the first prototype algorithm is fast, just like the first option of the second, third, and fourth ranking algorithm. The only algorithm that is a bit slower when using the same similarity functions is the k-means algorithm. This could be caused by the embedding the k-means algorithm uses to transform the queries into vectors. Another reason could be the runtime of the algorithm itself. So in total, we can say that the runtimes of the algorithms do not differ much for the same similarity functions. When taking a closer look at the similarity functions we can see that in all cases the first intent-similarity function is quickest, followed by the second and the third one. This is always the case, irrelevant of the ranking algorithm or the query similarity function. As the second and third function make possibly multiple LLM calls, this was to be expected. As explained at the start of this section, the API calls take longer the longer the output is. We can see this in the different runtimes of the second and third intent similarity function. While the second function only needs numbers, the third needs intents for each query. Moreover, in addition to asking the LLM for the original query’s intent, the third function also uses embeddings after getting the intents to calculate the similarity, making it even slower. So, in total it takes around twice as long as the second function. Regarding the SQL similarity function, we can see that the second version is always slower compared to the

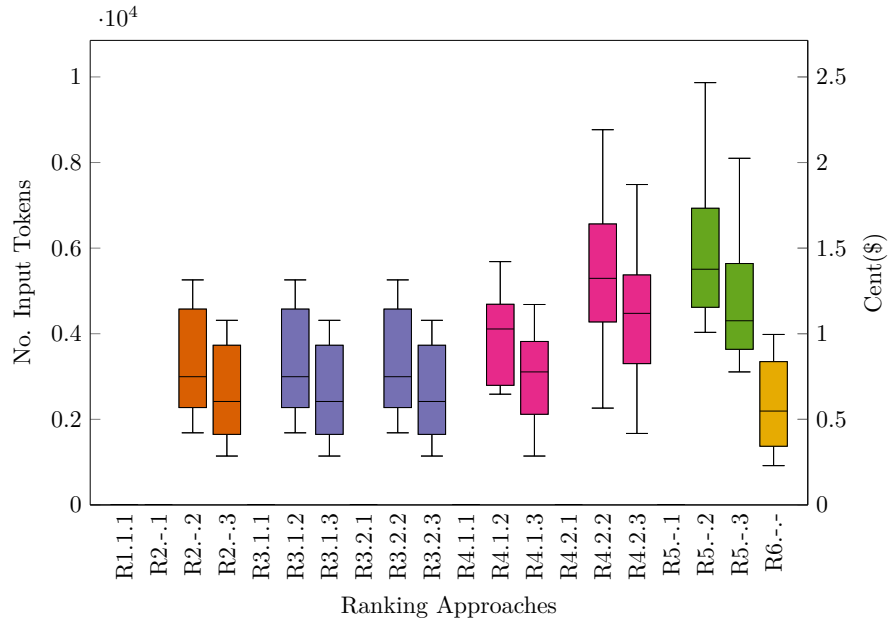
first one, which was to be expected, since it is more complicated than the first one, extracting different parts of the SQL query and using them in multiple calculations. As we can see this makes quite a big difference between the runtimes (really noticeable for the fourth ranking algorithm). Another noticeable thing is the runtime of the fifth ranking algorithm using the third intent similarity function. In our implementation, one API call is made for each cluster to get the similarities inside them and thus get the best point of the cluster (if there is more than one element in there). Since our k-means algorithm forms k clusters, with k being the amount of queries to rank (i.e., 25 here), there are many API calls to be made if many clusters have more than one query. This can happen for DBSCAN as well, however, it often forms fewer clusters (it still has a high variance in its timings). We will have a closer look at the effect of the number of clusters in DBSCAN in Section 5.2.4. One algorithm we have not mentioned yet is the LLM ranking. Just like all other approaches using LLMs, it has quite a high variance due to the API call. The latency can get quite large here, as the output consists of the ranked queries, and thus is quite large. However, compared to other variants its runtime is still in an acceptable range.

So, generally, we can say that a lot of API calls can potentially have a bad impact on the runtime and also introduce a large variance. Moreover, the first SQL similarity function is the fastest, together with the first intent-similarity function. The runtimes of the ranking algorithms vary depending on the used functions. However, when using the same functions they have similar runtimes. Another insight is that the runtimes can vary a lot for the same approach (for some approaches they do), which might be a result of the amount of alternative queries to rank and their length.

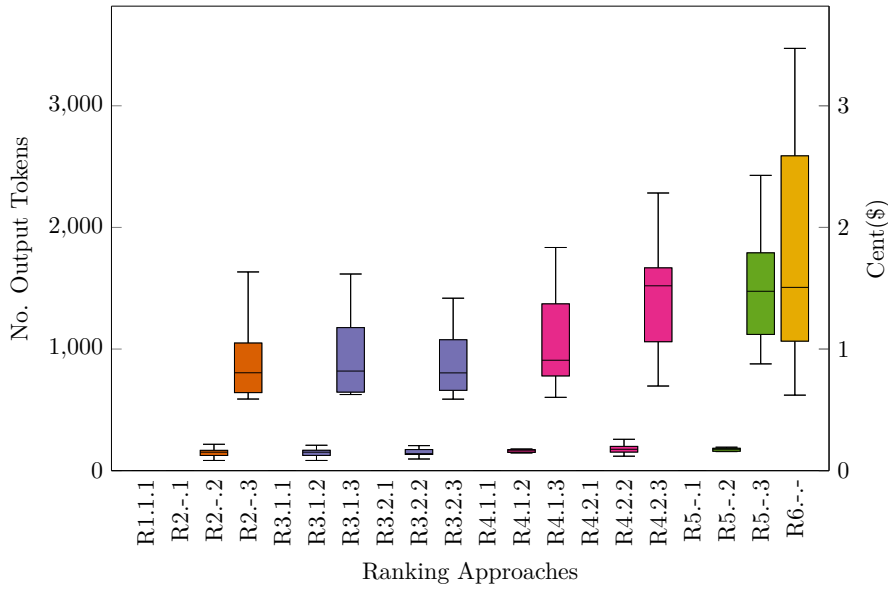
Tokens and Cost

The next thing we want to compare is the token usage and thus cost of the approaches per execution. These can be found in Figure 5.9 separated for in- and output tokens and cost. There are multiple approaches with zero tokens, which are all approaches not using an LLM, i.e., all combinations using only the first intent similarity function, regardless of the used SQL similarity function (and the used ranking algorithm except the last one). For all other approaches we can see that regardless of the used ranking algorithm, the second intent similarity function always has more input tokens and fewer output tokens than the third one. The input tokens can be explained with the length of the prompt: While both functions give a certain amount of queries to the LLM, the instructions for the LLM are much longer for the second function. So, although the third function needs to make one additional API request to get the original query's intent, it still needs fewer tokens. For the output, the third version has to return sentences describing intent, which is a lot more than just a number for each query. This is what the second function returns. So, there are quite large differences regarding the token in- and output of the different intent similarity functions.

Another thing we notice is the difference in used tokens and thus cost for the fourth and fifth ranking algorithms, i.e., the DBSCAN and k-means algorithm, when DBSCAN uses the first SQL similarity function. Here, we can see the dependency of the cost on the amount of clusters. For DBSCAN we will have a



(a) Number of Input Tokens and Cost per Ranking Combination



(b) Number of Output Tokens and Cost per Ranking Combination

Figure 5.9: Tokens and Costs of Query Ranking

closer look at this in the next section. For k-means, this is explainable—just like the runtime—with the amount of API calls made. Since there are 25 clusters, there are a lot of API calls if each of them has roughly the same amount of alternative queries (roughly 2 queries per cluster). To internally sort them, it can happen that one API call per cluster is needed, increasing the cost, as the instructions have to be sent with every prompt in the input. For the output, at least for the third intent similarity function, we can also see an increase. This happens as our algorithm does not save the intent of the original query, but asks the LLM every time it has to rank inside the clusters. This increases both the in- and output tokens. The last ranking approach has a comparably small amount of input tokens. Here, the LLM is given all queries and some ranking instructions in a single API call. As the intent similarity functions also give all queries to the LLM, but in multiple calls (thus duplicating instruction tokens needed), the last ranking approach does not need as many input tokens. However, as it returns the ranked queries, the amount of output tokens is really high. They also vary quite a lot, as they depend on the length of the alternative queries.

In total the different approaches do vary regarding input tokens and costs (between 0.25 and 2.5 cents per ranking execution), however, the variance in output costs is higher (between 0.2 and 3.5 cents). Also, within the approaches, there is some variance (at least in some), which is a result of the different query lengths and number of alternative queries. As we saw, some approaches do not cost a single cent. Thus, it is important to check if the price of some approaches is worth it for better quality. This will be done in a later section with the user study results.

DBSCAN Ranking

Since we noticed some differences in the runtime and tokens of the DBSCAN ranking algorithm, we took a closer look at it. As we can see in Figure 5.10 both the amount of API accesses and runtime are dependent on the amount of clusters formed. In these figures, the first number of the labelling stands for the intent similarity measure, while the second is for the SQL similarity measure. As we can see, the third intent similarity measure always has more API accesses than the second one. This is due to the additional access to get the intent of the original query, which happens each time the similarities in a cluster are calculated. Moreover, we can see that only a few clusters are formed by the first SQL similarity, and the more clusters are needed, the more API accesses happen and the higher the runtime. If there are more clusters (but still not a lot) then the similarities have to be calculated within more clusters (as each cluster probably still has more than one query). This increases the amount of API accesses, and thus the runtime (as each access takes some time).

For the second SQL similarity function a lot more clusters are formed most of the time. However, if few clusters are formed, the API accesses and runtime are still higher than for the first SQL similarity. This happens if the clusters have a relatively equal distribution of points for the following reason: After the ranking inside the clusters, the first elements of each cluster are again ranked to find an order. This is again done for the second element of each cluster and so on, until we have enough ranked queries. In our implementation, we recalculate the similarities for this. If the distribution in clusters is not equal, this only

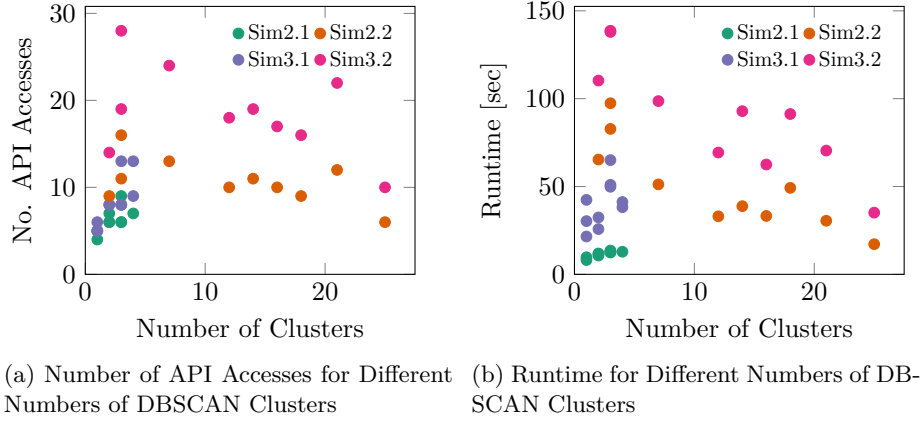


Figure 5.10: Dependencies on the Number of DBSCAN Clusters

happens a few times until all elements from the last cluster are chosen without comparison. If the distribution is equal, a lot of additional API accesses are needed, also increasing the runtime. Another thing to notice is that the API accesses and runtime decrease with more clusters for the second SQL similarity. If there are more clusters, some of them will have less elements. It can happen that the distribution is not equal between the clusters anymore. The more clusters that have only one element, the fewer comparisons inside the cluster and between the clusters are needed, thus decreasing runtime and API accesses.

5.2.5 Query Correction and Execution

Just like for the execution check at the start of the whole workflow, the query correction and execution only have one implementation. We will still have a look at the runtime and cost of this step. Moreover, we can measure how many iterations it took for a query to be corrected to see how effective the correction mechanism is. In the following, we will refer to correction iterations, where one correction iteration means one call to the LLM with the correction prompt for a single alternative query.

Runtime

One important thing to note for the runtime is that the overall runtime of this part can differ a lot, as some queries need multiple correction rounds while others do not need any correction. To avoid this bias we broke the timings down as follows: For each timing, we checked how many queries of—in our case—10 alternative queries needed a correction. We divided this into two categories: No correction was needed for any query in this iteration or at least one correction was needed. If no correction was needed we took the time as is, i.e., it displays how much time was needed to go through all alternative queries and check if they needed correction. This gives us an average of how much time is required for this necessary step. We then checked the iterations where at least one correction was performed. We deducted the average runtime to perform the execution check for all queries from the total runtimes (as this step is always

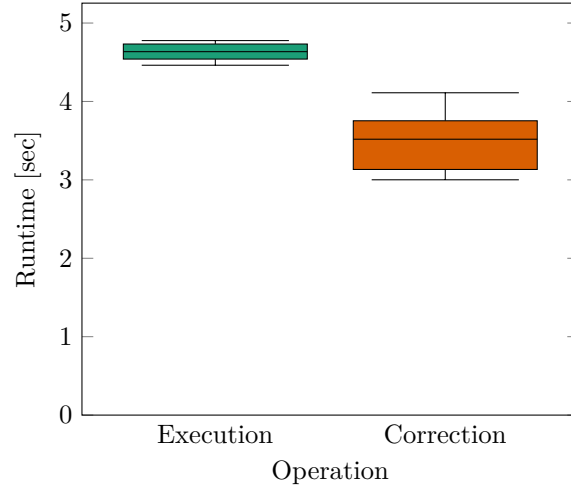
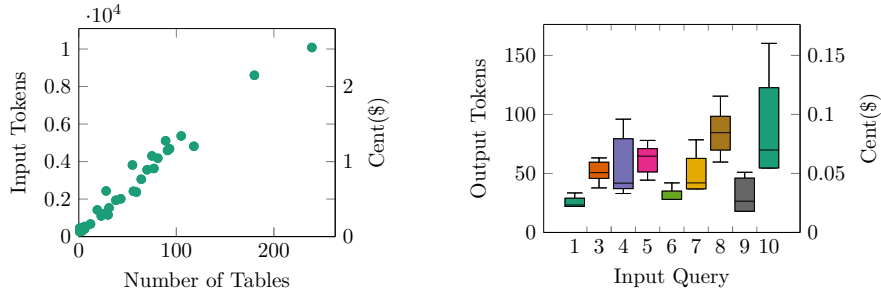


Figure 5.11: Runtimes of Execution and Correction

executed) and then divided the remaining time by the number of correction iterations. This way we can assess two things: How long does the process of checking and executing take? And how long does one correction need? The numbers for this can be seen in Figure 5.11. As we can see the total time for the execution check for all 10 queries is pretty stable at around 4.6 seconds. This mostly depends on the execution time of the query in the database, i.e., can be sped up depending on the database management system. In the figure, we can also see that per correction round roughly 3.5 seconds are needed. This delay is probably due to the latency of the LLM API since we do self-correction via the LLM. Moreover, the variance here is a bit higher, which is most likely also due to the API. Depending on the load and the speed of the connection the latency can vary a bit. Since the rewrites from the LLM only sometimes need a correction, as shown in Section 5.2.3, this correction time is feasible for a single correction iteration. However, it can still add up if multiple iterations per query are needed. Thus, the maximum amount of correction iterations should be kept low.

Tokens and Cost

A summarization of the tokens and cost for the correction can be seen in Figure 5.12. Since the correction prompt shown in Appendix A.9 also incorporates the usable tables, the length of the prompt is dependent on the number of tables that were chosen for the query, as seen in Figure 5.12a. Moreover, since the alternative query and its error message are also incorporated, the length is also dependent on the query length and the existing error. While the output tokens are only up to 160 in extreme cases, the input tokens scale linearly with the amount of tables deemed relevant. The amount of input tokens is always greater than 142, as according to the OpenAI tokenizer [68] our correction prompt itself has 142 tokens. As we can see, for roughly 100 tables we already have around 5000 input tokens needed per correction iteration. Due to this, the input tokens do not depend on the query or error message too much (as both are normally



(a) Input Tokens and Cost per Correction Iteration dependent on the Relevant Tables
(b) Output Tokens and Cost per Correction Iteration dependent on the Input Query

Figure 5.12: Tokens and Cost of Query Correction

comparably short). However, the output tokens depend on the query as seen in Figure 5.12b (Query 2 is missing as it never needed correction). While there is still some variance in some queries, most of them have a pretty small range for the amount of output tokens produced. We suspected this since the output is only the rewritten SQL query. If the original query is quite complicated and long, the output often is as well, while for short and simple inputs the rewrites are often also short. Thus, the output length depends on the original query in many cases (as explained in Section 5.2.3). It does not depend on the amount of tables, as only a small amount of them are used in the rewrite anyway.

Looking at the cost of the correction in Figure 5.12 we notice the following: The input tokens cost and the cost for output tokens vary a lot depending on the number of tables (which was to be expected after looking at the token amount). In the worst case, e.g., on our data, one iteration can cost roughly 2.3 cents (for query 10 and over 200 qualifying tables). This may seem little, however depending on the number of tables in the database and the amount of wanted alternative queries, this can add up quickly. So, just like for the runtime of the correction, we can say: One should be careful about how many iterations are used in the correction mechanism, even though not a lot of queries need a correction (as seen in Section 5.2.3), as the cost can add up. Moreover, filtering out tables before the rewrite is important for this step as well since costs can get quite high otherwise.

Effectiveness

To see how effective the correction is we can measure the amount of iterations needed per query to correct it, i.e., the amount of times the LLM needs to be prompted to correct the SQL query. We can see the number of iterations that are needed in Table 5.1. As we can see most queries are corrected in one iteration, there are only a few outliers that need 2 or even 3 iterations, which was the maximum of iterations allowed. Thus, we can say that the prompt for the correction seems to work quite well.

We also checked how many queries could be corrected with this mechanism. In total, there were only 20 out of 880 queries that were not correctable via this mechanism (around 2%). So, from the 101 queries that needed correction

Iterations x	Corrected Queries	Not Corrected Queries
1	97	34
2	11	23
3	3	20

Table 5.1: Amount of Corrected Queries after x Correction Iterations

around 20% could not be corrected. In 18 of these cases, this was due to the LLM repeatedly using tables that were not in the database. Together with the insight that most queries only needed one correction, we can say that the correction works quite efficiently.

5.2.6 Metrics Summary

As we saw depending on the used approaches for each of the components our runtime and cost can vary a lot. We can both choose fast and cheap or slow and pricey algorithms regarding all metrics, however, we have not checked them for quality yet, only for measurable metrics from the execution. So, regarding these metrics we can say that there are high differences between the approaches and most of them have some advantages and disadvantages (some are faster, while others are cheaper). Moreover, for the correction, we have to be very careful regarding the maximum number of correction iterations to keep both runtime and cost at a reasonable value.

As already mentioned we only used GPT-4o [65] for our tests. Depending on the LLM model both LLM latency (and thus runtime) and cost can also vary a lot. Regarding cost, this already happens for GPT-4o mini (the smaller OpenAI model), which is significantly cheaper than GPT-4o, as mentioned at the start of this chapter, but also performs worse. To reduce cost from over ten dollars to less than one dollar per million tokens, using a smaller model could however still be an option. In our case, this would have reduced the cost of the evaluation from roughly \$10 to less than a dollar.

5.3 User Study

To evaluate the quality of the rewrites we conducted a user study. Here, the main focus was to evaluate the quality of the table filtering and the rewriting in different combinations, to possibly find one or multiple configurations that outperform the other options.

Each of the users was given instructions (found in Appendix B) together with some rewrites. Those rewrites were generated as follows: We had 10 different original queries which can be found in Appendix C. The queries are separated into three categories (easy, medium, and hard) depending on how complicated the queries themselves and finding rewrites for them are. So, we assess how complicated it is to understand the query and—knowing the database and thus possible rewrites—how hard it is to find an acceptable rewrite. The reasoning for the category of each of the queries is as follows:

1. The needed column already exists in a fitting database. A rewrite needs the right table and column and the right order of the results. $\Rightarrow$ Easy

2. The challenge is to find the relevant tables, as there are a lot of tables related to students in the database. Moreover, the aggregation must still be correct. $\Rightarrow$ Easy
3. We want a calculation for the duration of a meeting, which is a column that has to be calculated from existing ones. $\Rightarrow$ Easy
4. The challenge in this query is to find the right tables and then perform a join and filter on them. Moreover, the full name needs to be assembled using first and last name. $\Rightarrow$ Easy
5. This query needs some calculation for the total budget, which is a bit more complex than before (the rewriter has to understand the meaning of the columns). Another aspect is to find the right join and apply a filter. This is in total a bit more complex than the queries before. $\Rightarrow$ Medium
6. For this query the rewriter needs to find a possible replacement for the customer reviews (there are multiple options in the database), which—depending on the column—needs a group clause. Moreover, a join and filter have to be performed. $\Rightarrow$ Medium
7. This query is a bit more complicated to understand due to the exists clause. There are multiple tables to join correctly while keeping the right filter and again combining the full actor name. $\Rightarrow$ Medium
8. We now have a longer and more complicated query, since we have multiple filters and also an aggregation in the query. The rewriter needs to find the correct tables and join them correctly, which is more complicated than before, as there are more tables to join. Moreover, the filter condition needs to be kept and the right columns have to be grouped for the correct accumulation. This is more complex than the queries before. $\Rightarrow$ Hard
9. Here we have a seemingly easy query. However, the concept of attractiveness of a district is relatively abstract and we do not have this measure in our database. So the rewriter has to find tables and columns to try and represent the attractiveness of a district. $\Rightarrow$ Hard
10. This query is a bit more complex due to the window function. First, the rewriter has to find some way to represent the popularity—which is an abstract concept not directly available in the tables—and second, also has to correctly identify and use columns for the window function (including a correct partition). $\Rightarrow$ Hard

For each of the queries, we executed our system in different configurations, to allow all possible combinations of the various algorithms implemented for each part of the system. These 12 combinations can be found in Table 5.2. The algorithms are those described in Sections 4.3 and 4.4, where *Prefixes* refers to the prefix version of some of the table filters. For the executions, we set the amount of alternative queries to be generated to 5. Thus, each original query got a maximum of 60 alternative queries.

Since the goal of the user study was to evaluate the rewriting part of the system, the ranker was not touched for these executions. However, the correction and pruning mechanism was used. This way we can deduce if non-correctable or

Number	Table Filter	Rewriter	Prefixes
R1.1.0	Embedding	Normal Rewrite	No
R1.2.0	Embedding	NL Rewrite	No
R2.1.0	Simple LLM	Normal Rewrite	No
R2.1.1	Simple LLM	Normal Rewrite	Yes
R2.2.0	Simple LLM	NL Rewrite	No
R2.2.1	Simple LLM	NL Rewrite	Yes
R3.1.0	Complex LLM	Normal Rewrite	No
R3.1.1	Complex LLM	Normal Rewrite	Yes
R3.2.0	Complex LLM	NL Rewrite	No
R3.2.1	Complex LLM	NL Rewrite	Yes
R4.1.0	Summaries	Normal Rewrite	No
R4.2.0	Summaries	NL Rewrite	No

Table 5.2: Configurations for the Rewriter Evaluation

pruned queries could still seem useful to the user. After generating the alternative queries they were shuffled and put into a single document per original query. This way we were able to eradicate all duplicate queries so that the users did not have to deal with them. Each user then received between 2 and 5 of these documents and the corresponding original query. They then evaluated which of the queries were relevant or irrelevant rewrites regarding the intent. They were not given any information on the database but were asked to imagine if tables could fit the intent. We marked a query as relevant if a majority of the users (more than 50%) found it to be relevant. Since we had 3 user assessments per query, in our case 2 users had to agree on its relevance. By combining the results of this user study we are able to get an insight into the quality of the different approaches, which we will describe in the following section. Since the rewriting is based on an LLM the results by no means fully show all possible results from the LLM. However, after multiple runs of the system, we concluded that the results of those runs are normally similar to each other. Thus, we used one representative run for the user study.

In addition to using the user study to evaluate the rewriting system, we also used it to evaluate the ranking algorithms and similarity functions. As described above we put all of the alternative queries for one query into a single document. Using these documents as an input we then take all possible combinations of ranking algorithms and similarity functions (20 combinations) to rank the queries per document. Using the results from the user study we can then assess which queries were relevant rewrites and thus use different measures to determine the quality of the ranking.

We will look at all the results of the user study in the following sections.

5.3.1 Rewriting Evaluation

We did the evaluation for the rewriters as described above. Thus, for each rewriting approach, we have 10 executions, i.e., one execution per original query. For each of them, we can calculate the precision, i.e., how many of the produced rewrites were relevant. These results can then be accumulated both for the approaches and queries. We will look at that in the following paragraphs. Since it

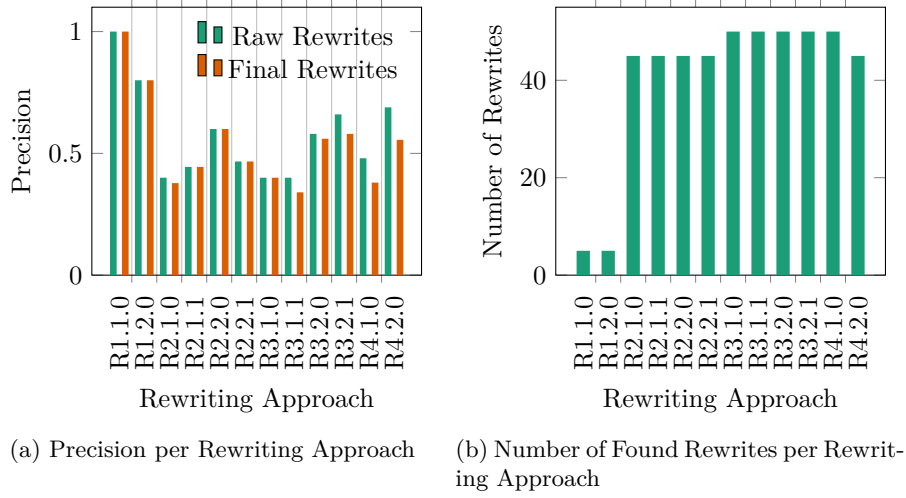


Figure 5.13: User Study Evaluation per Rewriting Approach

is also relevant for the quality of the rewriter if non-correctable or pruned queries were produced we both measured how many queries were deemed relevant by the users without looking at the fact if they were pruned or non-correctable and then took both options into account as well. One thing that is important to mention here is that not all approaches got rewrites for all queries, which also tells us something about the quality of the rewriting combination depending on the reason why no rewrites were found.

Quality of Rewriting Approaches

Looking at the user study results we now deduce the quality of the different rewriting approaches, regarding the amount of queries produced by each approach and how many of them are relevant. The results for this can be seen in Figure 5.13. We can see that the first and second approach seem to perform quite well regarding their precision. However, they each only produced 5 rewrites for one of the simplest queries, as visible in Figure 5.13b. This is due to their table filter not finding relevant tables since it was only the prototype filter using embeddings. Thus, both rewriters perform badly with this table filter.

All other configurations found more rewrites. Out of these approaches, we can see that those using the second rewriter often perform better than the same approaches with the first rewriter, i.e., the second one has a better quality. This already answers the question of the better rewriter, leaving us with the task of finding the best-performing table filter. Here, we can both look at the precision of the raw queries, i.e., before pruning and correction, or the precision of the final rewrites after pruning and correction. Both are shown in Figure 5.13a. As we can see there is no apparent configuration for the second filter that is better regarding the prefixes being used or not. So although we have prefixes in the database, the approach using it does not have an apparent advantage. A reason for this could be that the LLM does not really take the additional prompt part into account. For the third filter, the approach with prefixes seems to work better than the one without prefixes. Since we have prefixes in the database this

seems logical (the filter can thereby possibly find more relevant tables). Overall both Filter 2 without prefixes, Filter 3 with prefixes and Filter 4 seem to perform quite well. The filter that selects the most tables (Filter 3) performs pretty well. However, when taking pruning and correction into account, i.e., looking at the final rewrites, it is not the best anymore. This observation supports our statement that the LLM rewrites can lose quality if there are too many options. This can result in hallucinations or weird combinations that are pruned or do not work on the database. When taking this into account Filter 2 performed better than the third and fourth one. Both select multiple tables per query, but Filter 3 has more tables per query on average. Generally, we can still say that all three perform well, but we have to be careful if too many tables are selected. As already mentioned, there are some approaches where queries were not correctable or pruned. However, for none of the approaches it was a great percentage of the rewrites. Therefore, we can say that it did not have a great impact overall and the rewrites of the LLM seem to be relatively robust regarding those two things (if not too many tables are given, as explained above for Filter 3). In total, we can say that the second rewriter together with the second or third filter works the best when taking all quality measures (including pruned queries etc.) into account.

Rewriting Performance on Different Queries

Other than checking how well our approaches performed, we can also have a look at the performance for the different queries, i.e., how many rewrites were found for each one and how many of them were relevant. This can be seen in Figure 5.14. For most of the queries, a good amount of rewrites was found, which can be seen in Figure 5.14b. The only exception was Query 4. Here, there were many problems with finding the right tables for the query. Often no tables were found for the rewrite, thus directly stopping the process at an early stage. The reason for this may be the combination of topics. Since the query combines students and pets it was hard to find the tables that had both topics. As we can see in Figure 5.14a there were big differences regarding the precision of the rewrites for each query. The first 4 queries were deemed as easy, which is recognizable in the precision of the first three queries. The only easy query that did not work great was Query 4. Other than the fact that only a few rewrites were found, they were also not great, especially looking at the bar including non-correctable and pruned queries. Here, many filters found tables that either belonged to the student or pets topic. However, they did not find the tables having both topics. Therefore, the LLM hallucinated a lot to fill in the missing table, resulting in many bad queries.

For the medium queries, the rewriting was a bit worse than for the easy queries. The LLM did seem to have more problems here. This can especially be seen for Query 6, where a lot of rewrites were not relevant. Here, the LLM had two problems: First, there was a comparison for the book-length, which the LLM did not always correctly take into account. The second (and harder) problem, was to find something resembling customer reviews. There were only a few cases where this worked. One reason could be that the used tables were not good for doing this, or that the LLM did not correctly interpret the intent behind getting customer reviews.

When looking at the hardest queries we can see another drop in precision. The

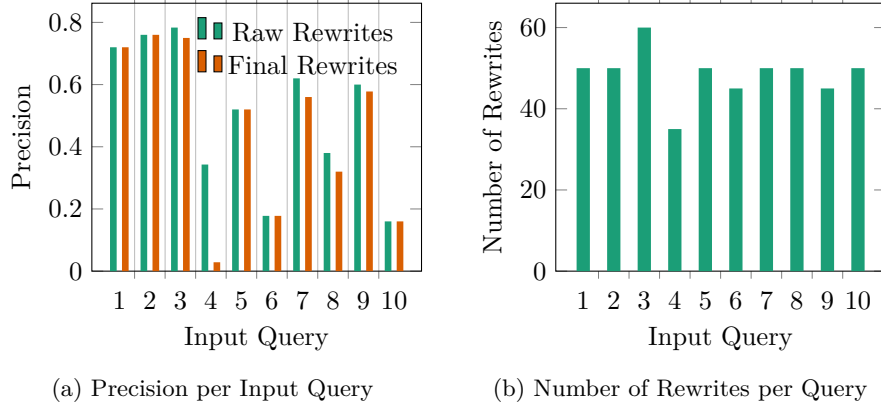


Figure 5.14: User Study Evaluation per Input Query

only exception is Query 9. The task here was to find something to resemble district attractiveness. A lot of the alternative queries used the population, which was deemed reasonable by most users. So, although there was not a lot of variance, many results were deemed relevant.

Overall we can say that there is definitely a difference in the performance of the LLM depending on the complexity of the rewrites. Generally, the harder we thought the rewrites to be, the worse the actual rewrites are. There are some exceptions to this, but they were all explainable (although not expected).

Other than for the fourth query pruning and non-correctability did not make a big difference. As explained above this could be due to this query bringing two relatively different topics together. So the quality and executability of the rewrites do not really differ due to the hardness of the rewrites. So, the LLM is good at using tables that actually belong together and writing executable SQL.

5.3.2 Post-processing Evaluation

As described before, we also used the user study results to determine the quality of the ranking algorithms. We will not look at the pruning or correction here, since those steps are mostly needed to check the quality of the rewriter. Thus, they are already looked at in the rewriting evaluation. Therefore, we only focus on the quality of the different ranking approaches. For this evaluation, we used the same combinations of ranking algorithms and similarity functions as described in Section 5.2.4 (shown in Table 4.1). In short, the first number stands for the used ranking algorithm, the second for the SQL similarity function, and the third for the intent similarity function. We also used the same setup with 10 original queries. For each original query, we requested a ranking containing the top 25 alternative queries (each original query had 33 to 56 alternative queries). We did this for all ranking combinations. We used the top 25 queries to be able to compare the quality at different points in the ranking, i.e., after 5 queries, after 10 queries, and so on.

We measured the quality using three different measures: We used Recall to check how many of the relevant queries from the rewrites were returned as a top 25 query. We also used Precision@k, calculating how many of the first

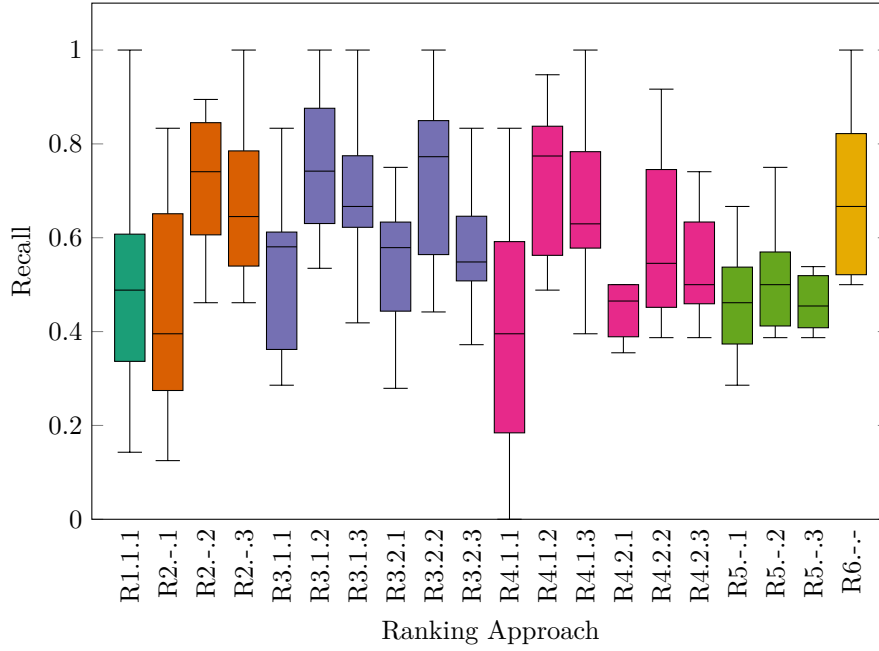


Figure 5.15: Recall of the Ranking Results per Approach

k elements were relevant and dividing it by k. The third measurement is to measure homogeneity. Since we want heterogeneous results we calculated the average similarity (regarding query strings) of the top k results. Thus, the higher the average similarity, the worse the heterogeneity of the results. These measures were taken per combination of original query and approach to get meaningful results regarding averages etc. We will talk about all of the three measurements and the results of the executions separately in the following paragraphs.

Recall

The recall of all the approaches can be seen in Figure 5.15. As we can see, there is a pattern of how many relevant queries were found by the ranking algorithms. The first thing one can notice is the pattern for the intent similarity function. In each of the ranking approaches, we can see that the approach using the second intent similarity function is better than the other two approaches. It seems that the LLM is quite good at evaluating if a query is similar regarding the intent and giving it a fitting score.

Looking at the SQL similarity functions, we can see that for the third ranking approach, the second SQL similarity function does not make much of a difference in recall to the first function. However, the combinations using the second function in ranking Algorithm 4 (DBSCAN) are in total worse than those using the first SQL similarity function. As the SQL similarity function is used to cluster in the DBSCAN algorithm, it seems like the clustering is worse using the second SQL similarity.

For the ranking algorithms, both the first and the fifth ranking algorithm do not perform well. For the first version, this was to be expected, since it is only a

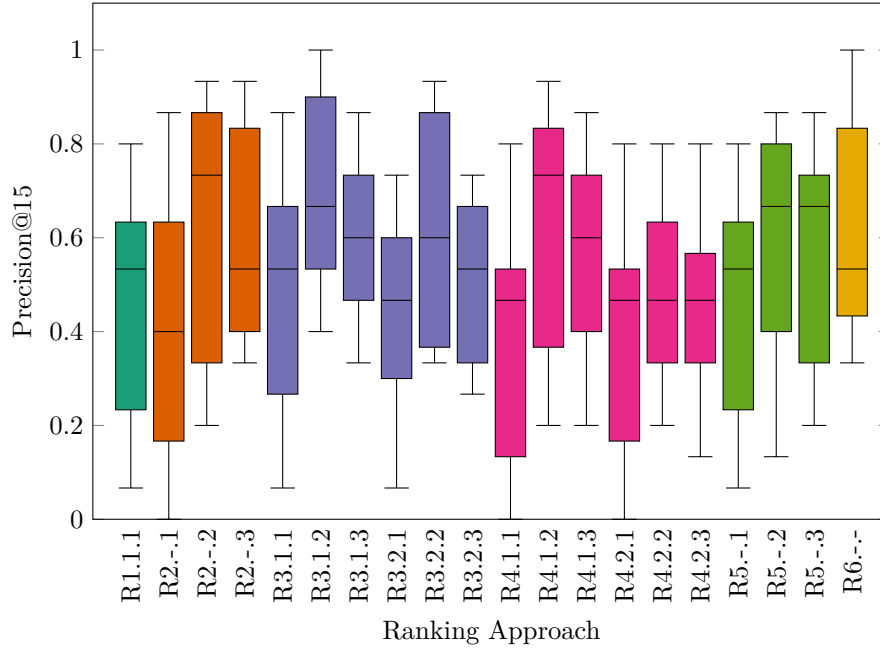


Figure 5.16: Precision of the First 15 Ranking Results per Approach

prototype (and embeddings do not work well for intent, as explained in Section 4.5.3). The k-means algorithm might not cluster well using the embeddings. It might be an option to change the number of clusters formed by the algorithm to improve it. Regarding the other ranking algorithms (simple ranking, MMR, DBSCAN and LLM ranking), they seem to perform equally well with the same similarity function configurations.

So in total, we can say that for the recall the second intent similarity function and the first SQL similarity function seem to be a good combination for simple ranking, MMR, and DBSCAN. Also, the LLM ranking performs nearly equally well (with a slightly lower average in the recall).

Precision

The second measurement we will look at is the precision. There are multiple options here: For consistency with the recall we could analyse the Precision@25, i.e., for all 25 ranked queries. However, we decided to visualize Precision@15 in Figure 5.16 since the patterns of the different ranking combinations are clearly visible here. The other measurements, i.e., Precision@5, Precision@10, and so on, show the same pattern but not as clearly recognizable as for Precision@15. So, although a user would most likely only look at top-5 or top-10 results, we choose to display Precision@15.

When looking at the results, we can see that the second intent similarity function is the best in all approaches, just like for the recall. Moreover, the first SQL similarity function is also better for MMR (third ranking) and DBSCAN (fourth ranking) than the second one.

Again, the first ranking approach is not the best, but since it was only a proto-

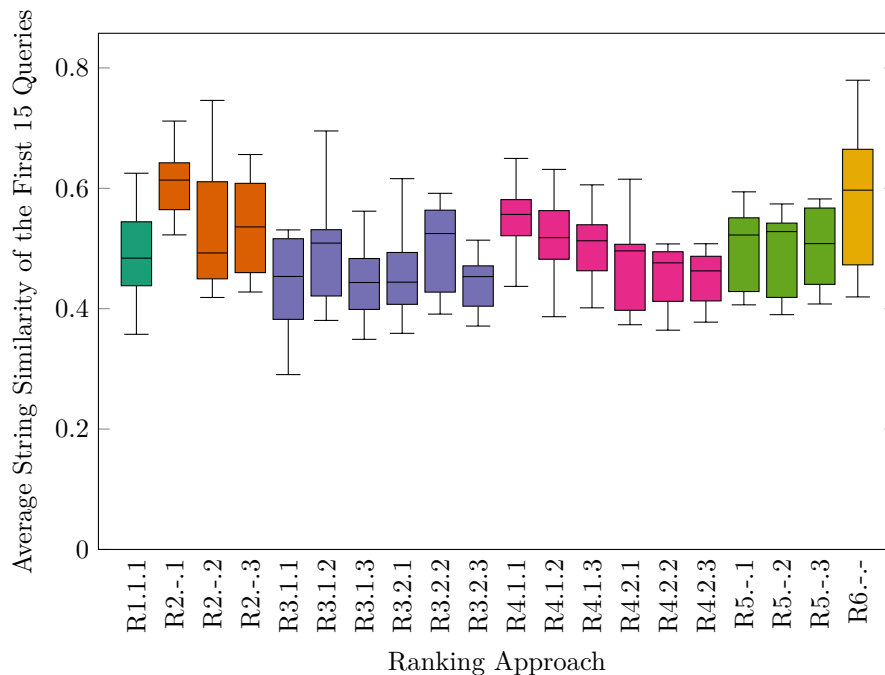


Figure 5.17: Homogeneity of the First 15 Ranking Results per Approach

type (and embeddings are not really good for measuring intent, as explained in Section 4.5.3), this is to be expected. The fifth ranking approach—k-means—seems to perform a lot better here than for the recall, however still not as well as the other approaches with the right similarity function configuration. This might still be due to the clustering into 25 clusters. If there are only over 30 queries this can get quite hard to do. The third ranking approach, i.e., MMR, seems to outperform the other approaches, closely followed by the simple ranking algorithm and DBSCAN together with the LLM ranking. So, even though MMR also looks at the heterogeneity of the queries—which could make it worse if there are not many distinct relevant queries—it still performs well in giving out meaningful alternatives as the first 15 queries.

Heterogeneity

The last measurement we will look at is the homogeneity of the ranked queries. As described above, we measured how similar the first 15 queries were to each other. The results can be seen in Figure 5.17. Since we want heterogeneous results, the higher the string similarity values, the worse the ranking. Here, we also measured the homogeneity at other points, e.g., measuring after 5 or 10 results. We also found the patterns to be the same as with the top-15 results, just like for the Precision@15. Thus, to get the picture of the first 15 queries more consistently we also opted to look at the first 15 queries here.

Out of our ranking algorithms, there are three algorithms which should decrease the homogeneity: MMR, DBSCAN and k-means. As we can see for MMR and DBSCAN (third and fourth approach) this worked quite well. There were still

some differences regarding the used similarity functions. The third intent similarity seemed to be the best for heterogeneity in both cases, while the second was the worst. Also—at least for DBSCAN—the second SQL similarity function seemed to perform better. However, the differences here are quite small compared to the other ranking approaches. K-means also performed reasonably well but still worse than the other two approaches for heterogeneity. Here, the second intent function was better than the rest.

Both the simple ranking and the LLM ranking did not perform well. For the simple ranking, this is to be expected, since it does not take heterogeneity into account. The LLM however had this task in its prompt, but it seems like this does not really work in a zero-shot setting. In contrast, the first prototype also performed well. As it uses the MMR this was to be expected as well. It, however, does not perform as well as other ranking configurations with different similarity functions than those from the prototype.

So, overall most of the approaches that take heterogeneity into account were able to make the results at least a bit more heterogeneous, regardless of the used similarity functions.

Conclusion

In all the measures we can see quite a large variance in the results. This is due to the different difficulties of the queries. For some, it was harder to find the intent and check which are similar during the ranking. Thus, the results varied quite a lot.

Overall there are big differences in the quality of the ranking algorithms. Without taking heterogeneity into account the best combinations use the second intent similarity and the first SQL similarity. The ranking algorithm can be chosen between simple ranking, MMR, DBSCAN and the LLM ranking. If we also take heterogeneity into account we would probably want to erase simple ranking and the LLM ranking from the list of possible options. This leaves us with MMR and DBSCAN as the best-performing ranking approaches regarding quality and heterogeneity. They both have a high precision and high level of heterogeneity, making them the best choices.

Chapter 6

Conclusion and Outlook

The goal of this thesis was to design, implement, and evaluate a prototype for an intent-based rewriting system. Other than existing rewriting systems the goal wasn't to have a query returning the same results, but to rewrite a query on the available data while still keeping the intent, allowing the user to get the wanted information out of the results.

With all restrictions and our goal in mind, we designed a workflow for a prototype system. Our implemented prototype has two main phases: The first produces the rewrites, while the second post-processes all available rewrites before presenting them—and the results—to the user. During the first phase (after checking that we actually need a rewrite) there are two main parts. These consist of a table filter, to reduce the amount of tables the rewriter can use, and the rewriter itself. In the post-processing phase, another three parts are needed: We first prune irrelevant queries and rank the remaining ones. After this, the queries are corrected (in case the rewriter produced non-executable SQL). Finally, the queries are executed on the given database. For most of the mentioned parts, multiple different approaches were implemented, many of them using an LLM.

After the evaluation, we came to the following conclusion: Most of the implemented approaches have individual advantages and drawbacks. There are large differences regarding runtime, costs and quality of the different parts of the workflow. For the table filters, we determined the two approaches using the LLM as the best approaches regarding quality, however, they either need a lot of tokens or are pretty slow. When looking at the rewriters we found that the rewriter with the better quality is both slower and more costly. Regarding the post-processing this was a bit different: The best-performing ranking combinations were also comparably fast and cheap, although there were better combinations regarding speed and costs. However, these performed worse regarding the quality. Overall the results showed that there is an important decision to make when working with the implemented system: How much time and cost are we able to invest for better quality of the resulting queries?

Regardless of the important question regarding the tradeoff between the approaches, we were able to show that it is possible to implement an automated system for intent-oriented query rewriting. While there is still room for improvement, the system was already able to achieve a precision of over 50% for easy and medium hard queries. This is a promising first step, which in the future could allow us to make intent-oriented rewriting an additional part of data analytics for incomplete or non-available data.

There are still a few different aspects that could either be improved or extended regarding the implemented system.

One possible extension would be to consider additional workflows, e.g., starting with a request in natural language instead of SQL. Another option would be to use traditional query rewriting on the input query as a first step to make the

query simpler and possibly easier to understand for the LLM. We could also adapt the output to show more information to the user, e.g., the query intent suggested by the LLM, or the queries in one cluster (for ranking algorithms with clustering). It would also be possible for our system to suggest tables the user could search for, such that they can be added to the database before doing the intent-oriented rewrite.

There are also different aspects to improve the current rewriting system (some already mentioned in Chapter 4). Since it consists of different parts, there is most probably room for improvement for all of them. Some ideas for those parts concern the table filter and pruning. Regarding the table filter, it would be possible to also take the data of the tables into account, not just the schema. This could further improve the assessment of the importance of the table, as some queries only look for a certain type of data, e.g., only data from a certain country. Furthermore, it would be possible to add tables after the table filtering, e.g., those tables that have foreign keys to the found tables or tables that connect two found tables via two foreign keys. For the post-processing phase there currently only exists one approach for the pruning. Further approaches could be added, e.g., pruning queries that join tables that do not have any foreign key to one another. Something like this would however only work with a correctly designed and deployed database schema. It would also be an option to improve the implementation of the ranking algorithms to avoid too many unnecessary LLM calls.

There are also some aspects that concern the whole system. It would for example be possible to combine multiple table filters or rankers, e.g., a first quick one and a second accurate one to improve the runtime and cost of an execution. Moreover, all of the LLM prompts are currently simple zero-shot prompts designed for OpenAI models. Since we use the LLM in many stages of our process this would be a great option for improvement. There are multiple opportunities here: It would be possible to improve the prompts using well-known prompting tricks. Furthermore, trying out the performance of different LLMs for this task is an interesting option. It would also be possible to not only use basic pre-trained LLMs but to do fine-tuning on one of them for this specific task (if enough data is available). Other than the LLM we also use embeddings quite often. Since there are a lot of pre-trained models, further testing could potentially show better embedders for our use case. It would also be possible to design a new embedder for the task of embedding SQL queries.

As described above there are multiple options to improve on the system itself. One other aspect that we stumbled upon when designing the system was the question of evaluation methods. We used a user study for this, but doing this is quite time-consuming. Therefore, it would be an option to further look into the different queries that could be rewritten and find other options for evaluation. Having some kind of benchmark for comparing different approaches for intent-oriented rewriting would be a big improvement for the future.

Bibliography

- [1] K. Affolter, K. Stockinger, and A. Bernstein. A comparative survey of recent natural language interfaces for databases. *VLDB J.*, 28(5):793–819, 2019.
- [2] F. Almeida and G. Xexéo. Word embeddings: A survey. *CoRR*, abs/1901.09069, 2019.
- [3] A. Anand, J. Leonhardt, V. V., and A. Anand. Understanding the user: An intent-based ranking dataset. In E. Serra and F. Spezzano, editors, *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, CIKM 2024, Boise, ID, USA, October 21-25, 2024*, pages 5323–5327. ACM, 2024.
- [4] Anthropic. Anthropic models. <https://docs.anthropic.com/en/docs/about-claude/models>. accessed on 14.02.2025.
- [5] N. Arzamasova, K. Böhm, B. Goldman, C. Saaler, and M. Schäler. On the usefulness of sql-query-similarity measures to find user interests. *IEEE Trans. Knowl. Data Eng.*, 32(10):1982–1999, 2020.
- [6] A. Aynedinov and A. Akbik. Semscore: Automated evaluation of instruction-tuned llms based on semantic textual similarity. *CoRR*, abs/2401.17072, 2024.
- [7] S. Bird. NLTK: the natural language toolkit. In N. Calzolari, C. Cardie, and P. Isabelle, editors, *ACL 2006, 21st International Conference on Computational Linguistics and 44th Annual Meeting of the Association for Computational Linguistics, Proceedings of the Conference, Sydney, Australia, 17-21 July 2006*. The Association for Computer Linguistics, 2006.
- [8] L. Blunschi, C. Jossen, D. Kossmann, M. Mori, and K. Stockinger. SODA: generating SQL for business users. *Proc. VLDB Endow.*, 5(10):932–943, 2012.
- [9] K. D. Bollacker, R. P. Cook, and P. Tufts. Freebase: A shared database of structured general human knowledge. In *Proceedings of the Twenty-Second AAAI Conference on Artificial Intelligence, July 22-26, 2007, Vancouver, British Columbia, Canada*, pages 1962–1963. AAAI Press, 2007.
- [10] G. Borodin and Y. Kanza. Search-by-example over SQL repositories using structural and intent-driven similarity. *Data Knowl. Eng.*, 128:101811, 2020.
- [11] M. Brencz. Python sql-metadata package. <https://github.com/macbre/sql-metadata>. accessed on 03.03.2025, Copyright 2025 GitHub Inc.
- [12] U. Brunner and K. Stockinger. Valuenet: A natural language-to-sql system that learns from database information. In *37th IEEE International Conference on Data Engineering, ICDE 2021, Chania, Greece, April 19-22, 2021*, pages 2177–2182. IEEE, 2021.
- [13] K. S. Candan, H. Cao, Y. Qi, and M. L. Sapino. Alphasum: size-constrained table summarization using value lattices. In M. L. Kersten, B. Novikov,

- J. Teubner, V. Polutin, and S. Manegold, editors, *EDBT 2009, 12th International Conference on Extending Database Technology, Saint Petersburg, Russia, March 24-26, 2009, Proceedings*, volume 360 of *ACM International Conference Proceeding Series*, pages 96–107. ACM, 2009.
- [14] J. Cao. Generating natural language descriptions from tables. *IEEE Access*, 8:46206–46216, 2020.
- [15] J. G. Carbonell and J. Goldstein. The use of mmr, diversity-based reranking for reordering documents and producing summaries. In W. B. Croft, A. Moffat, C. J. van Rijsbergen, R. Wilkinson, and J. Zobel, editors, *SIGIR '98: Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, August 24-28 1998, Melbourne, Australia*, pages 335–336. ACM, 1998.
- [16] A. Chapman, E. Simperl, L. Koesten, G. Konstantinidis, L. Ibáñez, E. Kacprzak, and P. Groth. Dataset search: a survey. *VLDB J.*, 29(1):251–272, 2020.
- [17] W. Chen, J. Tian, Y. Li, H. He, and Y. Jin. De-confounded variational encoder-decoder for logical table-to-text generation. In C. Zong, F. Xia, W. Li, and R. Navigli, editors, *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL/IJCNLP 2021, (Volume 1: Long Papers), Virtual Event, August 1-6, 2021*, pages 5532–5542. Association for Computational Linguistics, 2021.
- [18] DeepSeek. Deepseek models. <https://www.deepseek.com/>. accessed on 04.03.2025, Copyright 2025.
- [19] S. C. Deerwester, S. T. Dumais, T. K. Landauer, G. W. Furnas, and R. A. Harshman. Indexing by latent semantic analysis. *Journal of the American society for information science*, 41(6):391–407, 1990.
- [20] H. Dong, F. K. Hussain, and E. Chang. A survey in semantic search technologies. In *2008 2nd IEEE international conference on digital ecosystems and technologies*, pages 403–408. IEEE, 2008.
- [21] R. Dong, J. Liu, Y. Zhu, C. Yan, B. Mozafari, and X. Wang. Slabcity: Whole-query optimization using program synthesis. *Proc. VLDB Endow.*, 16(11):3151–3164, 2023.
- [22] M. Ester, H. Kriegel, J. Sander, and X. Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In E. Simoudis, J. Han, and U. M. Fayyad, editors, *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96), Portland, Oregon, USA*, pages 226–231. AAAI Press, 1996.
- [23] J. Evermann. Theories of meaning in schema matching: A review. *J. Database Manag.*, 19(3):55–82, 2008.
- [24] C. Fellbaum. Wordnet: An electronic lexical database. *Cambridge, MA: MIT Press*, 2:678–686, 1998.
- [25] D. Foundation. Duckdb database system. <https://duckdb.org/>. accessed on 03.03.2025, Copyright 2025.

- [26] P. S. Foundation. Python. <https://www.python.org/>. accessed on 04.03.2025, Copyright 2001-2025.
- [27] P. S. Foundation. Python difflib library. <https://docs.python.org/3/library/difflib.html>. accessed on 04.02.2025, Copyright 2001-2025.
- [28] H. Fu, C. Liu, B. Wu, F. Li, J. Tan, and J. Sun. Catsql: Towards real world natural language to SQL applications. *Proc. VLDB Endow.*, 16(6):1534–1547, 2023.
- [29] R. A. Ganski and H. K. T. Wong. Optimization of nested SQL queries revisited. In U. Dayal and I. L. Traiger, editors, *Proceedings of the Association for Computing Machinery Special Interest Group on Management of Data 1987 Annual Conference, San Francisco, CA, USA, May 27-29, 1987*, pages 23–33. ACM Press, 1987.
- [30] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou. Text-to-sql empowered by large language models: A benchmark evaluation. *CoRR*, abs/2308.15363, 2023.
- [31] H. Gong, Y. Sun, X. Feng, B. Qin, W. Bi, X. Liu, and T. Liu. Tablegpt: Few-shot table-to-text generation with table structure reconstruction and content matching. In D. Scott, N. Bel, and C. Zong, editors, *Proceedings of the 28th International Conference on Computational Linguistics, COLING 2020, Barcelona, Spain (Online), December 8-13, 2020*, pages 1978–1988. International Committee on Computational Linguistics, 2020.
- [32] Google. Gemini models. <https://ai.google.dev/gemini-api/docs/models/gemini>. accessed on 14.02.2025.
- [33] Google. Google dataset search. <https://datasetsearch.research.google.com/>. accessed on 11.02.2025.
- [34] J. Guo, X. Cheng, G. Xu, and X. Zhu. Intent-aware query similarity. In C. Macdonald, I. Ounis, and I. Ruthven, editors, *Proceedings of the 20th ACM Conference on Information and Knowledge Management, CIKM 2011, Glasgow, United Kingdom, October 24-28, 2011*, pages 259–268. ACM, 2011.
- [35] M. Honnibal, I. Montani, S. Van Landeghem, A. Boyd, et al. spacy: Industrial-strength natural language processing in python. <https://spacy.io/>, 2020. accessed on 04.02.2025, Copyright 2016-2025.
- [36] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Trans. Inf. Syst.*, 43(2), Jan. 2025.
- [37] HuggingFace. Hugging face models. <https://huggingface.co/models>. accessed on 17.02.2025.
- [38] Y. E. Ioannidis. Query optimization. *ACM Comput. Surv.*, 28(1):121–123, 1996.
- [39] P. Jaccard. The distribution of the flora in the alpine zone. 1. *New phytologist*, 11:37–50, 1912.

- [40] Kaggle. Kaggle datasets. <https://www.kaggle.com/datasets>. accessed on 11.02.2025.
- [41] A. Kemper and A. Eickler. *Datenbanksysteme - Eine Einführung, 8. Auflage*. Oldenbourg, 2011.
- [42] H. Kim, B. So, W. Han, and H. Lee. Natural language to SQL: where are we today? *Proc. VLDB Endow.*, 13(10):1737–1750, 2020.
- [43] L. Köberlein, D. Probst, and R. Lenz. Quantifying semantic query similarity for automated linear SQL grading: A graph-based approach. *CoRR*, abs/2403.14441, 2024.
- [44] G. Kul, D. T. A. Luong, T. Xie, V. Chandola, O. Kennedy, and S. J. Upadhyaya. Similarity metrics for SQL query clustering. *IEEE Trans. Knowl. Data Eng.*, 30(12):2408–2420, 2018.
- [45] F. Lei, J. Chen, Y. Ye, R. Cao, D. Shin, H. Su, Z. Suo, H. Gao, W. Hu, P. Yin, V. Zhong, C. Xiong, R. Sun, Q. Liu, S. I. Wang, and T. Yu. Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows. *CoRR*, abs/2411.07763, 2024.
- [46] V. I. Levenshtein et al. Binary codes capable of correcting deletions, insertions, and reversals. In *Soviet physics doklady*, volume 10, pages 707–710. Soviet Union, 1966.
- [47] B. Li, Y. Luo, C. Chai, G. Li, and N. Tang. The dawn of natural language to SQL: are we fully ready? *CoRR*, abs/2406.01265, 2024.
- [48] Z. Li and T. Xie. Using LLM to select the right SQL query from candidates. *CoRR*, abs/2401.02115, 2024.
- [49] J. Liu and B. Mozafari. Query rewriting via large language models. *CoRR*, abs/2403.09060, 2024.
- [50] T. Liu. Learning to rank for information retrieval. *Found. Trends Inf. Retr.*, 3(3):225–331, 2009.
- [51] T. Liu, K. Wang, L. Sha, B. Chang, and Z. Sui. Table-to-text generation by structure-aware seq2seq learning. In S. A. McIlraith and K. Q. Weinberger, editors, *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence, (AAAI-18), the 30th innovative Applications of Artificial Intelligence (IAAI-18), and the 8th AAAI Symposium on Educational Advances in Artificial Intelligence (EAAI-18), New Orleans, Louisiana, USA, February 2-7, 2018*, pages 4881–4888. AAAI Press, 2018.
- [52] S. P. Lloyd. Least squares quantization in PCM. *IEEE Trans. Inf. Theory*, 28(2):129–136, 1982.
- [53] M. Lo, K. Wu, and P. S. Yu. Tabsum: A flexible and dynamic table summarization approach. In *Proceedings of the 20th International Conference on Distributed Computing Systems, Taipei, Taiwan, April 10-13, 2000*, pages 628–635. IEEE Computer Society, 2000.
- [54] X. Ma, X. Zhang, R. Pradeep, and J. Lin. Zero-shot listwise document reranking with a large language model. *CoRR*, abs/2305.02156, 2023.

- [55] Meta. Llama models. <https://www.llama.com/docs/model-cards-and-prompt-formats/>. accessed on 14.02.2025.
- [56] G. A. Miller. Wordnet: A lexical database for english. *Commun. ACM*, 38(11):39–41, 1995.
- [57] MistralAI. Mistral models. https://docs.mistral.ai/getting-started/models/models_overview/. accessed on 14.02.2025, Copyright 2025.
- [58] L. Nan, Y. Zhao, W. Zou, N. Ri, J. Tae, E. Zhang, A. Cohan, and D. Radev. Enhancing few-shot text-to-sql capabilities of large language models: A study on prompt design strategies. *CoRR*, abs/2305.12586, 2023.
- [59] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Barnes, and A. Mian. A comprehensive overview of large language models. *CoRR*, abs/2307.06435, 2023.
- [60] OpenAI. Chatgpt. <https://chatgpt.com/>. accessed on 14.02.2025.
- [61] OpenAI. Openai api reference chat documentation. <https://platform.openai.com/docs/api-reference/chat/create>. accessed on 04.03.2025.
- [62] OpenAI. Openai api reference documentation. <https://platform.openai.com/docs/api-reference/introduction>. accessed on 04.03.2025.
- [63] OpenAI. Openai detailed pricing. <https://platform.openai.com/docs/pricing>. accessed on 04.03.2025.
- [64] OpenAI. Openai models. <https://platform.openai.com/docs/models>. accessed on 14.02.2025.
- [65] OpenAI. Openai models gpt-4o. <https://platform.openai.com/docs/models/gpt-4o#gpt-4o>. accessed on 04.02.2025.
- [66] OpenAI. Openai overview. <https://openai.com/>. accessed on 14.02.2025, Copyright 2015-2025.
- [67] OpenAI. Openai pricing. <https://openai.com/api/pricing/>. accessed on 14.02.2025, Copyright 2015-2025.
- [68] OpenAI. Openai tokenizer. <https://platform.openai.com/tokenizer>. checked on 21.02.2025.
- [69] OpenAI. Openai tokens. <https://platform.openai.com/docs/concepts/tokens>. accessed on 14.02.2025.
- [70] C. Ordonez. Optimization of linear recursive queries in SQL. *IEEE Trans. Knowl. Data Eng.*, 22(2):264–277, 2010.
- [71] M. F. Porter. An algorithm for suffix stripping. *Program*, 14(3):130–137, 1980.
- [72] M. Pourreza and D. Rafiei. DIN-SQL: decomposed in-context learning of text-to-sql with self-correction. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information*

Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023, 2023.

- [73] N. Project. Nltk porter stemmer module. <https://www.nltk.org/api/nltk.stem.porter.html>. accessed on 07.02.2025, Copyright 2024.
- [74] D. R. Radev. Spider website. <https://yale-lily.github.io/spider>. accessed on 19.02.2025, data downloaded in November 2024.
- [75] N. Reimers and I. Gurevych. Sbert overview. <https://www.sbert.net/>. accessed on 06.02.2025, Copyright 2025.
- [76] N. Reimers and I. Gurevych. Sbert semantic search models. https://www.sbert.net/docs/sentence_transformer/pretrained_models.html#semantic-search-models. accessed on 06.02.2025, Copyright 2025.
- [77] N. Reimers and I. Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In K. Inui, J. Jiang, V. Ng, and X. Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019, Hong Kong, China, November 3-7, 2019*, pages 3980–3990. Association for Computational Linguistics, 2019.
- [78] scikit learn. Python scikit-learn package. <https://scikit-learn.org/stable/index.html>. accessed on 03.03.2025.
- [79] M. Shanahan. Talking about large language models. *Commun. ACM*, 67(2):68–79, 2024.
- [80] M. A. Silva-Fuentes, H. D. Calderon-Vilca, E. F. Calderon-Vilca, and F. C. Cárdenas-Mariño. Semantic search system using word embeddings for query expansion. In *2019 IEEE PES Innovative Smart Grid Technologies Conference-Latin America (ISGT Latin America)*, pages 1–6. IEEE, 2019.
- [81] spaCy. English spacy models. <https://spacy.io/models/en>. accessed on 03.02.2025, Copyright 2016-2025.
- [82] spaCy. Large english spacy models. https://spacy.io/models/en/#en_core_web_lg. accessed on 03.02.2025, Copyright 2016-2025.
- [83] K. Srinivasan, K. Raman, A. Samanta, L. Liao, L. Bertelli, and M. Bendersky. QUILL: query intent with large language models using retrieval augmentation and multi-stage distillation. *CoRR*, abs/2210.15718, 2022.
- [84] W. Sun, L. Yan, X. Ma, S. Wang, P. Ren, Z. Chen, D. Yin, and Z. Ren. Is chatgpt good at search? investigating large language models as re-ranking agents. In H. Bouamor, J. Pino, and K. Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pages 14918–14937. Association for Computational Linguistics, 2023.

- [85] Z. Wang, Z. Chu, T. V. Doan, S. Ni, M. Yang, and W. Zhang. History, development, and principles of large language models: an introductory survey. *AI and Ethics*, pages 1–17, 2024.
- [86] A. C. Weigand, D. Lange, and M. Rauschenberger. How can small data sets be clustered? In C. Wienrich, P. Wintersberger, and B. Weyers, editors, *Mensch und Computer 2021 - Workshopband, Ingolstadt, Germany, September 5-8, 2021*. Gesellschaft für Informatik e.V., 2021.
- [87] F. Ye, M. Fang, S. Li, and E. Yilmaz. Enhancing conversational search: Large language model-aided informative query rewriting. *CoRR*, abs/2310.09716, 2023.
- [88] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. R. Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. *CoRR*, abs/1809.08887, 2018.
- [89] R. Zhang, J. Guo, Y. Fan, Y. Lan, and X. Cheng. Query understanding via intent description generation. In M. d’Aquin, S. Dietze, C. Hauff, E. Curry, and P. Cudré-Mauroux, editors, *CIKM ’20: The 29th ACM International Conference on Information and Knowledge Management, Virtual Event, Ireland, October 19-23, 2020*, pages 1823–1832. ACM, 2020.
- [90] S. Zhang, Z. Dai, K. Balog, and J. Callan. Summarizing and exploring tabular data in conversational search. In J. X. Huang, Y. Chang, X. Cheng, J. Kamps, V. Murdock, J. Wen, and Y. Liu, editors, *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval, SIGIR 2020, Virtual Event, China, July 25-30, 2020*, pages 1537–1540. ACM, 2020.
- [91] T. Zhang, X. Yue, Y. Li, and H. Sun. Tablellama: Towards open large generalist models for tables. In K. Duh, H. Gómez-Adorno, and S. Bethard, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), NAACL 2024, Mexico City, Mexico, June 16-21, 2024*, pages 6024–6044. Association for Computational Linguistics, 2024.
- [92] X. Zhou, G. Li, J. Wu, J. Liu, Z. Sun, and X. Zhang. A learned query rewrite system. *Proc.VLDB Endow.*, 16(12):4110–4113, 2023.
- [93] Y. Zhu, H. Yuan, S. Wang, J. Liu, W. Liu, C. Deng, Z. Dou, and J. Wen. Large language models for information retrieval: A survey. *CoRR*, abs/2308.07107, 2023.

Appendix A

LLM Prompts

All parts written in curly brackets (“{ }”) are parameters that are replaced by actual values in the final prompts.

A.1 Simple Table Prefilter

```
1 I want you to decide which from the given tables are useful to answer
  a given query.
2 It is important that the chosen tables can help to answer the query
  either directly or indirectly.
3 If a table cannot be used directly, think about if it can be used due
  to it being correlated regarding human intuition.
4 E.g. tables on park locations and crime rates could help to know areas
  with high rent.
5 So in conclusion, the tables should be usable to answer the query such
  that the information gain stays the same for a human.
6 So synonyms, hyponyms, correlations and similar topics should be
  important when choosing the usable tables.
7 I have the following SQL query:
8 {query}
9 Here are tables from my database (just the names):
10 {name_list}
11 If none of these tables are usable, then only respond with 'No tables
   usable'.
12 If some tables are usable, only respond with the names, separated by
   semicolons.
```

Optional (regarding table prefixes):

```
1 When choosing tables, please also take into account that tables from
  the same schema are prefixed with the same string (the name of
  the schema).
2 Such tables could be joined and used together to answer the query.
```

A.2 Complex Table Prefilter

```
1 I want you to give me tables that are usable to answer the SQL query I
  give you.
2 These tables should include synonyms and hyponyms of the tables in the
  query, but also tables that could have correlated values to those
  needed in the query.
3 Such tables could maybe not directly answer the query, but keep the
  same information gain for a human.
4 An example for this is that e.g. school and park locations influence
  housing prices.
```

```

5 Other important tables could be ones that can be joined to produce the
   needed answer together.
6 So incorporate all of these options in your answer and give me all the
   tables you can think of.
7 I have the following SQL query:
8 {input_query}
9 Only respond with possible tables separated with semicolons.
10 If you think there exist no tables to answer the query only respond
    with 'No tables usable'.

```

A.3 Table Prefilter Summaries

```

1 Given the following CREATE TABLE statements in SQL:
2 {statements}
3 I want you to do three things:
4 1. Summarize
5 Generate a summary to describe the table with maximal {max_tokens}
   tokens regarding the information need it satisfies.
6 Think about how you would describe an entity in this table and use
   this as a guideline for the table description.
7 Only give the description in natural language sentences, mimicking the
   human understanding of the table.
8 Do not describe the table column by column. Be concise.
9 2. Topics
10 Give me {num_topics} topics that fit the information need of the
   table, i.e., what a human would see as topics in the table.
11 Do not just repeat the query tables and columns themselves as topics,
   but abstract from them. Be concise.
12 3. Keywords
13 Give me {num_keywords} keywords that fit the information need of the
   table, i.e., what a human would see as keywords in the table.
14 Do not just repeat the query tables and columns themselves as
   keywords, but abstract from them. Be concise.
15 I want you to give me these three things in the following output
   format (adapting the number of topics and keywords to the above
   given numbers):
16 table_name[summary;topic1,topic2;keyword1,keyword2]
17 Return exactly this format for each given table of the CREATE
   statements in the same order as the CREATE statements given to
   you.
18 Separate the outputs for the tables by using a new line for each
   output.

```

A.4 Finding Metadata

Intent:

```

1 I have the following query in SQL:
2 {input_request}
3 Give me the information need of the query, i.e., what a humans intent
   is when writing the query.

```

- 4 Do not just repeat the query itself as an intent, but abstract from it. Be concise.
- 5 Only return the intent preceded by the keyword 'Intent:'.

Intents of multiple queries:

- 1 I have the following queries in SQL:
- 2 {input_requests}
- 3 For each query give me the information need of the query, i.e., what a humans intent is when writing the query.
- 4 Do not just repeat the query itself as an intent, but abstract from it. Be concise.
- 5 For each query only return the intent.
- 6 Give the intents in the same order as the input queries.
- 7 Separate the intents of the single queries with a new line each.

Keywords:

- 1 I have the following query in SQL:
- 2 {input_request}
- 3 Give me {num_keywords} keywords that fit the information need of the query, i.e., what a humans intent is when writing the query.
- 4 Do not just repeat the query tables and columns themselves as keywords, but abstract from them. Be concise.
- 5 Only return the keywords separated with commas.

Topics:

- 1 I have the following query in SQL:
- 2 {input_request}
- 3 Give me {num_topics} topics that fit the information need of the query, i.e., what a humans intent is when writing the query.
- 4 Do not just repeat the query tables and columns themselves as topics, but abstract from them. Be concise.
- 5 Only return the topics separated with commas.

A.5 Simple Rewriting

- 1 I have the following SQL query:
- 2 {query}
- 3 I do not have access to the tables needed in the query.
- 4 I do have the following tables in my database (written in the format:
- 5 table: column1 type1, column2 type2 \n Foreign keys (if existent)):
- 6 {filtered_tables}
- 7 Keep the foreign keys in mind if you join any tables.
- 8 I want to have queries that keep the same human intent and satisfy the information need as the given query.
- 9 These new queries should use the provided tables and columns from my database.
- 10 Give me {number_of_alternatives} alternative queries. They should be as diverse as possible.
- 11 Only give the SQL queries and nothing more. Use a semicolon to separate the queries.

A.6 Complex Rewriting

```
1 I have the following request deducted from a SQL query:
2 '{intent}'
3 I have the following tables in my database (written in the format:
4 table: column1 type1, column2 type2 \n Foreign keys (if existent)):
5 {filtered_tables}
6 I want to have SQL queries that have the same human intent and satisfy
   the same information need as the given request.
7 These new queries should use the provided tables and columns from my
   database.
8 Keep the foreign keys in mind if you join any tables.
9 Give me {number_of_alternatives} alternative queries. They should be
   as diverse as possible.
10 Only give the SQL queries and nothing more. Use a semicolon to
    separate the queries.
```

A.7 Ranking

```
1 I will give you a single SQL query called original query.
2 I will also give you multiple other SQL queries called alternative
   queries.
3 I want you to rank the alternative queries according to their
   similarity to the original query (the highest similarity should
   be at the top).
4 The similarity of two SQL queries in this context is defined as
   follows:
5 Two queries are similar if the insight they give to a human being when
   executed on a database is similar.
6 I.e. the intents of the two queries are similar from a human
   perspective.
7 So to get the similarity think step-by-step:
8
9 1. What is the intent from the alternative query when abstracting from
   the exact SQL?
10 Think about the intent the user has in mind.
11 2. Which similarity value would this alternative query get to the
   original one?
12 Take into consideration that queries that have results that somehow
   correlate with the original query should have a high similarity.
13 3. Sort the alternative queries based on this rank.
14 The ranking should also be heterogeneous, so if multiple queries
   are really similar based on their strings, you might reorder
   them, such that they are not directly after each other.
15
16 Give back nothing else but all the alternative queries sorted by the
   ranking and separated with new lines.
17 Here is the original query:
18 {input_query}
19 Here are the alternative queries (separated by semicolons):
20 {alternative_queries}
```

A.8 Similarity Function

```
1 I will give you a single SQL query called original query.
2 I will also give you multiple other SQL queries called alternative
  queries.
3 For each of the alternative queries, I want to calculate its
  similarity to the original query.
4 These similarities should be floating point values between 0 and 1,
  and you should use the following guidelines to appoint them:
5 The similarity between two queries is solely based on their intent.
6 If the expected result of the queries gives the same insight to a
  human being, their similarity should be 1.
7 This can include results that are correlated (e.g. rent and crime rate
  in a city) or that are virtually the same.
8 If the given insight is similar, but not the same, the value should be
  a floating point number between 0 and 1, depending on how high
  the similarity is.
9 If the query intents do not have anything to do with each other, the
  similarity should be 0.
10 For each given alternative query only return the assigned similarity
    value between 0 and 1.
11 Separate the similarity values using semicolons.
12 Here is the original query:
13 {input_query}
14 Here are the alternative queries:
15 {alternative_queries}
16 Give me only the similarity values separated with semicolons.
```

A.9 Self Correction

```
1 For the given SQL query, use the provided tables and columns to fix it
  for the problems arising during its execution in DuckDB.
2 Only change parts of the query that need correction and keep the
  intent of the query when correcting it.
3 Use the following error message as a hint to identify the issue with
  the query:
4 {error_msg}
5 I have the following query in SQL:
6 {input_query}
7 I have the following tables in my database (format: table1: column1
  type1, column2 type2 \n Foreign keys (if existent)):
8 {filtered_tables}
9 Keep the foreign keys in mind if the query joins any tables.
10 You should provide only a corrected version of the query that is
    executable on the database, nothing else.
```

Appendix B

User Study Instructions

You will get some files with SQL queries in them.

The first line of each file starts with a '#'. Following this is the original SQL query.

All the other queries in the file are queries produced by a rewriter as alternative queries to this original query.

The goal of the rewriter is to produce queries that are not necessarily equivalent in the traditional sense, but satisfy the same information need as the original query. This can include queries with table names that are synonyms to the original query, but also results that can be deduced from other correlated columns. Some examples include:

- Calculating a column that is wanted in the original query using other columns from a related table (e.g., using sales and cost to calculate profit)
- Using tables and columns that are in the human eye correlated to get the result (e.g., rent prices correlating with crime rate, nearby infrastructure,...)
- Using tables and columns that are synonyms to the ones in the original query (e.g., employee and worker)

You are not given any information on the schema for the tables and columns. Your job is to rate the queries in the document compared to the original query. If an alternative query satisfies the same information need as the original query, please mark it either by putting a tab directly in front of it OR an 'x' followed by a comma (in the same line as the query). Otherwise, just leave the line with the query as is.

Here is an example for relevant/irrelevant queries:

```
# SELECT name FROM company ORDER BY customerRanking DESC;
```

```
x, SELECT b.name FROM business b JOIN order o ON b.ID = o.companyID  
GROUP BY b.ID,b.name ORDER BY sum(o.review)/count(o.review) DESC;
```

```
SELECT name FROM employees ORDER BY salary DESC;
```

Appendix C

User Study Queries

The queries are separated by how they are categorized and numbered consistent with the descriptions and graphs in Chapter 5.

Easy:

1.
`SELECT best_team FROM baseball;`
2.
`SELECT count(*) FROM college_students;`
3.
`SELECT meeting_id, start_time, end_time, duration
FROM advertising_agencies_Meetings;`
4.
`SELECT full_student_name, pet_name
FROM students_and_pets
WHERE pet_type = 'Cat';`

Medium:

5.
`SELECT school_name, total_budget, budgeted_money, invested_money
FROM school_finance
WHERE year = '2024';`
6.
`SELECT book_name
FROM books
WHERE length > 500
ORDER BY customer_review DESC;`
7.
`SELECT full_actor_name
FROM actors
WHERE EXISTS (SELECT *
FROM film f JOIN actors_films a ON f.ID = a.film_ID
WHERE title LIKE '%study%' AND a.actorID = actors.ID);`

Hard:

8.

```
SELECT a.author_id, a.author_name, COUNT(b.book_id) as total_sales
FROM authors a JOIN book_orders b ON a.author_id = b.book_author
WHERE a.author_name LIKE 'M%' AND b.purchase_date > '2020-01-01'
GROUP BY a.author_id, a.author_name
ORDER BY total_sales DESC;
```

9.

```
SELECT district, attractiveness
FROM city
ORDER BY attractiveness DESC;
```

10.

```
SELECT Group_ID,
       RANK() OVER (PARTITION BY region ORDER BY popularity)
FROM drama_workshop_groups;
```

Eidesstattliche Erklärung

Ich erkläre hiermit an Eides statt, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe.

Statement in Lieu of an Oath

I hereby confirm that I have written this thesis on my own and that I have not used any other media or materials than the ones referred to in this thesis.

Kaiserslautern, 18.03.2025

Date



Signature